

C O D E R H O U S E  
M A T E R I A L D E L E C T U R A

# Programación Backend I

Guía de lectura del curso

# Contenido



|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>MÓDULO 1 • Principios Básicos de JavaScript y Backend</b> | <b>5</b>  |
| Introducción al Backend y JavaScript                         | 5         |
| Variables y Estructuras en JavaScript                        | 8         |
| Template Strings y Clases en JavaScript                      | 11        |
| Funciones y Scopes en JavaScript                             | 14        |
| Síntesis y Conclusiones                                      | 16        |
| <b>MÓDULO 2 • Programación Backend Avanzada</b>              | <b>19</b> |
| ES7 y ES8 – Sintaxis Moderna                                 | 19        |
| Simulación de Calculadora Positiva                           | 21        |
| ES9 – finally y operadores rest/spread                       | 24        |
| Sincronismo vs. Asincronismo                                 | 26        |
| Callbacks, Promesas y Async/Await                            | 28        |
| ES10 – Dinamicidad y optimización                            | 35        |
| Gestión de Eventos en JavaScript                             | 37        |
| ES11 – Nullish y variables privadas                          | 39        |
| <b>MÓDULO 3 • Desarrollo Avanzado de Backend con Node.js</b> | <b>42</b> |
| Breve repaso y mapa de módulos nativos                       | 42        |
| Conceptos de NPM y package.json                              | 44        |
| Git y GitHub: pasos esenciales                               | 47        |
| <b>MÓDULO 4 • Desarrollo de Backend Avanzado</b>             | <b>49</b> |
| Arquitectura Cliente-Servidor y REST (breve)                 | 49        |
| Códigos de estado HTTP: cuándo y por qué                     | 51        |
| Inicializar proyecto y gestionar dependencias (npm / pnpm)   | 52        |
| <b>MÓDULO 5 • Motores de Plantillas y Routers en Express</b> | <b>55</b> |
| Integración con vistas                                       | 55        |
| Introducción a motores de plantillas                         | 57        |
| Configuración de Handlebars                                  | 59        |
| Condicionales if                                             | 62        |
| Iteraciones each                                             | 65        |
| Renderizado dinámico de listas                               | 67        |

|                                                                                  |            |
|----------------------------------------------------------------------------------|------------|
| Configuración de Multer .....                                                    | 70         |
| Validación y manejo de errores .....                                             | 72         |
| Layouts y vistas dinámicas .....                                                 | 75         |
| Repositorio con Aplicación Express .....                                         | 78         |
| <b>MÓDULO 6 • Desarrollo de Aplicaciones en Tiempo Real con Websockets .....</b> | <b>81</b>  |
| Preparación del entorno y estructura del proyecto .....                          | 81         |
| ¿Qué son los WebSockets y para qué se utilizan? .....                            | 84         |
| Diferencias entre HTTP y WebSockets .....                                        | 85         |
| Creación del servidor WebSocket con Node.js y Socket.io .....                    | 87         |
| Implementación del cliente WebSocket y manejo de mensajes .....                  | 90         |
| Casos de uso y arquitectura cliente-servidor .....                               | 93         |
| <b>MÓDULO 7 • CRUD y Manejo de Bases de Datos con MongoDB .....</b>              | <b>95</b>  |
| Bases de datos NoSQL y modelo de documentos en MongoDB .....                     | 95         |
| Estructura de bases de datos, colecciones y documentos .....                     | 97         |
| Inserción y lectura de documentos en MongoDB .....                               | 99         |
| Consultas avanzadas con operadores y condiciones lógicas .....                   | 102        |
| Proyecciones y ordenamiento de resultados .....                                  | 104        |
| Consultas con filtros y operadores básicos .....                                 | 106        |
| Paginación y optimización de consultas .....                                     | 108        |
| Actualización y eliminación de documentos .....                                  | 111        |
| <b>MÓDULO 8 • Desarrollo Avanzado de Backend con Node.js y MongoDB .....</b>     | <b>114</b> |
| Schemas, modelos y validaciones en Mongoose .....                                | 114        |
| Conceptos clave de Mongoose y ODM en MongoDB .....                               | 117        |
| Operaciones CRUD: Create y Read con Mongoose .....                               | 119        |
| Clientes de base de datos y DBaaS con MongoDB Atlas .....                        | 121        |
| Creación y configuración de un proyecto Node.js .....                            | 123        |
| Operaciones CRUD: Update y Delete con Mongoose .....                             | 126        |
| Instalación y configuración de Mongoose en Node.js .....                         | 128        |
| Estructura de rutas y flujo del CRUD en Node.js .....                            | 131        |
| <b>MÓDULO 9 • MongoDB Avanzado y Optimización de Consultas .....</b>             | <b>135</b> |
| Importancia de la indexación y su impacto en las consultas .....                 | 135        |
| Tipos de índices en MongoDB: compuestos, multikey, text y geoespaciales .....    | 137        |
| Situaciones clave donde conviene indexar .....                                   | 139        |

|                                                                                     |            |
|-------------------------------------------------------------------------------------|------------|
| Concepto de population en MongoDB y referencias entre colecciones .....             | 142        |
| Buenas prácticas y riesgos del populate (profundidad, circularidad) .....           | 143        |
| Full collection scan vs consultas optimizadas con índices .....                     | 145        |
| Middlewares pre en Mongoose para automatizar populate .....                         | 148        |
| Aggregations en MongoDB y etapas del pipeline (\$match, \$group, etc.) .....        | 150        |
| <b>MÓDULO 10 • Proyecto Final .....</b>                                             | <b>153</b> |
| <b>ENTREGA FINAL Evaluación Final: Proyecto backend con Node.js y Express .....</b> | <b>154</b> |
| <b>MÓDULO • APÉNDICE — Material complementario y cierre .....</b>                   | <b>161</b> |
| Palabras clave .....                                                                | 161        |
| Análisis profundo .....                                                             | 162        |
| Buenas prácticas y ejemplos .....                                                   | 162        |
| Ejemplo y consideraciones .....                                                     | 163        |
| Ejemplo práctico .....                                                              | 164        |
| Herramientas y ejemplo .....                                                        | 165        |
| Recomendaciones prácticas .....                                                     | 166        |
| Preguntas frecuentes .....                                                          | 166        |
| Conclusiones .....                                                                  | 168        |
| Aplicación inmediata .....                                                          | 168        |
| Crecimiento en el puesto .....                                                      | 169        |

# Principios Básicos de JavaScript y Backend

Comienza tu camino en el desarrollo backend entendiendo qué es el backend y cómo se relaciona con el frontend. En este módulo introductorio, descubrirás las tecnologías que conforman el stack MERN y el rol que desempeña cada una en el flujo de una aplicación web. Aprenderás a ejecutar código JavaScript en Node.js desde la terminal, sentando las bases para manipular variables y estructuras en módulos posteriores. Este conocimiento inicial te permitirá comprender el ecosistema backend y prepararte para construir aplicaciones completas y funcionales.

## Introducción al Backend y JavaScript

En esta unidad exploraremos qué es el Backend y cómo se relaciona con el Frontend, conoceremos el stack MERN y el rol de cada tecnología que lo compone. Además, aprenderemos a ejecutar JavaScript en Node.js desde la terminal y revisaremos los tipos de datos básicos en JavaScript, sentando las bases para el desarrollo backend y full stack.

Antes de empezar a programar, es clave entender **qué problema estamos resolviendo**.

Cuando usás una app como Instagram, Mercado Libre o Netflix, pasan muchas cosas “por detrás” que no ves:

- se guardan tus datos
- se validan usuarios
- se procesan pedidos
- se envía información al instante

Todo eso ocurre en el **Backend**.

### Una forma simple de entenderlo

Podemos pensar una aplicación como un restaurante:

- **Backend** → la cocina (donde se prepara todo)
- **Frontend** → el mozo (lo que ves e interactuás)
- **Usuario** → vos, haciendo el pedido

Vos pedís algo → el mozo lleva el pedido → la cocina lo prepara → vuelve la respuesta

Ese “ida y vuelta” es exactamente cómo funcionan las aplicaciones web.

### ¿Qué vas a aprender en esta unidad?

En esta unidad vas a:

- Entender qué es el **Backend** y para qué sirve

- Ver cómo se conecta con el **Frontend**
- Conocer el stack **MERN**
- Ejecutar tu **primer programa en Node.js**
- Entender cómo fluye una aplicación completa

No necesitas experiencia previa en backend.

## ¿Qué es el Backend y cómo se relaciona con el Frontend?

El **Backend** es la parte de una aplicación que corre en el servidor y se encarga de la lógica de negocio, el almacenamiento de datos y la comunicación con el cliente (el Frontend). Mientras que el Frontend es la interfaz que el usuario ve y con la que interactúa, el Backend procesa las solicitudes, maneja la información y envía respuestas.

### Ejemplo

Cuando inicias sesión en una red social, el Frontend muestra el formulario, pero el Backend verifica tus credenciales y recupera tus datos.

## ¿Qué es un stack de tecnologías?

Un **stack** es un conjunto de tecnologías que trabajan juntas para construir una aplicación completa. Cada tecnología tiene un rol específico, desde la base de datos hasta la interfaz de usuario.

### Stack MERN

El stack MERN usa **JavaScript en todo**:

| Tecnología | Rol           | Explicación simple          |
|------------|---------------|-----------------------------|
| MongoDB    | Base de datos | Guarda la información       |
| Express    | Backend       | Maneja las rutas            |
| React      | Frontend      | Lo que ve el usuario        |
| Node.js    | Entorno       | Permite usar JS en servidor |

### Flujo completo (muy importante)

1. El usuario hace clic (React)
2. Se envía una petición (Express + Node)
3. Se consulta la base de datos (MongoDB)
4. Se devuelve la respuesta
5. React muestra el resultado

Este flujo lo vas a repetir TODO el curso.

## ¿Por qué es tan popular?

- Usás **un solo lenguaje (JavaScript)**
- Es rápido de aprender
- Tiene muchísima comunidad
- Se usa en proyectos reales

## JavaScript en Backend

Hasta ahora probablemente usabas JavaScript en el navegador.

Con Node.js ahora podés usarlo para:

- crear servidores
- manejar datos
- construir APIs

## Tipos de datos

Antes de programar, necesitás entender qué tipo de datos vas a manejar.

### Primitivos

#### CÓDIGO

```
"Hola" // string
42 // number
true // boolean
null // sin valor
undefined // no definido
```

### Objetos

#### CÓDIGO

```
const usuario = {
  nombre: "Ana",
  edad: 25
};
```

### Arrays

#### CÓDIGO

```
const numeros = [1, 2, 3];
```

Esto lo vas a usar **TODO** el tiempo en backend.

## ¿Qué es Node.js?

Node.js permite ejecutar JavaScript **fuera del navegador**.

Es lo que usamos para crear backend.

Antes:

CÓDIGO

```
JavaScript → solo en el navegador
```

Ahora:

CÓDIGO

```
JavaScript → también en el servidor (Node.js)
```

En el desarrollo backend, entender cómo funciona el Backend y cómo se conecta con el Frontend es fundamental para crear aplicaciones completas y funcionales. El stack MERN es una opción muy popular en la industria por su uso exclusivo de JavaScript, lo que facilita el aprendizaje y la integración.

Saber ejecutar código JavaScript en Node.js desde la terminal es el primer paso para trabajar con servidores y desarrollar aplicaciones backend. Además, conocer los tipos de datos en JavaScript te permitirá manejar la información correctamente y prepararás el terreno para trabajar con variables, funciones y estructuras más complejas en las siguientes unidades.

Esta unidad te prepara para entender el flujo completo de una aplicación web y para comenzar a programar en un entorno backend real, conectando teoría con práctica desde el inicio.

## Variables y Estructuras en JavaScript

En esta unidad aprenderás a declarar variables usando `let` y `const`, entenderás por qué evitar `var` en código moderno y conocerás los diferentes scopes que determinan la accesibilidad de las variables. Además, te introducirás a las novedades de ES6, como las arrow functions y mejoras de sintaxis, con ejemplos prácticos para backend en Node.js.

¿Alguna vez te has preguntado por qué en JavaScript existen diferentes formas de declarar variables y cómo esto afecta el comportamiento de tu código? En el desarrollo backend, entender cómo y dónde se almacenan tus variables es fundamental para evitar errores y escribir código limpio y eficiente. En esta unidad, exploraremos las formas modernas de declarar variables con `let` y `const`, por qué es recomendable evitar `var`, y cómo el concepto de `scope` o alcance determina la visibilidad y vida útil de estas variables. Además, conocerás las novedades que trajo ES6, como las arrow functions, que facilitan la escritura de funciones en JavaScript. A través de ejemplos prácticos que podrás ejecutar en Node.js, consolidarás estos conceptos clave para tu camino como desarrollador backend.



## Declaración de Variables en JavaScript

En JavaScript, existen tres formas principales de declarar variables: `var`, `let` y `const`. Cada una tiene características y comportamientos diferentes que afectan cómo y dónde puedes usar esas variables.

### `var`

- Tiene **scope de función**, es decir, la variable es accesible dentro de la función donde se declara.
- Sufre de **hoisting**, lo que significa que su declaración se mueve al inicio de la función, aunque su asignación no.
- Puede generar errores difíciles de detectar debido a su alcance y hoisting.

Ejemplo de hoisting con `var`:

#### CÓDIGO

```
console.log(miVar); // undefined
var miVar = 10;
```

### `let` y `const`

- Introducidos en ES6, tienen **scope de bloque**, lo que significa que solo existen dentro del bloque `{}` donde se declaran.
- No sufren hoisting como `var`.
- `let` permite reasignar valores, mientras que `const` declara constantes que no pueden cambiar.

Ejemplo con `let` y `const`:

#### CÓDIGO

```
{
  let edad = 25;
  const nombre = 'Ana';
  edad = 26; // válido
  // nombre = 'Luis'; // Error: asignación a constante
}
// edad y nombre no existen aquí
```

### ¿Por qué evitar `var`?

- Su comportamiento puede causar bugs difíciles de detectar.
- `let` y `const` ofrecen un control más claro y seguro del alcance de las variables.
- El código moderno y profesional prefiere `let` y `const` para mayor legibilidad y mantenimiento.

## Concepto de Scope (Alcance)

El *scope* determina dónde una variable es accesible dentro del código.

- **Scope global:** Variables accesibles en todo el programa.
- **Scope de función:** Variables accesibles solo dentro de la función donde se declaran.
- **Scope de bloque:** Variables accesibles solo dentro del bloque `{ }` donde se declaran (introducido con `let` y `const`).

### Ejemplo

#### CÓDIGO

```
let globalVar = 'Soy global';
function saludar() {
  let funcionVar = 'Soy local a la función';
  if(true) {
    let bloqueVar = 'Soy local al bloque';
    console.log(globalVar); // Accede a global
    console.log(funcionVar); // Accede a función
    console.log(bloqueVar); // Accede a bloque
  }
  // console.log(bloqueVar); // Error: no existe fuera del bloque
}
saludar();
// console.log(funcionVar); // Error: no existe fuera de la función
```

## Introducción a ECMAScript y ES6

ECMAScript es el estándar que define cómo debe funcionar JavaScript. ES6 (o ECMAScript 2015) fue una versión clave que introdujo mejoras importantes para escribir código más limpio y eficiente.

### Novedades de ES6 relevantes para Backend

- **Arrow functions:** Sintaxis más corta para funciones, con manejo especial del contexto `this`.
- **Template strings:** Cadenas de texto que permiten interpolar variables fácilmente.
- `let` y `const`: Declaración de variables con scope de bloque.
- **Clases:** Sintaxis para programación orientada a objetos más clara.

### Ejemplo

#### CÓDIGO

```
const sumar = (a, b) => a + b;
console.log(sumar(2, 3)); // 5
```

Estas novedades facilitan la escritura de código backend moderno, más legible y menos propenso a errores.

En el desarrollo backend con Node.js, manejar correctamente las variables y su alcance es crucial para evitar problemas como fugas de memoria, errores de referencia y comportamientos inesperados. Por ejemplo, declarar variables con `var` en lugar de `let` o `const` puede causar que datos sensibles queden accesibles fuera del contexto esperado, lo que puede generar vulnerabilidades o bugs difíciles de depurar.

Además, las arrow functions y otras mejoras de ES6 permiten escribir funciones más concisas y claras, lo que mejora la mantenibilidad del código en proyectos backend donde la legibilidad es clave para el trabajo en equipo y la escalabilidad.

En esta unidad, los ejemplos prácticos que ejecutarás en Node.js te ayudarán a observar directamente cómo funcionan los scopes y las declaraciones de variables, preparándote para escribir código backend robusto y moderno.

## Template Strings y Clases en JavaScript

En esta unidad aprenderás a usar template strings para construir cadenas de texto más legibles y dinámicas, y a introducirte en la Programación Orientada a Objetos (POO) en JavaScript mediante la definición de clases, constructores, propiedades y métodos. También exploraremos cómo encapsular validaciones básicas y manejo de errores dentro de métodos de clase, preparando el camino para modelar entidades en aplicaciones backend.

¿Alguna vez has tenido que construir mensajes o logs en tu código que resultan difíciles de leer o mantener? Imagina poder escribir cadenas de texto que integren variables y expresiones de forma clara y sencilla, mejorando la legibilidad y facilitando el mantenimiento. En esta unidad, descubrirás cómo las **template strings** de JavaScript ES6 te permiten hacer justamente eso, usando una sintaxis moderna y práctica.

Además, darás tus primeros pasos en la **Programación Orientada a Objetos (POO)** en JavaScript, aprendiendo a definir clases, crear instancias y encapsular lógica dentro de métodos. Esto es fundamental para modelar entidades y comportamientos en aplicaciones backend, como usuarios, productos o procesos.

Al finalizar, serás capaz de construir mensajes claros con template strings, definir clases con sus propiedades y métodos, y aplicar validaciones básicas dentro de tus métodos para asegurar la integridad de los datos. ¡Vamos a comenzar!

### Template Strings en JavaScript

Las **template strings** son una forma moderna de construir cadenas de texto en JavaScript, introducidas en ES6. Se escriben usando **backticks** (```) en lugar de comillas simples o dobles.

## Sintaxis básica

### CÓDIGO

```
const nombre = 'Ana';
const edad = 28;
const mensaje = `Hola, mi nombre es ${nombre} y tengo ${edad} años.`;
console.log(mensaje);
// Salida: Hola, mi nombre es Ana y tengo 28 años.
```

**Ventajas:** - Permiten insertar variables y expresiones directamente dentro de la cadena usando `${}`. - Mejoran la legibilidad y evitan concatenaciones largas con `+`. - Soportan cadenas multilínea sin necesidad de caracteres especiales.

## Ejemplo con expresiones

### CÓDIGO

```
const a = 5;
const b = 10;
console.log(`La suma de ${a} y ${b} es ${a + b}.`);
// Salida: La suma de 5 y 10 es 15.
```

## Introducción a la Programación Orientada a Objetos (POO) en JavaScript

La POO es un paradigma que permite modelar entidades del mundo real como objetos con propiedades y comportamientos. En JavaScript, desde ES6, podemos definir **clases** que actúan como plantillas para crear objetos.

## Definición de una clase

### CÓDIGO

```
class Producto {
  constructor(nombre, precio) {
    this.nombre = nombre; // propiedad
    this.precio = precio; // propiedad
  }

  mostrarInfo() { // método
    return `Producto: ${this.nombre}, Precio: ${this.precio}`;
  }
}
```

- `constructor` es un método especial que se ejecuta al crear una instancia.
- `this` se refiere al objeto actual.

## Crear instancias

### CÓDIGO

```
const producto1 = new Producto('Camiseta', 20);
console.log(producto1.mostrarInfo());
// Salida: Producto: Camiseta, Precio: $20
```

## Buenas prácticas: Validaciones y manejo de errores

Es común que los métodos incluyan validaciones para asegurar que los datos sean correctos. Por ejemplo:

### CÓDIGO

```
class Usuario {
  constructor(nombre, edad) {
    this.nombre = nombre;
    this.edad = edad;
  }

  setEdad(nuevaEdad) {
    if (typeof nuevaEdad !== 'number' || nuevaEdad <= 0) {
      throw new Error('Edad inválida');
    }
    this.edad = nuevaEdad;
  }
}

try {
  const user = new Usuario('Luis', 30);
  user.setEdad(-5);
} catch (error) {
  console.error('Error:', error.message);
}
```

Aquí usamos `throw` para lanzar un error y `try/catch` para capturarlo y manejarlo, una práctica común en backend para controlar fallos y mantener la estabilidad.

En el desarrollo backend, es fundamental construir mensajes claros para logs, respuestas o errores que faciliten la comprensión y el mantenimiento del código. Las template strings permiten crear estos mensajes de forma sencilla y legible, evitando errores comunes en concatenaciones.

Por otro lado, la POO nos ayuda a organizar el código modelando entidades reales, como usuarios, productos o pedidos, encapsulando sus datos y comportamientos. Esto mejora la escalabilidad y mantenibilidad de las aplicaciones.

Además, incluir validaciones y manejo de errores dentro de los métodos de clase es clave para garantizar que los datos sean consistentes y que la aplicación pueda responder adecuadamente ante situaciones inesperadas, un aspecto crítico en sistemas backend donde la robustez es esencial.

Esta unidad te prepara para aplicar estos conceptos en la creación de componentes backend que sean claros, organizados y confiables, sentando las bases para el desarrollo de funcionalidades más complejas en módulos posteriores.

## Funciones y Scopes en JavaScript

Explora las funciones tradicionales y arrow functions en JavaScript, entendiendo el comportamiento de hoisting, el manejo del contexto `this` y la importancia del scope para evitar errores comunes en desarrollo backend. Aprende con ejemplos prácticos ejecutables en Node.js y consejos para debugging en consola.

¿Alguna vez te has preguntado por qué algunas funciones en JavaScript parecen "levantarse" antes de ser declaradas? Este fenómeno, conocido como *hoisting*, es clave para entender cómo el lenguaje ejecuta tu código. En este módulo, descubrirás las diferencias entre las funciones tradicionales y las arrow functions, dos formas esenciales de definir comportamiento en JavaScript. Además, exploraremos cómo el contexto de ejecución, especialmente el valor de `this`, cambia según el tipo de función y cómo el scope —el alcance de las variables y funciones— influye directamente en la estabilidad y seguridad de tus aplicaciones backend. A través de ejemplos prácticos en Node.js, aprenderás a escribir código más claro, evitar errores comunes y preparar tu camino hacia un desarrollo backend sólido y profesional.

### Funciones tradicionales y hoisting

En JavaScript, una *función tradicional* se declara con la palabra clave `function` y puede ser llamada antes de su definición gracias al **hoisting**. El hoisting es un comportamiento del motor de JavaScript que mueve las declaraciones de funciones y variables al inicio del contexto de ejecución.

#### CÓDIGO

```
// Ejemplo de hoisting con función tradicional
saludar(); // Funciona correctamente

function saludar() {
  console.log('¡Hola desde función tradicional!');
}
```

#### Nota

Solo las declaraciones de funciones completas son "hoisted", no las expresiones de función.

## Arrow functions: sintaxis y ventajas

Las *arrow functions* (`=>`) son una forma más concisa de escribir funciones, introducidas en ES6. No tienen su propio `this`, lo que las hace ideales para mantener el contexto léxico.

### CÓDIGO

```
const saludar = () => {  
  console.log('¡Hola desde arrow function!');  
};  
saludar();
```

Ventajas: - Sintaxis más corta - No crean su propio `this`, lo que evita confusiones en callbacks

## Diferencias clave entre funciones tradicionales y arrow functions

| Característica           | Función Tradicional               | Arrow Function                 |
|--------------------------|-----------------------------------|--------------------------------|
| Sintaxis                 | <code>function nombre() {}</code> | <code>() =&gt; {}</code>       |
| Hoisting                 | Sí                                | No                             |
| <code>this</code> propio | Sí                                | No (hereda del contexto padre) |
| Uso como constructor     | Sí                                | No                             |

## Scope: global y local

El *scope* determina la accesibilidad de variables y funciones. En JavaScript existen principalmente dos scopes:

- **Global:** Variables accesibles en todo el programa.
- **Local:** Variables accesibles solo dentro de una función o bloque.

### CÓDIGO

```
let globalVar = 'Soy global';  
  
function mostrar() {  
  let localVar = 'Soy local';  
  console.log(globalVar); // Accede a global  
  console.log(localVar); // Accede a local  
}  
  
mostrar();  
console.log(localVar); // Error: no está definida
```

### Importante

Usar scopes correctamente evita errores y fugas de memoria, especialmente en backend donde la gestión de estado es crítica.

En el desarrollo backend con Node.js, entender cómo funcionan las funciones y el scope es fundamental para escribir código eficiente y libre de errores. Por ejemplo, el hoisting puede causar comportamientos inesperados si no se conoce bien, y el manejo incorrecto de `this` puede generar bugs difíciles de detectar en callbacks o métodos de objetos.

Además, el scope ayuda a controlar la visibilidad de variables, evitando conflictos y mejorando la seguridad del código. En aplicaciones backend, donde múltiples procesos y usuarios interactúan, mantener un scope limpio y predecible es clave para la estabilidad.

Los ejemplos que veremos están diseñados para ejecutarse en Node.js, permitiéndote experimentar directamente en tu entorno de desarrollo. También incluiremos consejos para usar la consola y herramientas de debugging que te ayudarán a identificar y corregir problemas relacionados con funciones y scopes.

## Síntesis y Conclusiones

En esta unidad final, repasaremos y aplicaremos los conceptos clave de JavaScript para Backend que hemos aprendido a lo largo del programa. Integraremos la programación orientada a objetos, manejo de errores, pruebas básicas en Node.js y el uso de la terminal para ejecutar nuestros scripts. Además, te guiaremos para crear un entregable completo que demuestre tu capacidad para desarrollar y validar una clase `ProductManager` funcional, con código, pruebas y documentación clara para su ejecución en Node.js.

¿Te imaginas poder construir una clase en JavaScript que gestione productos, valide datos, maneje errores y además puedas probarla fácilmente desde la consola? En esta unidad final, sintetizaremos todo lo aprendido para que puedas diseñar y entregar un proyecto funcional que refleje tus habilidades en Backend con Node.js. Revisaremos cómo ejecutar tus scripts desde la terminal, cómo estructurar tu código con clases y métodos, y cómo validar y manejar errores para garantizar un backend robusto. Además, te guiaremos para crear un entregable completo con código, pruebas y documentación clara, listo para compartir y demostrar tu progreso. Prepárate para consolidar tu aprendizaje y dar un paso firme hacia el desarrollo backend profesional.



## Repaso de conceptos clave para Backend con JavaScript

### Node.js y la Terminal (CLI)

Node.js es un entorno que permite ejecutar JavaScript fuera del navegador, ideal para crear aplicaciones backend. La terminal o línea de comandos (CLI) es la herramienta para interactuar con Node.js, ejecutar scripts y gestionar proyectos.

- **Ejemplo:** Ejecutar un script con `node test.js` para ver resultados en consola.

### Programación Orientada a Objetos (POO) en JavaScript

Las clases en JavaScript permiten modelar entidades con propiedades y métodos.

- **Constructor:** Método especial para inicializar objetos.
- **Métodos:** Funciones que definen comportamientos.

#### CÓDIGO

```
class ProductManager {
  constructor() {
    this.products = [];
    this.nextId = 1;
  }

  addProduct(product) {
    product.id = this.nextId++;
    this.products.push(product);
  }
}
```

### Manejo de errores y validaciones

Para garantizar que nuestro backend sea confiable, validamos datos y manejamos errores con `try/catch`.

#### CÓDIGO

```
try {
  productManager.addProduct(invalidProduct);
} catch (error) {
  console.error('Error al agregar producto:', error.message);
}
```

## Pruebas básicas en Node.js

Ejecutar scripts de prueba en consola permite validar que nuestras funciones y clases funcionan correctamente.

- Mostrar resultados y errores en la consola ayuda a detectar problemas rápidamente.

## Template Strings

Facilitan la creación de mensajes claros y legibles.

### CÓDIGO

```
console.log(`Producto agregado: ${product.name} con ID ${product.id}`);
```

### Nota

Estos conceptos se integran para crear un backend funcional, robusto y fácil de mantener.

En el desarrollo backend moderno, es fundamental no solo escribir código que funcione, sino también que sea mantenible, testeable y claro para otros desarrolladores. La clase `ProductManager` que diseñarás será un ejemplo práctico de cómo aplicar POO para modelar la gestión de productos, incluyendo validaciones y manejo de errores para evitar fallos inesperados.

Ejecutar tus pruebas desde la terminal con Node.js te permitirá validar rápidamente el comportamiento de tu código y asegurar que cumple con los requisitos. Además, documentar tu proyecto con un README claro facilitará la comprensión y uso de tu trabajo por parte de otros.

Este entregable es una oportunidad para demostrar que puedes integrar todos los conocimientos adquiridos y aplicarlos en un proyecto real, siguiendo buenas prácticas de desarrollo backend.

# Programación Backend Avanzada

Avanza en tu dominio de JavaScript explorando las funcionalidades introducidas en ES7 y ES8. Este módulo te guía desde el operador exponencial y el método `Array.includes` hasta la manipulación eficiente de objetos con `Object.entries` y `Object.values`. Además, descubrirás cómo `async/await` simplifica el manejo de operaciones asíncronas, mejorando la legibilidad y mantenimiento del código. Partiendo de tus conocimientos básicos, aprenderás a aplicar estas herramientas en entornos backend con Node.js, preparando el terreno para optimizaciones y patrones asíncronos más avanzados en módulos posteriores.

## ES7 y ES8 – Sintaxis Moderna

Explora las funcionalidades introducidas en ECMAScript 2016 y 2017, incluyendo el operador exponencial, `Array.includes()`, `async/await` y las APIs `Object.entries()` y `Object.values()`. Aprende a aplicar estas herramientas para escribir código backend más legible, eficiente y moderno en Node.js.

Cuando desarrollamos aplicaciones backend con **Node.js**, escribimos código JavaScript que corre en el servidor. Con el paso de los años, el lenguaje fue incorporando nuevas funcionalidades en el estándar **ECMAScript**, que permiten escribir código más claro, más corto y más fácil de mantener.

En este workshop veremos algunas herramientas modernas que se utilizan con frecuencia en proyectos reales de backend.

### Operador Exponencial (``)

Introducido en ES7, el operador `**` es un operador aritmético que eleva un número a la potencia de otro. Es una alternativa más legible y concisa a la función `Math.pow()`.

#### CÓDIGO

```
const base = 2;
const exponente = 3;
const resultado = base ** exponente; // 8
```

Este operador es especialmente útil en cálculos matemáticos y algoritmos que requieren potencias, reduciendo la verbosidad del código.

### Método `Array.includes()`

Este método permite verificar si un array contiene un elemento específico, devolviendo un valor booleano (`true` o `false`). Es más legible que usar `indexOf()` para este propósito.

#### CÓDIGO

```
const frutas = ['manzana', 'banana', 'cereza'];
console.log(frutas.includes('banana')); // true
console.log(frutas.includes('pera')); // false
```

`Array.includes()` mejora la claridad en condiciones y filtros, facilitando la lectura y mantenimiento.

## Sintaxis `async/await`

`async/await` es una sintaxis introducida en ES8 que permite escribir código asíncrono de manera secuencial y más legible, evitando el encadenamiento excesivo de promesas.

- Una función marcada con `async` siempre devuelve una promesa.
- La palabra clave `await` pausa la ejecución hasta que la promesa se resuelve o rechaza.

Ejemplo comparativo:

**Con promesas encadenadas:**

#### CÓDIGO

```
function obtenerDatos() {
  return fetch('https://api.example.com/data')
    .then(response => response.json())
    .then(data => {
      console.log(data);
    })
    .catch(error => {
      console.error('Error:', error);
    });
}
```

**Con `async/await`:**

#### CÓDIGO

```
async function obtenerDatos() {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error:', error);
  }
}
```

Esta sintaxis mejora la legibilidad y el manejo de errores, especialmente en flujos complejos.

## **Object.entries() y Object.values()**

Estas funciones permiten obtener arrays con las entradas (pares clave-valor) o solo los valores de un objeto, facilitando su iteración y transformación.

### **CÓDIGO**

```
const usuario = { nombre: 'Ana', edad: 30 };

console.log(Object.entries(usuario));
// [['nombre', 'Ana'], ['edad', 30]]

console.log(Object.values(usuario));
// ['Ana', 30]
```

Son muy útiles para convertir objetos en estructuras iterables, aplicar transformaciones o filtrar datos en backend.

En el desarrollo backend con Node.js, estas funcionalidades modernas de ECMAScript permiten escribir código más limpio y eficiente. Por ejemplo, el operador exponencial puede usarse en cálculos matemáticos para procesamiento de datos o algoritmos de negocio.

`Array.includes()` es ideal para validar la existencia de elementos en listas, como verificar roles de usuario o permisos.

La sintaxis `async/await` es fundamental para manejar operaciones asíncronas comunes en backend, como consultas a bases de datos, llamadas a APIs externas o lectura de archivos, mejorando la legibilidad y el manejo de errores.

Finalmente, `Object.entries()` y `Object.values()` facilitan la manipulación de objetos JSON recibidos o enviados en APIs, permitiendo transformaciones y filtrados sencillos.

Estas herramientas son esenciales para mantener código backend moderno, legible y fácil de mantener, alineado con las mejores prácticas actuales en desarrollo con Node.js.

## **Simulación de Calculadora Positiva**

En esta unidad, aprenderás a implementar una función asíncrona que simula una calculadora que devuelve una promesa resuelta si el resultado es positivo y rechazada en caso contrario. Trabajaremos con promesas usando `then/catch` y reescribiremos la función con `async/await` y manejo de errores con `try/catch`. Además, aplicarás operadores modernos como `rest/spread`, `exponencial` y `nullish` para optimizar el código.

## Simulación de Calculadora Positiva: Un reto práctico para dominar asincronía y operadores modernos

Imagina que necesitas crear una función en JavaScript que simule una calculadora, pero con una particularidad: solo acepta resultados positivos. Si el resultado es positivo, la función debe devolver una promesa resuelta; si no, debe rechazarla. ¿Cómo manejarías esta lógica usando las herramientas modernas de ECMAScript? En esta unidad, te sumergirás en la implementación práctica de esta "calculadora positiva" para consolidar tu comprensión de promesas, `async/await` y operadores avanzados.

Este ejercicio te permitirá aplicar conceptos clave que ya has visto, como las promesas y la sintaxis `async/await`, y te desafiará a usar operadores modernos como `rest/spread`, `exponencial` y `nullish` para escribir código limpio y eficiente. Al finalizar, estarás preparado para manejar asincronía en backend con confianza y prepararás el terreno para el manejo avanzado de eventos en la siguiente unidad.

### Fundamentos para la Calculadora Positiva

#### Promesas (Promise)

Una `Promise` es un objeto que representa la eventual finalización o fallo de una operación asíncrona y su valor resultante. En nuestro caso, la función calculadora devolverá una promesa que se resolverá si el resultado es positivo o se rechazará si no lo es.

#### CÓDIGO

```
const calculadoraPositiva = (a, b) => {
  return new Promise((resolve, reject) => {
    const resultado = a + b; // ejemplo simple
    if (resultado > 0) {
      resolve(resultado);
    } else {
      reject(new Error('Resultado no es positivo'));
    }
  });
};
```

#### Async/Await

La sintaxis `async/await` permite escribir código asíncrono de forma secuencial y más legible. Usamos `try/catch` para manejar errores.

#### CÓDIGO

```
async function usarCalculadora(a, b) {  
  try {  
    const resultado = await calculadoraPositiva(a, b);  
    console.log('Resultado positivo:', resultado);  
  } catch (error) {  
    console.error('Error:', error.message);  
  }  
}
```

### Operadores rest y spread en objetos/arrays - Nivel APPLY

El operador `...` permite expandir (spread) o agrupar (rest) elementos. Por ejemplo, para recibir múltiples argumentos o combinar objetos.

#### CÓDIGO

```
function sumar(...numeros) {  
  return numeros.reduce((acc, val) => acc + val, 0);  
}  
  
const obj1 = {a: 1, b: 2};  
const obj2 = {...obj1, c: 3};
```

### Operador exponencial ()

Eleva un número a la potencia de otro.

#### CÓDIGO

```
const potencia = 2 ** 3; // 8
```

### Operador nullish (??)

Devuelve el operando derecho solo si el izquierdo es `null` o `undefined`.

#### CÓDIGO

```
const valor = null ?? 'valor por defecto'; // 'valor por defecto'
```

#### Nota

Estos operadores mejoran la legibilidad y robustez del código, especialmente en funciones asíncronas y manejo de parámetros.

## Aplicación práctica en backend

Aunque este ejercicio es simple, refleja situaciones reales del backend:

- operaciones que pueden fallar
- manejo de errores
- funciones que reciben múltiples parámetros
- manipulación de datos

Practicar estos conceptos te prepara para trabajar con:

- consultas a bases de datos
- APIs externas
- procesamiento de datos

## ES9 – finally y operadores rest/spread

En esta unidad exploraremos dos funcionalidades clave introducidas en ECMAScript 2018 (ES9): el método `Promise.prototype.finally()` y los operadores rest/spread aplicados a objetos y arrays. Aprenderás cómo usar `finally` para garantizar la ejecución de código de limpieza en promesas, y cómo aplicar rest/spread para manipular estructuras de datos de forma inmutable, mejorando la claridad y robustez de tu código backend en Node.js.

¿Alguna vez has necesitado ejecutar una acción después de que una promesa se resuelva o rechace, sin importar el resultado? O quizás has enfrentado problemas al combinar o clonar objetos y arrays en JavaScript, terminando con código complejo y propenso a errores. En esta unidad, descubrirás cómo ECMAScript 2018 (ES9) introduce herramientas que simplifican estas tareas comunes en el desarrollo backend con Node.js.

Exploraremos el método `Promise.prototype.finally()`, que permite ejecutar código de limpieza o liberación de recursos tras la resolución o rechazo de una promesa, mejorando la gestión de asincronía. Además, profundizaremos en los operadores rest y spread aplicados a objetos y arrays, que facilitan la manipulación inmutable de datos, un patrón esencial para escribir código backend robusto y mantenible.

Al finalizar, serás capaz de implementar estas funcionalidades para optimizar tus flujos asíncronos y refactorizar código impuro, alineándote con las mejores prácticas modernas de ECMAScript y desarrollo backend.

### Promise.prototype.finally()

El método `finally()` es una adición a la cadena de promesas que permite ejecutar una función una vez que la promesa se ha resuelto o rechazado, sin importar el resultado. Su sintaxis es:



#### CÓDIGO

```
promise
  .then(resultado => { /* manejo éxito */ })
  .catch(error => { /* manejo error */ })
  .finally(() => { /* código que siempre se ejecuta */ });
```

#### Características clave:

- No recibe argumentos.
- Se ejecuta después de `then` o `catch`.
- Ideal para liberar recursos, cerrar conexiones o limpiar estados.

#### Ejemplo

##### CÓDIGO

```
const fs = require('fs').promises;

fs.readFile('archivo.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error('Error:', err))
  .finally(() => console.log('Operación finalizada.)); // Siempre se ejecu-
ta
```

## Operadores rest y spread en arrays y objetos

Los operadores rest y spread usan la sintaxis `...` para agrupar o expandir elementos.

### En arrays

- **Spread:** Expande elementos de un array.

#### CÓDIGO

```
const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4]; // [1, 2, 3, 4]
```

- **Rest:** Agrupa elementos restantes en un nuevo array.

#### CÓDIGO

```
const [primero, ...resto] = [10, 20, 30, 40];
console.log(resto); // [20, 30, 40]
```

## En objetos

- **Spread:** Clona o combina propiedades.

### CÓDIGO

```
const obj1 = { a: 1, b: 2 };  
const obj2 = { ...obj1, c: 3 }; // { a:1, b:2, c:3 }
```

- **Rest:** Extrae propiedades específicas y agrupa el resto.

### CÓDIGO

```
const { a, ...resto } = { a: 10, b: 20, c: 30 };  
console.log(resto); // { b: 20, c: 30 }
```

## Ventajas:

- Facilitan la inmutabilidad evitando mutar objetos o arrays originales.
- Mejoran la legibilidad y mantenibilidad del código.

### Nota

En backend, mantener inmutabilidad ayuda a evitar efectos secundarios inesperados en aplicaciones concurrentes o con múltiples procesos.

En el desarrollo backend con Node.js, manejar correctamente la asincronía y la manipulación de datos es fundamental para construir aplicaciones robustas y escalables.

El método `finally()` permite asegurar que ciertas acciones, como cerrar conexiones a bases de datos o liberar recursos, se ejecuten siempre, evitando fugas o estados inconsistentes.

Por otro lado, los operadores rest/spread son herramientas poderosas para trabajar con datos de forma inmutable, lo que es crucial en entornos donde múltiples procesos o peticiones pueden interactuar con los mismos objetos o arrays. Por ejemplo, al combinar configuraciones o actualizar estados sin modificar los originales, se previenen errores difíciles de detectar.

Estos conceptos se integran con lo aprendido previamente sobre promesas y sintaxis moderna, y preparan el terreno para temas posteriores como importaciones dinámicas y encapsulación en clases, que también requieren un manejo cuidadoso de recursos y estructuras de datos.

## Sincronismo vs. Asincronismo

Explora las diferencias fundamentales entre la ejecución sincrónica y asincrónica en aplicaciones backend, su impacto en el rendimiento y cuándo es preferible utilizar cada enfoque para optimizar procesos y evitar bloqueos.

¿Alguna vez te has preguntado por qué algunas aplicaciones backend responden rápidamente mientras otras parecen quedarse "congeladas" durante ciertos procesos? Este comportamiento está estrechamente ligado a cómo el código se ejecuta: de forma sincrónica o asincrónica. En este módulo, descubrirás qué significa que una operación sea bloqueante o no bloqueante, y cómo estas diferencias impactan directamente en el rendimiento y la experiencia del usuario en aplicaciones backend. A través de ejemplos prácticos en Node.js, entenderás cuándo es crucial adoptar la asincronía para evitar cuellos de botella y mantener tus servicios ágiles y escalables.

## Sincronismo y Asincronismo en Backend

### ¿Qué es la ejecución sincrónica?

La ejecución **sincrónica** se refiere a un modelo donde las instrucciones se procesan una tras otra, esperando que cada operación termine antes de iniciar la siguiente. Esto implica que si una tarea tarda mucho, el programa queda bloqueado hasta su finalización.

#### Ejemplo

##### CÓDIGO

```
const data = readFileSync('archivo.txt'); // Bloquea hasta leer el archivo
console.log(data);
```

En este caso, `readFileSync` es una operación bloqueante que detiene el flujo hasta completar la lectura.

### ¿Qué es la ejecución asincrónica?

La ejecución **asincrónica** permite que el programa inicie una operación y continúe con otras tareas sin esperar a que la primera termine. Esto se logra mediante callbacks, promesas o `async/await`.

#### Ejemplo

##### CÓDIGO

```
fs.readFile('archivo.txt', (err, data) => {
  if (err) throw err;
  console.log(data);
});
console.log('Lectura iniciada');
```

Aquí, la lectura del archivo es no bloqueante; el programa sigue ejecutando mientras espera la respuesta.

## Impacto en el rendimiento

- **Sincronismo:** Puede causar bloqueos, afectando la capacidad de respuesta y escalabilidad.
- **Asincronismo:** Mejora la eficiencia al aprovechar el tiempo de espera para otras tareas, esencial en operaciones I/O intensivas.

## Terminología clave

| Término       | Definición                                                                        |
|---------------|-----------------------------------------------------------------------------------|
| Bloqueante    | Operación que detiene el flujo hasta completarse                                  |
| No bloqueante | Operación que permite continuar sin esperar resultado inmediato                   |
| Callback      | Función que se ejecuta tras completar una operación asíncrona                     |
| Promesa       | Objeto que representa la eventual finalización o falla de una operación asíncrona |

**Conoce más:** La asincronía es fundamental en Node.js para manejar múltiples conexiones simultáneas sin bloquear el event loop, lo que veremos en detalle en unidades posteriores.

En el desarrollo backend, especialmente con Node.js, la mayoría de las operaciones que involucran entrada/salida (I/O) —como acceder a bases de datos, leer archivos o hacer peticiones HTTP— son inherentemente lentas comparadas con la velocidad del procesador. Si estas operaciones se ejecutan de forma sincrónica, el servidor queda bloqueado, incapaz de atender otras solicitudes, lo que degrada la experiencia del usuario y limita la escalabilidad.

Por ejemplo, un servidor que lee archivos de configuración o responde a consultas de base de datos debe hacerlo sin detener el procesamiento de otras peticiones. Aquí es donde la asincronía se vuelve crítica: permite que el backend inicie una operación y continúe atendiendo otras tareas, mejorando la eficiencia y el rendimiento general.

Este conocimiento es esencial para diseñar aplicaciones backend robustas y escalables, y sienta las bases para dominar técnicas avanzadas como promesas y `async/await`, que abordaremos en la siguiente unidad.

## Callbacks, Promesas y Async/Await

En esta unidad profundizaremos en los patrones fundamentales para manejar la asincronía en JavaScript backend: callbacks, promesas y `async/await`. Revisaremos cómo transformar código con callbacks anidados en promesas, encadenar promesas con manejo robusto de errores usando `then`, `catch` y `finally`, y migrar flujos a `async/await` con `try/catch` para un código más legible y mantenible.

¿Alguna vez te has encontrado con código JavaScript donde las funciones anidadas se vuelven difíciles de leer y mantener? Este fenómeno, conocido como *callback hell*, es común en el desarrollo backend cuando manejamos operaciones asíncronas con callbacks tradicionales. En esta unidad, exploraremos cómo superar estas dificultades utilizando promesas y la sintaxis moderna `async/await`, herramientas

que no solo mejoran la legibilidad sino también el manejo robusto de errores en aplicaciones backend con Node.js.

A lo largo de esta unidad, aprenderás a transformar código con callbacks anidados en promesas encadenadas, a utilizar el método `finally` para ejecutar acciones al concluir cualquier promesa, y a reescribir flujos asíncronos con `async/await`, capturando errores de forma clara y efectiva. Estos conocimientos son esenciales para desarrollar aplicaciones backend modernas, eficientes y mantenibles.

## Cómo manejar operaciones asíncronas en JavaScript

Cuando desarrollamos aplicaciones con **Node.js**, muchas tareas no se completan inmediatamente. Por ejemplo:

- consultar una base de datos
- leer un archivo
- llamar a una API externa
- enviar un email

Estas operaciones pueden tardar algunos milisegundos o segundos. Por eso JavaScript utiliza **programación asíncrona**, lo que permite continuar ejecutando otras tareas mientras esperamos el resultado.

En esta unidad aprenderemos tres formas de manejar operaciones asíncronas:

1. **Callbacks**
2. **Promesas**
3. **Async / Await**

Además veremos cómo evitar un problema común llamado **Callback Hell**.

### Callbacks

Un **callback** es simplemente una función que se pasa como argumento a otra función para ejecutarse cuando una tarea termina.

Ejemplo sencillo:

#### CÓDIGO

```
function saludar(nombre, callback) {  
  console.log("Hola " + nombre);  
  callback();  
}  
  
saludar("Ana", () => {  
  console.log("Saludo finalizado");  
});
```

Salida:

#### CÓDIGO

```
Hola Ana  
Saludo finalizado
```

## Ejemplo real en Node.js

En Node.js es común usar callbacks para manejar operaciones del sistema.

Por ejemplo, leer un archivo:

#### CÓDIGO

```
const fs = require("fs");  
  
fs.readFile("archivo.txt", (error, data) => {  
  if (error) {  
    console.error("Error al leer archivo:", error);  
  } else {  
    console.log("Contenido:", data.toString());  
  }  
});
```

Aquí el callback se ejecuta **cuando termina la lectura del archivo**.

## Problema: Callback Hell

Cuando varias operaciones dependen unas de otras, los callbacks empiezan a **anidarse**.

Ejemplo:

#### CÓDIGO

```
login(user, (err, userData) => {  
  if (err) return handleError(err);  
  
  getProfile(userData.id, (err, profile) => {  
    if (err) return handleError(err);  
  
    getSettings(profile.id, (err, settings) => {  
      if (err) return handleError(err);  
  
      console.log(settings);  
    });  
  });  
});
```

Esto genera una estructura difícil de leer llamada:

#### “Callback Hell”

o también conocida como “**Pirámide de la perdición**”

Problemas:

- código difícil de entender
- manejo de errores repetido
- mantenimiento complicado

Para solucionar esto se introdujeron **las Promesas**.

#### Promesas (Promise)

Una **Promise** representa el resultado de una operación que puede completarse **en el futuro**.

Una promesa tiene tres estados:

| Estado    | Significado                 |
|-----------|-----------------------------|
| pending   | la operación aún no terminó |
| fulfilled | la operación fue exitosa    |
| rejected  | ocurrió un error            |

## Crear una promesa

### CÓDIGO

```
const promesa = new Promise((resolve, reject) => {  
  
  const todoBien = true;  
  
  if (todoBien) {  
    resolve("Operación exitosa");  
  } else {  
    reject("Ocurrió un error");  
  }  
  
});
```

## Usar una promesa

### CÓDIGO

```
promesa  
  .then(resultado => {  
    console.log(resultado);  
  })  
  .catch(error => {  
    console.error(error);  
  });
```

Salida:

### CÓDIGO

```
Operación exitosa
```

## Encadenar Promesas

Una ventaja importante es que podemos **encadenar operaciones**.

### CÓDIGO

```
login(user)  
  .then(userData => getProfile(userData.id))  
  .then(profile => getSettings(profile.id))  
  .then(settings => console.log(settings))  
  .catch(error => console.error(error));
```

Esto evita la anidación profunda de callbacks.



## Promise.finally()

El método `finally()` permite ejecutar código **siempre al final**, sin importar si la promesa tuvo éxito o error.

### CÓDIGO

```
promesa
  .then(resultado => console.log(resultado))
  .catch(error => console.error(error))
  .finally(() => {
    console.log("Operación finalizada");
  });
```

Esto es útil para tareas como:

- cerrar conexiones
- liberar recursos
- mostrar mensajes de finalización

## Async / Await

`async/await` es una forma moderna de trabajar con promesas.

Permite escribir código asíncrono **como si fuera síncrono**, lo que mejora mucho la legibilidad.

## Ejemplo

### CÓDIGO

```
async function obtenerDatos() {
  try {
    const user = await login(userId);
    const profile = await getProfile(user.id);

    console.log(profile);

  } catch (error) {
    console.error("Error:", error);
  }
}
```

Aquí ocurre lo siguiente:

`await` espera el resultado de la promesa

`try/catch` maneja errores

El código es **más fácil de leer y mantener**.

## Comparación de los tres enfoques

| Método      | Ventajas         | Problemas                       |
|-------------|------------------|---------------------------------|
| Callbacks   | simples          | difícil de escalar              |
| Promesas    | mejor estructura | aún puede crecer en complejidad |
| Async/Await | código más claro | requiere entender promesas      |

Hoy en día, **async/await** es la forma más utilizada en **Node.js** moderno.

## Aplicación en Backend

En aplicaciones backend con Node.js estas técnicas se usan constantemente para manejar:

- consultas a bases de datos
- llamadas a APIs externas
- operaciones de archivos
- autenticación de usuarios
- procesamiento de datos

Usar promesas o `async/await` permite:

- evitar bloqueos innecesarios
- manejar errores correctamente
- mantener el código limpio y mantenible

En el desarrollo backend con Node.js, manejar operaciones asíncronas es fundamental para no bloquear el hilo principal y mantener la aplicación responsiva. Sin embargo, el uso tradicional de callbacks puede complicar el código y aumentar la probabilidad de errores difíciles de depurar.

Las promesas y `async/await` son herramientas estándar en ECMAScript que permiten escribir código asíncrono más limpio, legible y mantenible. Por ejemplo, al interactuar con bases de datos, servicios externos o sistemas de archivos, estas técnicas facilitan el control del flujo y el manejo de errores.

Además, el método `finally` en promesas es especialmente útil para liberar recursos o realizar acciones comunes tras la finalización de cualquier operación, independientemente de su éxito o fallo.

Esta unidad te prepara para aplicar estas técnicas en tus proyectos backend, mejorando la calidad y robustez de tu código, y facilitando la colaboración en equipos de desarrollo.

## ES10 – Dinamicidad y optimización

Explora las novedades de ES10 enfocadas en la carga dinámica de módulos, la optimización de estructuras de datos con `Array.flat` y la normalización de cadenas mediante métodos de trimming. Aprende a implementar importaciones condicionales para mejorar el rendimiento backend, aplanar arrays anidados para simplificar pipelines de datos y limpiar entradas de usuario para robustecer tus aplicaciones.

¿Alguna vez has necesitado cargar módulos en tu aplicación backend solo cuando realmente se requieren, para optimizar recursos y tiempos de ejecución? ¿O has enfrentado estructuras de datos complejas y anidadas que dificultan su procesamiento? Además, ¿cómo asegurar que las entradas de usuario estén limpias y normalizadas para evitar errores inesperados? En esta unidad, abordaremos estas preguntas explorando tres funcionalidades clave introducidas en ECMAScript 2019 (ES10): la importación dinámica con `import()`, el método `Array.flat()` para aplanar arrays anidados y los métodos de trimming (`trimStart`, `trimEnd` y `trim`) para normalizar cadenas. Estas herramientas te permitirán escribir código backend más eficiente, modular y robusto, mejorando la gestión de dependencias, el procesamiento de datos y la validación de entradas. Al finalizar, serás capaz de implementar cargas condicionales de módulos, optimizar pipelines de datos y limpiar strings de forma efectiva, habilidades esenciales para desarrolladores backend modernos.

### Importación dinámica con `import()`

La función `import()` permite cargar módulos de forma dinámica en tiempo de ejecución, a diferencia de las importaciones estáticas con `import` al inicio del archivo. Esto habilita el *lazy-loading*, es decir, cargar código solo cuando es necesario, optimizando el rendimiento y reduciendo el consumo de recursos.

#### CÓDIGO

```
// Ejemplo básico de importación dinámica
async function cargarModulo(nombreModulo) {
  const modulo = await import(`./modulos/${nombreModulo}.js`);
  modulo.funcionPrincipal();
}
```

#### Nota

`import()` devuelve una promesa que se resuelve con el módulo importado.

### Método `Array.flat()`

`Array.flat()` es un método que permite aplanar arrays anidados hasta una profundidad específica (por defecto 1). Esto simplifica la manipulación de datos complejos, especialmente en pipelines donde se reciben estructuras anidadas.

#### CÓDIGO

```
const datos = [1, 2, [3, 4, [5, 6]]];
console.log(datos.flat()); // [1, 2, 3, 4, [5, 6]]
console.log(datos.flat(2)); // [1, 2, 3, 4, 5, 6]
```

Este método es especialmente útil para limpiar payloads o resultados de consultas que devuelven arrays anidados.

### Métodos de trimming: `trimStart()`, `trimEnd()` y `trim()`

Estos métodos permiten eliminar espacios en blanco o caracteres específicos al inicio ( `trimStart()` ), al final ( `trimEnd()` ) o en ambos extremos ( `trim()` ) de una cadena.

#### CÓDIGO

```
const entrada = ' usuario@example.com ';
console.log(entrada.trimStart()); // 'usuario@example.com '
console.log(entrada.trimEnd()); // ' usuario@example.com'
console.log(entrada.trim()); // 'usuario@example.com'
```

Son fundamentales para validar y normalizar entradas de usuario, evitando errores por espacios no deseados.

### Resumen de conceptos

| Concepto                                                                | Descripción breve                                |
|-------------------------------------------------------------------------|--------------------------------------------------|
| <code>import()</code>                                                   | Carga dinámica de módulos en tiempo de ejecución |
| <code>Array.flat()</code>                                               | Aplanamiento de arrays anidados                  |
| <code>trimStart()</code> , <code>trimEnd()</code> , <code>trim()</code> | Normalización de strings eliminando espacios     |

En el desarrollo backend, la eficiencia y la limpieza del código son cruciales para mantener aplicaciones escalables y mantenibles. La importación dinámica con `import()` permite cargar módulos solo cuando se necesitan, lo que es especialmente útil en aplicaciones con múltiples rutas o funcionalidades que no siempre se ejecutan, reduciendo el tiempo de arranque y el consumo de memoria.

Por otro lado, los datos que recibimos o procesamos suelen venir en estructuras complejas y anidadas, como respuestas de bases de datos o APIs. Utilizar `Array.flat()` facilita la transformación de estos datos para su posterior procesamiento o almacenamiento.

Finalmente, la validación y normalización de entradas es una práctica esencial para evitar errores y vulnerabilidades. Los métodos de trimming ayudan a limpiar cadenas de texto, asegurando que los datos sean consistentes antes de ser procesados o almacenados.

Estos conceptos se integran para mejorar la calidad y rendimiento de tus aplicaciones backend, alineándose con las mejores prácticas actuales en desarrollo con Node.js y ECMAScript moderno.

## Gestión de Eventos en JavaScript

En esta unidad aprenderás a manejar eventos en JavaScript utilizando `addEventListener` y `removeEventListener`, comprenderás el contexto de ejecución (`this`) en los manejadores, y aplicarás técnicas para controlar el flujo de eventos con `preventDefault()` y `stopPropagation()`. Estas habilidades son esenciales para desarrollar aplicaciones backend con interfaces interactivas o para gestionar eventos en entornos que simulan interacción de usuario.

¿Alguna vez te has preguntado cómo las aplicaciones web y backend con interfaces manejan las interacciones del usuario de forma eficiente y controlada? La gestión de eventos es la clave para responder a acciones como clics, pulsaciones de teclas o envíos de formularios. En esta unidad, exploraremos cómo JavaScript permite suscribirnos a estos eventos mediante `addEventListener`, cómo podemos eliminar esos suscriptores con `removeEventListener`, y cómo controlar el comportamiento predeterminado y la propagación de eventos con `preventDefault()` y `stopPropagation()`. Estas herramientas son fundamentales para construir aplicaciones backend modernas que interactúan con interfaces o manejan eventos de usuario de forma robusta y predecible. Al finalizar, serás capaz de implementar manejadores de eventos efectivos, controlar su flujo y contexto, y aplicar estos conocimientos en casos prácticos reales.

### ¿Qué es un evento?

Un *evento* es una acción o suceso que ocurre en el sistema, como un clic del mouse, una pulsación de tecla o el envío de un formulario. En JavaScript, podemos responder a estos eventos ejecutando funciones llamadas *manejadores* o *listeners*.

### `addEventListener` y `removeEventListener`

- `addEventListener` es un método que permite suscribir una función para que se ejecute cuando ocurra un evento específico en un elemento.

#### CÓDIGO

```
element.addEventListener('click', function(event) {  
  console.log('Se hizo clic en el elemento');  
});
```

- `removeEventListener` elimina un manejador previamente registrado, evitando que se ejecute en eventos futuros.

#### CÓDIGO

```
function handleClick(event) {  
  console.log('Click detectado');  
}  
element.addEventListener('click', handleClick);  
// Más tarde...  
element.removeEventListener('click', handleClick);
```

#### Nota

Para que `removeEventListener` funcione, la función debe ser una referencia nombrada, no una función anónima.

## Contexto de ejecución (`this`) en manejadores

Dentro de un manejador, `this` se refiere al elemento que recibió el evento. Esto es útil para acceder o modificar propiedades del elemento directamente.

#### CÓDIGO

```
button.addEventListener('click', function() {  
  console.log(this.id); // ID del botón clickeado  
});
```

Si usas funciones flecha, `this` no se vincula al elemento, sino al contexto externo, por lo que para manejadores es preferible usar funciones tradicionales.

## Control del flujo de eventos

### `preventDefault()`

Este método cancela la acción predeterminada que el navegador ejecutaría ante un evento.

Ejemplo: evitar que un formulario se envíe automáticamente para validar datos primero.

#### CÓDIGO

```
form.addEventListener('submit', function(event) {  
  event.preventDefault();  
  // Validar datos antes de enviar  
});
```

### `stopPropagation()`

Detiene la propagación del evento hacia elementos padres, evitando que otros manejadores se ejecuten.

#### CÓDIGO

```
childElement.addEventListener('click', function(event) {  
  event.stopPropagation();  
  console.log('Click manejado solo en el hijo');  
});
```

### Tipos de eventos comunes en aplicaciones con UI backend

- **Mouse:** `click`, `dblclick`, `mouseover`, `mouseout`
- **Teclado:** `keydown`, `keyup`, `keypress`
- **Formularios:** `submit`, `change`, `input`

Estos eventos permiten crear interfaces interactivas y controlar la experiencia del usuario desde el backend o entornos híbridos.

En el desarrollo backend moderno, especialmente cuando se trabaja con frameworks que integran interfaces o manejan eventos de usuario (como en aplicaciones con Electron, NW.js o servidores que gestionan eventos de entrada), la gestión adecuada de eventos es crucial. Saber suscribirse y desuscribirse de eventos evita fugas de memoria y comportamientos inesperados. Además, controlar el flujo de eventos con `preventDefault()` y `stopPropagation()` permite personalizar la interacción y evitar conflictos entre manejadores.

Por ejemplo, en un formulario de registro, puedes evitar que se envíe si los datos no son válidos, o impedir que un clic en un botón dentro de un contenedor active también el manejador del contenedor padre. Estas técnicas aseguran que la aplicación responda correctamente a las acciones del usuario y mantenga un comportamiento predecible y profesional.

Esta unidad te prepara para implementar estas prácticas en tus proyectos backend con interfaces o en cualquier entorno donde JavaScript maneje eventos, asegurando código limpio, eficiente y fácil de mantener.

## ES11 – Nullish y variables privadas

Explora el operador nullish (??) para asignaciones seguras de valores por defecto y aprende a definir campos privados en clases con la sintaxis `#propiedad`, mejorando la encapsulación y robustez en aplicaciones backend.

¿Alguna vez has querido asignar un valor por defecto a una variable pero te has encontrado con que valores como `0` o `''` son tratados como falsy y no funcionan como esperabas? En el desarrollo backend con JavaScript moderno, ES11 introduce el operador nullish (??), que permite asignar valores por defecto solo cuando el valor es `null` o `undefined`, evitando errores comunes en la lógica de asignación.

Además, la encapsulación es clave para mantener la integridad del estado en aplicaciones backend robustas. ES11 presenta la sintaxis para campos privados en clases usando `#propiedad`, una forma clara y segura de proteger datos internos y evitar accesos no deseados.

En esta unidad, aprenderás a aplicar el operador nullish para manejar valores por defecto de forma segura y a definir campos privados en clases para mejorar la encapsulación, dos herramientas esenciales para escribir código backend más limpio y confiable.

## Operador Nullish (??)

El operador nullish ( `??` ) es una adición de ES11 que permite asignar un valor por defecto solo cuando el operando izquierdo es `null` o `undefined`. A diferencia del operador OR lógico ( `||` ), que considera falsy valores como `0`, `''` o `false`, `??` es más preciso para casos donde esos valores son válidos.

### CÓDIGO

```
let valor = 0;
console.log(valor || 10); // Imprime 10 porque 0 es falsy
console.log(valor ?? 10); // Imprime 0 porque 0 no es null ni undefined
```

**Uso en backend:** - Asignar configuraciones con valores numéricos o cadenas vacías legítimas. - Evitar errores en validaciones donde `0` o `''` son valores válidos.

## Campos Privados en Clases (#propiedad)

ES11 introduce la sintaxis para definir campos privados en clases usando el prefijo `#`. Estos campos no son accesibles desde fuera de la clase, lo que mejora la encapsulación y evita modificaciones accidentales o malintencionadas.

### CÓDIGO

```
class Servicio {
  #estadoInterno = 0;

  incrementar() {
    this.#estadoInterno++;
  }

  obtenerEstado() {
    return this.#estadoInterno;
  }
}

const servicio = new Servicio();
servicio.incrementar();
console.log(servicio.obtenerEstado()); // 1
console.log(servicio.#estadoInterno); // Error: propiedad privada
```



**Beneficios en backend:** - Protección del estado interno de servicios y módulos. - Evita fugas de estado y accesos no autorizados. - Facilita mantenimiento y refactorización segura.

**Nota**

Los campos privados no pueden ser accedidos ni modificados desde fuera de la clase, ni siquiera mediante notación de corchetes o reflexión.

En aplicaciones backend, manejar valores por defecto correctamente es crucial para evitar errores lógicos y garantizar configuraciones robustas. Por ejemplo, al leer variables de entorno o parámetros de configuración, el operador nullish permite asignar valores seguros sin sobrescribir valores legítimos como `0`.

La encapsulación mediante campos privados en clases es fundamental para mantener la integridad del estado en servicios que manejan datos sensibles o estados internos complejos. Esto es especialmente importante en arquitecturas basadas en microservicios o en sistemas con concurrencia, donde el acceso no controlado puede causar inconsistencias.

Implementar estas características mejora la calidad del código, facilita la colaboración en equipos y reduce errores en producción, alineándose con las mejores prácticas modernas de desarrollo backend con JavaScript.

# Desarrollo Avanzado de Backend con Node.js

Comienza tu viaje en el desarrollo backend entendiendo qué es Node.js y cómo funciona como entorno de ejecución JavaScript fuera del navegador. En este módulo introductorio, descubrirás el papel clave del motor V8 y las diferencias básicas entre los sistemas de módulos CommonJS y ES Modules. Aprenderás a instalar Node.js en tu equipo y a ejecutar tus primeros scripts simples, sentando las bases para proyectos backend reales. Además, exploraremos conceptos básicos de servidores HTTP para contextualizar su uso. Este módulo te prepara para gestionar dependencias y automatizar tareas en el siguiente paso de tu formación.

## Breve repaso y mapa de módulos nativos

Introducción rápida a los módulos nativos de Node.js: `fs`, `path`, `http` y `crypto`. Conocerás para qué sirve cada uno y ejemplos comunes de uso en backend, preparando el camino para su aplicación práctica en proyectos reales.

¿Alguna vez te has preguntado cómo Node.js permite crear servidores web, manipular archivos o manejar rutas de forma eficiente sin depender de herramientas externas? La respuesta está en sus módulos nativos, que son piezas clave para desarrollar aplicaciones backend robustas y eficientes. En esta unidad, haremos un recorrido rápido por los módulos `fs`, `path`, `http` y `crypto`, entendiendo su propósito y ejemplos comunes de uso. Este repaso te preparará para profundizar en su aplicación práctica en las siguientes unidades, consolidando tu camino como desarrollador backend.

## Módulos nativos de Node.js: una visión general

Node.js incluye varios módulos integrados que permiten realizar tareas comunes en backend sin instalar librerías adicionales. Estos módulos están optimizados para rendimiento y portabilidad.

### 1. Módulo `fs` (File System)

El módulo `fs` permite **leer, escribir y manipular archivos** en el sistema de archivos.

- *Ejemplo de uso:* leer un archivo de configuración o guardar datos generados por la aplicación.

#### CÓDIGO

```
const fs = require('fs');

// Leer contenido de un archivo
fs.readFile('config.json', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

### 1. Módulo `path`

El módulo `path` facilita el **manejo de rutas de archivos** de forma portátil entre diferentes sistemas operativos.

- *Ejemplo de uso:* construir rutas absolutas o relativas para acceder a archivos sin errores.

#### CÓDIGO

```
const path = require('path');

const fullPath = path.join(__dirname, 'data', 'users.json');
console.log(fullPath);
```

### 3. Módulo `http`

El módulo `http` permite **crear servidores web básicos** y manejar solicitudes y respuestas HTTP sin frameworks externos.

- *Ejemplo de uso:* levantar un servidor que responda con contenido estático o dinámico.

#### CÓDIGO

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hola desde Node.js!');
});

server.listen(3000, () => {
  console.log('Servidor escuchando en puerto 3000');
});
```

## 4. Módulo crypto

El módulo `crypto` ofrece funciones criptográficas básicas como **hashes**, **generación de tokens** y **cifrado**.

- *Ejemplo de uso:* crear un hash seguro para almacenar contraseñas o generar identificadores únicos.

### CÓDIGO

```
const crypto = require('crypto');

const hash = crypto.createHash('sha256').update('miTextoSecreto').digest('hex');
console.log(hash);
```

### Nota

Estos ejemplos son básicos y se ampliarán con prácticas y explicaciones detalladas en las próximas unidades.

Estos módulos nativos son la base para muchas funcionalidades en aplicaciones backend reales. Por ejemplo, `fs` es esencial para manejar archivos de configuración o logs; `path` asegura que tu código funcione correctamente en distintos sistemas operativos; `http` te permite crear servidores personalizados sin depender de frameworks; y `crypto` es clave para la seguridad, como en la gestión de contraseñas o tokens.

En las siguientes unidades, aplicarás estos módulos para construir servidores HTTP, manipular archivos y asegurar datos, consolidando tu comprensión y habilidades prácticas. Este repaso te da el mapa general para entender dónde y cuándo usar cada módulo en tus proyectos backend.

## Conceptos de NPM y package.json

En esta unidad, exploraremos qué es NPM, su función como administrador de paquetes para proyectos Node.js, y cómo se configura el archivo `package.json` para gestionar dependencias y scripts. También introduciremos el versionado semántico (semver) para entender cómo se manejan las versiones de los paquetes en proyectos reales.

¿Alguna vez te has preguntado cómo los proyectos Node.js gestionan las librerías y herramientas que utilizan? Aquí es donde NPM, el administrador de paquetes por excelencia, juega un papel fundamental. En esta unidad, descubrirás cómo NPM facilita la instalación y gestión de dependencias, y cómo el archivo `package.json` actúa como el corazón de cualquier proyecto Node, definiendo qué paquetes se usan y cómo se ejecutan scripts importantes. Además, conocerás el concepto de versionado semántico (semver), clave para mantener tus proyectos actualizados y compatibles. Prepárate para entender estos conceptos esenciales que te permitirán manejar proyectos backend con confianza y profesionalismo.

## ¿Qué es NPM?

NPM (Node Package Manager) es el **administrador de paquetes oficial para Node.js**. Su función principal es **instalar, compartir y gestionar las dependencias** que un proyecto necesita para funcionar correctamente.

- Permite descargar librerías y herramientas desde un repositorio central.
- Facilita la actualización y mantenimiento de estas dependencias.
- Automatiza tareas mediante scripts definidos en el proyecto.

### Ejemplo

Si tu proyecto necesita manejar fechas, puedes instalar el paquete `moment` usando NPM, y este se descargará y guardará para que tu código lo utilice.

## El archivo `package.json`

Este archivo es el **corazón de cualquier proyecto Node.js**. Es un archivo en formato JSON que contiene:

- **Metadatos del proyecto:** nombre, versión, descripción.
- **Dependencias:** librerías que el proyecto necesita para funcionar.
- **devDependencies:** herramientas necesarias solo para desarrollo (tests, compiladores).
- **Scripts:** comandos personalizados para automatizar tareas comunes.

**Campos clave en `package.json`:**

| Campo                        | Descripción                                 |
|------------------------------|---------------------------------------------|
| <code>name</code>            | Nombre del proyecto                         |
| <code>version</code>         | Versión actual del proyecto                 |
| <code>dependencies</code>    | Paquetes necesarios en producción           |
| <code>devDependencies</code> | Paquetes necesarios solo en desarrollo      |
| <code>scripts</code>         | Comandos personalizados para tareas comunes |

### Nota

La distinción entre `dependencies` y `devDependencies` es importante para optimizar el tamaño y rendimiento en producción.

## Introducción a Semver (Versionado Semántico)

Semver es una convención para asignar versiones a los paquetes, con el formato:

#### CÓDIGO

MAJOR.MINOR.PATCH

- **MAJOR:** Cambios incompatibles que pueden romper funcionalidades.
- **MINOR:** Nuevas funcionalidades compatibles con versiones anteriores.
- **PATCH:** Correcciones de errores sin cambios en la API.

#### Ejemplo

La versión 2.3.1 indica:

- Major: 2
- Minor: 3
- Patch: 1

Esta convención ayuda a los desarrolladores a entender rápidamente el impacto de actualizar una dependencia.

### Rango de versiones en package.json

En `package.json`, puedes especificar rangos para las versiones de tus dependencias, por ejemplo:

- `^1.2.3` : acepta actualizaciones que no cambien el número MAJOR (1.x.x)
- `~1.2.3` : acepta actualizaciones que no cambien el número MINOR (1.2.x)

Esto permite mantener las dependencias actualizadas sin romper la compatibilidad.

**Ejercicio mental:** ¿Qué versión acepta `^2.5.0` ? (Respuesta: cualquier versión desde 2.5.0 hasta <3.0.0)

En el desarrollo profesional backend, manejar correctamente las dependencias es fundamental para garantizar que tu aplicación funcione de manera estable y segura. NPM y `package.json` son herramientas estándar en la industria para lograrlo. Entender cómo definir y diferenciar dependencias de producción y desarrollo, así como cómo interpretar y aplicar semver, te permitirá colaborar en proyectos reales, evitar conflictos de versiones y automatizar tareas comunes mediante scripts.

Por ejemplo, cuando trabajas en un equipo, definir correctamente las dependencias asegura que todos usen las mismas versiones de librerías, evitando errores inesperados. Además, el uso de scripts en `package.json` facilita la ejecución de comandos como pruebas o despliegues con un solo comando, mejorando la productividad.

Esta unidad te prepara para crear y administrar tu propio `package.json`, base para cualquier proyecto Node.js, y para entender cómo mantener tus dependencias actualizadas y seguras.

## Git y GitHub: pasos esenciales

Introducción práctica a Git y GitHub para gestionar versiones y colaborar en proyectos Node.js, con comandos básicos y flujo de trabajo local a remoto.

¿Alguna vez te has preguntado cómo los desarrolladores mantienen el control y la organización de su código cuando trabajan en equipo? Aquí es donde entra Git, un sistema de control de versiones que permite gestionar los cambios en tu código de forma segura y eficiente. En esta unidad, descubrirás los comandos básicos de Git y cómo usar GitHub para colaborar con otros desarrolladores, un paso fundamental para cualquier proyecto backend con Node.js.

### ¿Qué es Git?

Git es un *sistema de control de versiones distribuido* que registra los cambios realizados en los archivos de un proyecto. Esto permite a los desarrolladores trabajar simultáneamente, mantener un historial claro y revertir cambios si es necesario.

### Comandos básicos de Git

- `git clone <url>` : Copia un repositorio remoto a tu máquina local.
- `git add <archivo>` : Prepara los archivos para ser guardados en el próximo commit.
- `git commit -m "mensaje"` : Guarda los cambios preparados con un mensaje descriptivo.
- `git push` : Envía tus commits locales al repositorio remoto.

### ¿Qué es GitHub?

GitHub es una plataforma basada en Git que facilita la colaboración en proyectos de software. Permite alojar repositorios, gestionar versiones, y coordinar el trabajo mediante herramientas como *pull requests* (PR).

### Flujo básico con GitHub

1. Clonar el repositorio remoto.
2. Crear cambios localmente y hacer commits.
3. Subir los cambios con `git push`.
4. Abrir un *pull request* para que otros revisen y fusionen tus cambios.

#### Nota

Este flujo es esencial para trabajar en equipo y mantener la calidad del código en proyectos Node.js.

En proyectos backend con Node.js, mantener un control riguroso de las versiones es vital para evitar conflictos y asegurar la estabilidad del software. Git y GitHub permiten que múltiples desarrolladores colaboren sin pisarse el trabajo, facilitando la integración continua y la revisión de código. Por ejemplo, al trabajar en una API REST, puedes desarrollar nuevas funcionalidades en ramas separadas y luego

integrarlas mediante pull requests, asegurando que el código se revisa antes de ser parte del proyecto principal.



# Desarrollo de Backend Avanzado

Inicia tu recorrido en desarrollo backend comprendiendo la arquitectura Cliente-Servidor y los principios básicos de REST. Este módulo introduce cómo los clientes y servidores interactúan mediante HTTP, y refuerza el uso correcto de los métodos HTTP para manipular recursos. A través de una demostración práctica, crearás un servidor HTTP mínimo con Node.js que ejemplifica estas interacciones. Además, validarás tus conocimientos con un quiz rápido. Este aprendizaje te prepara para construir servidores más complejos y avanzar hacia Express en el siguiente módulo.

## Arquitectura Cliente-Servidor y REST (breve)

Introducción clara y práctica al modelo arquitectónico cliente-servidor y a los principios básicos de REST, fundamentales para el desarrollo backend con APIs HTTP.

¿Alguna vez te has preguntado cómo tu navegador obtiene datos de un servidor o cómo una aplicación móvil se comunica con un backend? Todo esto se basa en un modelo arquitectónico fundamental: la **arquitectura cliente-servidor**. En esta unidad, exploraremos este modelo y los principios básicos de REST, que es la base para diseñar APIs HTTP eficientes y escalables. Al finalizar, podrás explicar cómo funcionan estas arquitecturas y por qué son esenciales en el desarrollo backend moderno.

### Arquitectura Cliente-Servidor

La **arquitectura cliente-servidor** es un modelo donde un *cliente* (por ejemplo, un navegador o una app) realiza peticiones a un *servidor*, que procesa esas solicitudes y devuelve respuestas. Este modelo es la base de la mayoría de las aplicaciones web y móviles.

- **Cliente:** Inicia la comunicación, solicita recursos o servicios.
- **Servidor:** Responde a las solicitudes, gestiona recursos y lógica de negocio.

Este intercambio se realiza comúnmente sobre el protocolo HTTP, que define cómo se estructuran las peticiones y respuestas.

### Principios Básicos de REST

REST (Representational State Transfer) es un conjunto de restricciones arquitectónicas para diseñar APIs HTTP enfocadas en recursos y operaciones claras. Fue definido por Roy Fielding y es ampliamente usado en desarrollo backend.

#### Conceptos clave:

- **Recursos:** Entidades accesibles a través de URLs, por ejemplo, `/usuarios` o `/productos/123`.
- **Representaciones:** Formatos en los que se entregan los recursos, típicamente JSON o XML.

- **Verbos HTTP:** Operaciones estándar para interactuar con recursos, como `GET` , `POST` , `PUT` , `DELETE` .
- **Statelessness:** Cada petición del cliente al servidor debe contener toda la información necesaria; el servidor no debe almacenar estado entre peticiones.

#### Nota

REST no es un protocolo, sino un estilo arquitectónico que guía el diseño de APIs para que sean simples, escalables y mantenibles.

## Ejemplo práctico

Imagina una API para gestionar usuarios:

| Método HTTP | URL         | Acción                    |
|-------------|-------------|---------------------------|
| GET         | /usuarios   | Obtener lista de usuarios |
| POST        | /usuarios   | Crear un nuevo usuario    |
| GET         | /usuarios/1 | Obtener usuario con ID 1  |
| PUT         | /usuarios/1 | Actualizar usuario 1      |
| DELETE      | /usuarios/1 | Eliminar usuario 1        |

Este diseño sigue los principios REST, usando recursos y verbos HTTP para operaciones claras y predecibles.

En el desarrollo backend, entender la arquitectura cliente-servidor y los principios REST es fundamental para crear APIs que otros desarrolladores puedan consumir fácilmente. Por ejemplo, cuando construyes un servicio con Node.js y Express, estarás implementando un servidor que expone recursos siguiendo estas reglas.

Este conocimiento te permitirá diseñar APIs que sean intuitivas, escalables y compatibles con herramientas estándar como Postman para pruebas.

Además, respetar el principio de statelessness facilita la escalabilidad horizontal, ya que cada petición es independiente y no requiere que el servidor mantenga información de sesiones.

En la siguiente unidad, aplicaremos estos conceptos para crear un servidor HTTP simple con Node.js, preparando el camino para usar Express y construir un CRUD completo.

## Códigos de estado HTTP: cuándo y por qué

En esta unidad reforzaremos el conocimiento sobre los códigos de estado HTTP, entendiendo sus grupos principales y aprendiendo a seleccionar el código adecuado según la operación y resultado en APIs REST, especialmente en operaciones CRUD con Node y Express.

¿Alguna vez te has preguntado cómo un cliente sabe si su petición a un servidor fue exitosa o si algo salió mal? Los códigos de estado HTTP son la clave para esta comunicación esencial en arquitecturas cliente-servidor. En esta unidad, reforzaremos tu comprensión sobre estos códigos, explorando sus grupos principales y cómo aplicarlos correctamente en tus APIs REST con Node y Express. Al finalizar, podrás seleccionar el código de estado adecuado para cada operación, mejorando la robustez y claridad de tus servicios backend.

### Códigos de estado HTTP: Refuerzo de conceptos

Los **códigos de estado HTTP** son números que el servidor devuelve en respuesta a una petición HTTP para indicar el resultado de la misma. Estos códigos guían al cliente sobre cómo interpretar la respuesta y qué acciones tomar.

### Grupos principales de códigos de estado

| Grupo | Rango   | Significado general                                                  |
|-------|---------|----------------------------------------------------------------------|
| 1xx   | 100-199 | Informativos (poco usados en APIs REST)                              |
| 2xx   | 200-299 | Éxito: la petición fue recibida, entendida y aceptada                |
| 3xx   | 300-399 | Redirección: se requiere acción adicional para completar la petición |
| 4xx   | 400-499 | Error del cliente: la petición contiene errores                      |
| 5xx   | 500-599 | Error del servidor: el servidor falló al procesar la petición        |

#### Nota

En APIs REST, los códigos 2xx, 4xx y 5xx son los más relevantes.

### Ejemplos concretos en operaciones CRUD

- **Crear (POST):**
  - 201 Created : Recurso creado exitosamente.
  - 400 Bad Request : Datos inválidos enviados por el cliente.
- **Leer (GET):**
  - 200 OK : Recurso encontrado y enviado.
  - 404 Not Found : Recurso no existe.
- **Actualizar (PUT/PATCH):**

- `200 OK` o `204 No Content` : Actualización exitosa.
- `400 Bad Request` : Datos inválidos.
- `404 Not Found` : Recurso no encontrado.
- **Eliminar (DELETE):**
  - `200 OK` o `204 No Content` : Eliminación exitosa.
  - `404 Not Found` : Recurso no encontrado.

## Selección adecuada de códigos

Elegir el código correcto mejora la comunicación entre cliente y servidor, facilita el manejo de errores y hace que tu API sea más predecible y profesional.

Por ejemplo, usar `201 Created` tras un POST exitoso indica claramente que se creó un recurso, mientras que un `200 OK` podría ser ambiguo.

**Conoce y aplica estos códigos para que tus APIs comuniquen con precisión el estado de cada operación.**

En el desarrollo backend con Node y Express, manejar correctamente los códigos de estado HTTP es fundamental para construir APIs REST robustas y claras. Cuando implementas un CRUD, cada operación debe devolver un código que refleje fielmente el resultado, permitiendo que clientes (como aplicaciones frontend o herramientas como Postman) interpreten correctamente la respuesta.

Por ejemplo, si un cliente intenta obtener un recurso que no existe, devolver un `404 Not Found` es la forma estándar de indicar este estado, evitando confusiones y facilitando el manejo de errores en el cliente.

Además, estos códigos son esenciales para la depuración y monitoreo de tus servicios, ya que permiten identificar rápidamente problemas de cliente o servidor.

En las siguientes unidades, aprenderás a manejar errores y a probar tus endpoints con Postman, donde el uso correcto de estos códigos será clave para validar el comportamiento esperado de tus APIs.

## Inicializar proyecto y gestionar dependencias (npm / pnpm)

Aprende a crear y configurar un proyecto Node.js desde cero usando npm o pnpm, instalar dependencias esenciales como Express y nodemon, y gestionar scripts útiles para un flujo de desarrollo eficiente.

¿Alguna vez has querido iniciar un proyecto Node.js de forma rápida y organizada, sin perder tiempo en configuraciones manuales? En este módulo, descubrirás cómo inicializar un proyecto usando las herramientas más comunes: npm y pnpm. Aprenderás a instalar dependencias clave como Express para crear servidores y nodemon para mejorar tu flujo de desarrollo con recargas automáticas. Además, configurarás scripts en el archivo `package.json` que te permitirán ejecutar comandos fre-

cuentas con facilidad. Este conocimiento es fundamental para avanzar en la creación de aplicaciones backend eficientes y prepararte para construir APIs REST completas.

## Inicializar un proyecto Node.js con npm y pnpm

Para comenzar un proyecto Node.js, es esencial crear un archivo `package.json` que gestione las dependencias y scripts. Esto se logra con los comandos:

- `npm init` o `npm init -y` (para aceptar valores por defecto)
- `pnpm init` o `pnpm init -y`

Ambos generan un `package.json` básico que puedes modificar según tus necesidades.

## Instalación de dependencias

Para instalar paquetes, usamos:

- `npm install <paquete>` o `pnpm add <paquete>` para dependencias de producción.
- `npm install --save-dev <paquete>` o `pnpm add -D <paquete>` para dependencias de desarrollo.

Ejemplo para instalar Express y nodemon:

### CÓDIGO

```
npm install express
npm install --save-dev nodemon
```

O con pnpm:

### CÓDIGO

```
pnpm add express
pnpm add -D nodemon
```

## Scripts en package.json

Los scripts permiten ejecutar comandos personalizados con `npm run <script>` o `pnpm run <script>`. Un ejemplo común es usar nodemon para desarrollo:

### CÓDIGO

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

- `start`: ejecuta la aplicación normalmente.

- `dev` : ejecuta la aplicación con nodemon, que reinicia el servidor automáticamente al detectar cambios.

## Buenas prácticas en flujo de desarrollo

- Mantén una estructura clara: carpeta `src` para código fuente, `index.js` o `app.js` como punto de entrada.
- Usa scripts para tareas repetitivas.
- Prefiere nodemon en desarrollo para acelerar el ciclo de prueba.

### Nota

npm es el gestor de paquetes oficial de Node.js, mientras que pnpm es una alternativa más rápida y eficiente en espacio, pero ambos son ampliamente usados en la industria.

En el desarrollo backend, iniciar un proyecto correctamente es clave para mantener la productividad y escalabilidad. Usar npm o pnpm para gestionar dependencias y scripts te permite automatizar tareas comunes y evitar errores manuales. Por ejemplo, nodemon es una herramienta estándar que mejora el flujo de trabajo al reiniciar el servidor automáticamente, ahorrando tiempo y facilitando pruebas rápidas.

Este conocimiento es la base para construir APIs REST con Express, que veremos en unidades posteriores. Además, configurar scripts claros en `package.json` es una práctica profesional que facilita la colaboración en equipos y la integración continua.

# Motores de Plantillas y Routers en Express

---

Inicia tu camino en el desarrollo backend con Node.js y Express aprendiendo a generar contenido HTML dinámico en el servidor. Este módulo introduce los motores de plantillas, enfocándose en Handlebars, y te guía en su configuración dentro de una aplicación Express. Descubre cómo crear layouts reutilizables y vistas dinámicas que facilitan la construcción de interfaces escalables y mantenibles. Además, reforzaremos conceptos clave como *partials* y el renderizado del lado del servidor para sentar una base sólida que permitirá implementar lógica avanzada, modularización y carga de archivos en los siguientes módulos.

## Integración con vistas

En esta unidad aprenderás a integrar routers en Express con el motor de plantillas Handlebars para renderizar vistas HTML dinámicas. Veremos cómo pasar datos desde los routers a las vistas, manteniendo una clara separación entre la lógica de negocio y la presentación, lo que mejora la mantenibilidad y escalabilidad de tus aplicaciones backend.

¿Alguna vez te has preguntado cómo un servidor Express puede generar páginas HTML dinámicas que cambian según los datos que maneja? En esta unidad, abordaremos cómo los routers en Express pueden integrarse con el motor de plantillas Handlebars para renderizar vistas que reflejen información dinámica, como listas de productos, perfiles de usuario o resultados de búsqueda.

Esta integración es clave para construir aplicaciones backend robustas y mantenibles, ya que permite separar claramente la lógica de negocio (en los routers y controladores) de la presentación (en las vistas). Al finalizar, serás capaz de crear routers que no solo gestionen rutas, sino que también entreguen vistas HTML con datos personalizados, mejorando la experiencia del usuario y la organización de tu código.

Conectaremos lo aprendido en la unidad anterior sobre routers y reforzaremos la configuración de Handlebars en Express, para luego aplicar estos conocimientos en ejemplos prácticos y ejercicios que consolidarán tu aprendizaje.

## Integración Handlebars con Express (Refuerzo)

Express facilita la integración con motores de plantillas como Handlebars para generar HTML dinámico. Para configurar Handlebars en Express, se deben establecer las siguientes propiedades:

- `app.set('view engine', 'hbs')`: Define Handlebars como motor de vistas.
  - `app.set('views', path.join(__dirname, 'views'))`: Indica la carpeta donde se almacenan las plantillas.
  - Registro de *helpers* personalizados para lógica sencilla en las vistas.
-

#### Nota

Handlebars es un motor de plantillas *logic-less*, lo que significa que la lógica compleja debe manejarse en el backend, manteniendo las vistas limpias y enfocadas en la presentación.

## Integración de vistas con routers (Aplicación)

Cada router en Express puede renderizar vistas específicas usando el método `res.render()`. Este método recibe dos argumentos principales:

1. El nombre de la plantilla a renderizar (sin extensión).
2. Un objeto con los datos dinámicos que se pasarán a la plantilla.

#### CÓDIGO

```
router.get('/perfil', (req, res) => {  
  const usuario = { nombre: 'Ana', edad: 28 };  
  res.render('perfil', { usuario });  
});
```

En este ejemplo, la vista `perfil.hbs` podrá acceder a las propiedades de `usuario` para mostrar información personalizada.

## Separación entre lógica de negocio y presentación

- **Lógica de negocio:** Se maneja en los routers o controladores, donde se obtienen y procesan los datos.
- **Presentación:** Se define en las vistas Handlebars, que solo deben encargarse de mostrar los datos recibidos.

Esta separación mejora la mantenibilidad y facilita la colaboración entre desarrolladores y diseñadores.

### Ejemplo de estructura de proyecto

| Carpeta/Archivo                     | Descripción                            |
|-------------------------------------|----------------------------------------|
| <code>/routes/productos.js</code>   | Router que maneja rutas de productos   |
| <code>/views/productos.hbs</code>   | Plantilla para mostrar productos       |
| <code>/helpers/handlebars.js</code> | Helpers personalizados para Handlebars |

**Conocimiento clave:** Para pasar datos dinámicos a las vistas, siempre prepara los datos en el router y pásalos como objeto a `res.render()`.

En el desarrollo backend con Node.js y Express, es común que las aplicaciones necesiten mostrar contenido HTML que varíe según la información disponible, como perfiles de usuario, listados o



resultados de formularios. La integración de routers con motores de plantillas como Handlebars permite generar estas vistas dinámicas de forma eficiente.

Por ejemplo, en una tienda en línea, un router puede obtener la lista de productos desde una base de datos o un controlador simulado, y luego renderizar una vista que muestre esos productos al usuario. Esta integración es fundamental para mantener el código organizado, ya que la lógica para obtener y preparar los datos se mantiene separada de la lógica para mostrar esos datos.

Además, esta práctica facilita la escalabilidad y el mantenimiento del proyecto, ya que los cambios en la presentación no afectan la lógica de negocio y viceversa. En esta unidad, prepararemos el terreno para que puedas implementar esta integración en tus proyectos reales, usando ejemplos claros y aplicables a escenarios comunes en la industria del desarrollo backend.

## Introducción a motores de plantillas

Este contenido introduce a los estudiantes en el concepto de motores de plantillas, explicando su función en aplicaciones backend con Node.js y Express. Se abordan los beneficios del renderizado del lado del servidor (SSR), se comparan con las aplicaciones de una sola página (SPA) y se presentan ejemplos relevantes como Handlebars, Pug y EJS. El objetivo es que los estudiantes comprendan cuándo y por qué usar motores de plantillas para generar vistas HTML dinámicas desde el backend.

### ¿Por qué necesitamos motores de plantillas en desarrollo backend?

Imagina que estás construyendo una aplicación web que debe mostrar información dinámica, como perfiles de usuario, listas de productos o resultados de búsqueda. ¿Cómo generas el HTML que el navegador mostrará, sin tener que escribir manualmente cada página para cada usuario o situación? Aquí es donde entran en juego los **motores de plantillas**.

En esta unidad, exploraremos qué son los motores de plantillas, cómo facilitan la generación de contenido HTML dinámico desde el servidor y por qué son una herramienta esencial en el desarrollo backend con Node.js y Express. También compararemos el renderizado del lado del servidor (SSR) con las aplicaciones de una sola página (SPA), para entender cuándo es más conveniente usar cada enfoque.

Al finalizar, podrás explicar claramente qué es un motor de plantillas y cuándo es apropiado utilizarlo, sentando las bases para configurar Handlebars en las próximas unidades.

### ¿Qué es un motor de plantillas?

Un **motor de plantillas** es una herramienta que permite generar código HTML dinámico en el servidor combinando plantillas predefinidas con datos específicos. En lugar de escribir páginas HTML estáticas, se crean plantillas con marcadores de posición que luego se reemplazan con información real en tiempo de ejecución.

### ¿Cómo funciona?

1. **Plantilla:** Archivo con estructura HTML y marcadores para datos dinámicos.
2. **Datos:** Información que se quiere mostrar, como nombres, listas o resultados.
3. **Renderizado:** El motor de plantillas procesa la plantilla y los datos para generar HTML completo.

### Beneficios clave:

- **Reutilización:** Las plantillas pueden usarse para múltiples páginas o secciones.
- **Separación de responsabilidades:** Diseñadores trabajan en plantillas, desarrolladores en lógica y datos.
- **Mantenimiento simplificado:** Cambios en la presentación no afectan la lógica backend.

### Ejemplos populares en Node.js:

| Motor de Plantillas | Características principales                       |
|---------------------|---------------------------------------------------|
| Handlebars          | Sintaxis sencilla, soporta layouts y helpers      |
| Pug                 | Sintaxis indentada, muy concisa                   |
| EJS                 | Similar a HTML con etiquetas JavaScript embebidas |

#### Nota

En este programa nos enfocaremos en Handlebars por su equilibrio entre simplicidad y potencia.

## Renderizado del lado del servidor (SSR) vs Aplicaciones de una sola página (SPA)

### Renderizado del lado del servidor (SSR)

- El servidor genera y envía páginas HTML completas al navegador.
- Ideal para SEO, tiempos de carga inicial rápidos y compatibilidad amplia.
- Los motores de plantillas son la base para SSR en aplicaciones Express.

### Aplicaciones de una sola página (SPA)

- El navegador carga una única página HTML y actualiza contenido dinámicamente usando JavaScript.
- Mayor interactividad y experiencia de usuario fluida.
- Requiere más lógica en el cliente y a menudo APIs REST o GraphQL en el backend.

### ¿Cuándo preferir SSR?

- Cuando la prioridad es el SEO y el acceso rápido a contenido.
- Proyectos con contenido principalmente estático o con cambios moderados.
- Cuando se busca simplicidad en la arquitectura backend.

### ¿Cuándo preferir SPA?

- Aplicaciones altamente interactivas y dinámicas.
- Experiencias similares a aplicaciones nativas.
- Cuando se dispone de recursos para manejar lógica compleja en frontend.

**Reflexión:** Entender estas diferencias te ayudará a elegir la mejor estrategia para tu proyecto y a implementar motores de plantillas de forma efectiva.

En el desarrollo backend con Node.js y Express, los motores de plantillas son una solución práctica para generar vistas HTML dinámicas que se envían al cliente. Por ejemplo, una tienda en línea puede usar Handlebars para mostrar productos personalizados según el usuario que accede, sin necesidad de recargar toda la página o depender exclusivamente de JavaScript en el cliente.

Además, el renderizado del lado del servidor mejora la experiencia inicial del usuario y es fundamental para aplicaciones que requieren buen posicionamiento en buscadores (SEO). En contraste, las SPAs delegan gran parte del renderizado al navegador, lo que puede ser más complejo y no siempre necesario.

Esta unidad prepara el terreno para que en las siguientes se configure Handlebars como motor de vistas en Express, creando layouts reutilizables y vistas dinámicas que aprovechen estas ventajas.

## Configuración de Handlebars

En esta unidad aprenderás a configurar Handlebars como motor de plantillas en una aplicación Express, entenderás la sintaxis básica para incrustar datos en vistas y cómo registrar helpers y partials para crear vistas dinámicas y reutilizables.

¿Alguna vez te has preguntado cómo las aplicaciones web generan páginas HTML dinámicas desde el servidor? En el desarrollo backend con Node.js y Express, los motores de plantillas son la clave para lograrlo. En esta unidad, nos enfocaremos en Handlebars, un motor de plantillas popular y eficiente que permite crear vistas dinámicas y reutilizables.

Partiendo de la base que ya conoces Express y su estructura básica, aprenderás a instalar y configurar Handlebars para que tu aplicación pueda renderizar vistas HTML dinámicas. Además, exploraremos la sintaxis básica de Handlebars para incrustar datos en las vistas y cómo extender su funcionalidad mediante helpers y partials.

Al finalizar esta unidad, estarás listo para crear aplicaciones Express con vistas dinámicas bien organizadas y preparadas para escalar, sentando las bases para implementar lógica más avanzada y modularización en las siguientes unidades.

### Instalación y Configuración de Handlebars en Express

Para integrar Handlebars en una aplicación Express, primero instalamos los paquetes necesarios:

#### CÓDIGO

```
npm install express-handlebars
```

Express no incluye Handlebars por defecto, por lo que usamos el paquete `express-handlebars` que actúa como adaptador.

### Configuración en `app.js`

En el archivo principal de la aplicación ( `app.js` ), configuramos Handlebars como motor de vistas:

#### CÓDIGO

```
const express = require('express');
const exphbs = require('express-handlebars');

const app = express();

// Configurar Handlebars como motor de vistas
app.engine('handlebars', exphbs.engine({
  defaultLayout: 'main',
  helpers: {
    // Aquí se registran helpers personalizados
  }
}));
app.set('view engine', 'handlebars');
app.set('views', './views');
```

- `app.engine` define el motor de plantillas y sus opciones.
- `defaultLayout` es el layout base que envuelve las vistas.
- `app.set('view engine', 'handlebars')` indica a Express que use Handlebars.
- `app.set('views', './views')` define la carpeta donde se almacenan las vistas.

### Sintaxis básica de Handlebars

Handlebars utiliza expresiones delimitadas por `{{}}` para incrustar datos y lógica simple en las vistas:

- `{{variable}}`: Inserta el valor de `variable` escapando HTML para seguridad.
- `{{{variable}}}`: Inserta el valor sin escapar (usar con precaución).
- `{{#if condicion}} ... {{/if}}`: Estructura condicional para mostrar contenido.

Ejemplo simple:

#### CÓDIGO

```
<h1>Hola, {{nombre}}!</h1>
{{#if esAdmin}}
  <p>Bienvenido, administrador.</p>
{{else}}
  <p>Bienvenido, usuario.</p>
{{/if}}
```

## Helpers y Partials

- **Helpers:** funciones que extienden la lógica en las vistas. Por ejemplo, un helper para formatear fechas.
- **Partials:** fragmentos de vistas reutilizables, como cabeceras o pies de página.

Registro de helpers y partials:

#### CÓDIGO

```
const hbs = exphbs.create({
  helpers: {
    mayusculas: (texto) => texto.toUpperCase()
  },
  partialsDir: ['./views/partial/']
});

app.engine('handlebars', hbs.engine);
```

Luego, en las vistas:

#### CÓDIGO

```
{{mayusculas nombre}}
{{> header}}
```

### Nota

La estructura de carpetas recomendada es:

#### CÓDIGO

```
/views  
/layouts  
main.handlebars  
/partials  
header.handlebars  
footer.handlebars  
home.handlebars
```

Esta organización facilita la mantenibilidad y escalabilidad de la aplicación.

En proyectos backend reales, la capacidad de generar vistas HTML dinámicas es fundamental para entregar contenido personalizado y mejorar la experiencia del usuario. Handlebars, al ser un motor de plantillas logic-less, permite separar la lógica de negocio del diseño, facilitando el mantenimiento y la colaboración entre desarrolladores y diseñadores.

Configurar correctamente Handlebars en Express es el primer paso para construir aplicaciones robustas que puedan escalar y adaptarse a requerimientos cambiantes. La sintaxis sencilla y la posibilidad de extender funcionalidades con helpers y partials hacen que Handlebars sea una opción popular en la industria.

Además, una estructura de carpetas bien definida para vistas, layouts y partials es clave para mantener el código organizado y facilitar futuras ampliaciones o modificaciones.

Esta unidad te prepara para implementar estas buenas prácticas, que serán la base para manejar rutas modulares y lógica avanzada en las siguientes unidades del módulo.

## Condicionales if

Aprende a usar condicionales en plantillas Handlebars para mostrar u ocultar contenido dinámico según los datos del backend, y cómo implementar helpers personalizados para condiciones más complejas, manteniendo vistas limpias y eficientes.

¿Alguna vez has necesitado mostrar contenido diferente en tu aplicación web según ciertas condiciones, como si un usuario está autenticado o si una lista está vacía? En el desarrollo backend con Node.js y Express, Handlebars es una herramienta clave para generar vistas HTML dinámicas que respondan a estos escenarios. En esta unidad, exploraremos cómo usar las condicionales `{{#if}}` y `{{else}}` en Handlebars para controlar la presentación de contenido según los datos que envía el backend.

Además, aprenderás a crear helpers personalizados que te permitirán manejar condiciones más complejas sin sobrecargar tus plantillas, manteniendo el código limpio y fácil de mantener. Este conocimiento es fundamental para construir interfaces dinámicas y profesionales, y te preparará para integrar lógica de presentación avanzada en tus proyectos backend.

Al finalizar, podrás aplicar condicionales básicas y avanzadas en tus vistas, mejorando la experiencia del usuario y la eficiencia de tu código. ¡Vamos a comenzar!

## Condicionales en Handlebars: Sintaxis Básica

Handlebars es un motor de plantillas **logic-less**, lo que significa que evita lógica compleja en las vistas para mantenerlas limpias y legibles. Sin embargo, permite estructuras condicionales básicas para mostrar u ocultar contenido según los datos.

### Sintaxis de `{{#if}}` y `{{else}}`

La estructura básica para condicionales es:

#### CÓDIGO

```
{{#if condición}}
  <!-- Contenido si la condición es verdadera -->
{{else}}
  <!-- Contenido si la condición es falsa -->
{{/if}}
```

- `condición` puede ser una variable booleana o cualquier valor que Handlebars evalúe como verdadero o falso.
- El bloque dentro de `{{#if}}` se renderiza si la condición es verdadera.
- El bloque dentro de `{{else}}` se renderiza si la condición es falsa.

### Ejemplo básico

#### CÓDIGO

```
{{#if isLoggedIn}}
  <p>Bienvenido, {{username}}!</p>
{{else}}
  <p>Por favor, inicia sesión.</p>
{{/if}}
```

Si `isLoggedIn` es `true`, se muestra el mensaje de bienvenida; si no, se invita a iniciar sesión.

## Helpers Condicionales Personalizados

Cuando las condiciones son más complejas que una simple variable booleana, puedes crear **helpers personalizados** en Express para extender la lógica de Handlebars.

## Definición de un helper

En tu archivo de configuración de Express:

### CÓDIGO

```
const hbs = require('hbs');

hbs.registerHelper('ifEquals', function(arg1, arg2, options) {
  return (arg1 == arg2) ? options.fn(this) : options.inverse(this);
});
```

## Uso en plantilla

### CÓDIGO

```
{{#ifEquals user.role 'admin'}}
  <p>Panel de administración</p>
{{else}}
  <p>Acceso restringido</p>
{{/ifEquals}}
```

Este helper compara dos valores y muestra contenido según el resultado.

## Buenas Prácticas

- **Evita lógica compleja en las vistas:** Mantén la lógica en el backend y pasa datos ya procesados.
- **Usa helpers para condiciones repetitivas o complejas:** Facilita mantenimiento y legibilidad.
- **No anides condicionales excesivamente:** Prefiere preparar datos en el backend para simplificar las plantillas.

### Nota

Handlebars escapa automáticamente el contenido para evitar vulnerabilidades XSS, pero siempre valida y sanitiza datos en el backend.

**Con este conocimiento, estás listo para aplicar condicionales básicas y personalizadas en tus vistas Handlebars.**

En aplicaciones backend con Node.js y Express, la presentación dinámica es clave para ofrecer experiencias personalizadas y eficientes. Por ejemplo, mostrar diferentes menús según el rol del usuario o mensajes específicos cuando no hay datos disponibles.

Handlebars permite integrar esta lógica de presentación directamente en las vistas, pero con un enfoque minimalista para evitar sobrecargar las plantillas. La responsabilidad principal de preparar los datos recae en los routers y controladores del backend, que deben enviar a las vistas la información ya estructurada para facilitar el renderizado.



Al dominar las condicionales `{{#if}}` y los helpers personalizados, podrás crear interfaces que respondan a múltiples escenarios sin sacrificar la claridad del código. Esto es especialmente útil en proyectos reales donde la lógica de negocio puede ser compleja y la mantenibilidad es crucial.

En la siguiente sección, pondremos en práctica estos conceptos con ejemplos concretos y ejercicios que reflejan situaciones comunes en desarrollo backend con Express y Handlebars.

## Iteraciones each

Aprende a utilizar la estructura de control `{{#each}}` en Handlebars para iterar colecciones y generar listas dinámicas en tus vistas. Además, descubre cómo crear *partials* para reutilizar fragmentos de plantilla y mantener tu código limpio y modular.

¿Alguna vez has necesitado mostrar una lista dinámica de productos, usuarios o cualquier conjunto de datos en tu aplicación Express? Handlebars ofrece la estructura de control `{{#each}}` que facilita recorrer arrays o colecciones y generar contenido HTML dinámico y ordenado. En esta unidad, descubrirás cómo usar `{{#each}}` para iterar sobre datos enviados desde el backend, acceder a propiedades específicas de cada elemento y manejar índices para mejorar la presentación.

Además, aprenderás a crear *partials*, fragmentos reutilizables de plantilla que te ayudarán a mantener tus vistas limpias y modulares, evitando la repetición de código. Esta habilidad es clave para desarrollar aplicaciones backend escalables y mantenibles con Node.js y Express.

Al finalizar, podrás construir listas dinámicas con Handlebars y organizar tu código de manera profesional, conectando la lógica de presentación con los datos que provienen del backend, tal como se espera en proyectos reales de desarrollo backend.

### Iteraciones con `{{#each}}` en Handlebars

La expresión `{{#each}}` es una estructura de control que permite recorrer un array o colección y renderizar un bloque de código para cada elemento.

#### CÓDIGO

```
<ul>
  {{#each items}}
    <li>{{this.name}} - {{this.price}}</li>
  {{/each}}
</ul>
```

- `items` es el array enviado desde el backend.
- `this` representa el elemento actual en la iteración.

### Acceso a índice con `@index`

Dentro de `{{#each}}`, puedes acceder al índice del elemento actual usando `@index`:

#### CÓDIGO

```
{{#each items}}  
  <p>{{@index}}: {{this.name}}</p>  
{{/each}}
```

Esto es útil para numerar listas o aplicar estilos alternos.

### Contexto dentro de each

El contexto dentro de `{{#each}}` cambia al elemento actual, por eso usamos `this` para acceder a sus propiedades. Si necesitas acceder a variables fuera del `each`, puedes usar `../` para subir un nivel en el contexto.

### Partials: Reutilización de fragmentos

Los *partials* son plantillas parciales que puedes definir y luego incluir en otras vistas para evitar repetir código.

Definición de un partial llamado `productCard`:

#### CÓDIGO

```
<!-- partials/productCard.handlebars -->  
<div class="card">  
  <h3>{{name}}</h3>  
  <p>Precio: {{price}}</p>  
</div>
```

Uso dentro de una vista principal:

#### CÓDIGO

```
{{#each products}}  
  {{> productCard}}  
{{/each}}
```

Esto hace que cada producto se renderice usando el mismo diseño, facilitando mantenimiento y escalabilidad.

#### Nota

Los partials deben registrarse en el backend para que Handlebars los reconozca.

**Resumen:** `{{#each}}` permite iterar colecciones para generar contenido dinámico, mientras que los partials ayudan a mantener el código limpio y modular.

En aplicaciones backend con Node.js y Express, es común enviar datos desde el servidor a las vistas para mostrarlos dinámicamente. Por ejemplo, una lista de usuarios, productos o mensajes.

Usar `{{#each}}` en Handlebars permite recorrer estos datos y generar el HTML necesario sin escribir código repetitivo o estático. Esto mejora la experiencia del usuario al mostrar información actualizada y personalizada.

Además, al combinar `{{#each}}` con `partials`, puedes crear componentes reutilizables como tarjetas de producto, filas de tabla o elementos de menú, que se mantienen consistentes y fáciles de modificar.

Este enfoque es fundamental para proyectos reales donde la presentación debe ser dinámica, eficiente y mantenible, alineándose con buenas prácticas de desarrollo backend.

En la siguiente sección, aplicarás estos conceptos creando una lista dinámica con `partials` para reforzar tu aprendizaje.

## Renderizado dinámico de listas

En esta unidad aprenderás a diseñar vistas que muestran listas dinámicas utilizando Handlebars en aplicaciones Express. Cubriremos patrones para renderizar colecciones, manejo de casos especiales como listas vacías, paginación simple y el uso de plantillas parciales para mejorar la modularidad y reutilización. Además, reforzaremos el concepto de renderizado del lado del servidor (SSR) para entender cómo se genera y entrega el HTML dinámico al cliente.

¿Alguna vez has visitado una tienda en línea y te has preguntado cómo el sitio muestra listas de productos que cambian constantemente? En esta unidad, exploraremos cómo construir vistas dinámicas que muestran colecciones de datos, como productos, entradas o usuarios, utilizando Handlebars en aplicaciones Express. Aprenderás a manejar situaciones comunes como listas vacías y a implementar paginación simple para mejorar la experiencia del usuario. Además, reforzaremos cómo el renderizado del lado del servidor (SSR) genera estas vistas dinámicas y las entrega al cliente. Al finalizar, estarás capacitado para diseñar interfaces que se adapten a datos variables y a preparar tus aplicaciones para escenarios reales de producción.

### Renderizado dinámico de listas con Handlebars

El **renderizado dinámico de listas** consiste en mostrar colecciones de datos en la interfaz de usuario, donde el contenido puede variar según la información que proviene del backend.

#### Uso de `{{#each}}` para iterar colecciones

En Handlebars, la estructura de control `{{#each}}` permite recorrer arrays y renderizar un bloque de código para cada elemento.

#### CÓDIGO

```
<ul>
  {{#each productos}}
    <li>{{this.nombre}} - ${{this.precio}}</li>
  {{/each}}
</ul>
```

- `productos` es un array de objetos enviado desde el backend.
- `this` representa el elemento actual en la iteración.

### Manejo de listas vacías con `{{else}}`

Para evitar mostrar listas vacías, Handlebars permite usar `{{else}}` dentro de `each` :

#### CÓDIGO

```
<ul>
  {{#each productos}}
    <li>{{this.nombre}} - ${{this.precio}}</li>
  {{else}}
    <li>No hay productos disponibles.</li>
  {{/each}}
</ul>
```

Esto mejora la experiencia del usuario al informar cuando no hay datos.

### Claves únicas y rendimiento

Aunque Handlebars no requiere explícitamente claves únicas como React, es buena práctica asegurar que cada elemento tenga un identificador único para evitar confusiones en la lógica del backend y facilitar futuras mejoras.

### Paginación simple

La paginación divide una lista larga en páginas más pequeñas. En el backend, se envía solo un subconjunto de datos y en la vista se muestran controles para navegar.

Ejemplo de controles básicos:

#### CÓDIGO

```
<nav>
  {{#if paginaAnterior}}
  <a href="?pagina={{paginaAnterior}}">Anterior</a>
  {{/if}}
  <span>Página {{paginaActual}} de {{totalPaginas}}</span>
  {{#if paginaSiguiente}}
  <a href="?pagina={{paginaSiguiente}}">Siguiente</a>
  {{/if}}
</nav>
```

## Plantillas parciales para modularidad

Las **plantillas parciales** permiten reutilizar fragmentos de código Handlebars, facilitando el mantenimiento.

Definición de un parcial `producto.hbs` :

#### CÓDIGO

```
<li>{{nombre}} - ${{precio}}</li>
```

Uso en la plantilla principal:

#### CÓDIGO

```
<ul>
  {{#each productos}}
  {{> producto}}
  {{else}}
  <li>No hay productos disponibles.</li>
  {{/each}}
</ul>
```

## Reforzando el Renderizado del Lado del Servidor (SSR)

El SSR genera el HTML completo en el servidor usando plantillas como Handlebars y lo envía al cliente. Esto mejora el SEO y la velocidad de carga inicial.

### Nota

En unidades anteriores aprendiste cómo Express integra Handlebars para renderizar vistas dinámicas. Aquí aplicamos ese conocimiento para listas complejas y casos especiales.

En aplicaciones backend con Node.js y Express, es común manejar colecciones de datos que deben mostrarse en la interfaz de usuario. Por ejemplo, un e-commerce debe listar productos, un blog mostrar entradas o un sistema administrativo listar usuarios.

El backend prepara estos datos y los pasa a las vistas Handlebars para que se rendericen dinámicamente. Sin embargo, no siempre las listas tienen elementos: puede que no haya productos disponibles o que el usuario esté en una página sin resultados. Por eso, es fundamental manejar estos casos para evitar interfaces vacías o confusas.

Además, cuando las listas son muy largas, la paginación mejora la usabilidad y el rendimiento, mostrando solo un subconjunto de elementos y permitiendo navegar entre páginas.

Finalmente, el uso de plantillas parciales ayuda a mantener el código limpio y reutilizable, especialmente cuando se repiten estructuras similares en diferentes vistas.

Esta unidad te prepara para implementar estas prácticas en tus proyectos backend, asegurando que las vistas sean dinámicas, claras y eficientes.

## Configuración de Multer

Aprende a instalar y configurar Multer para gestionar la carga de archivos en aplicaciones Express. Este contenido te guiará en la configuración de opciones de almacenamiento, límites de tamaño y la integración de Multer como middleware en rutas específicas, preparando el terreno para manejar archivos de forma segura y eficiente.

¿Alguna vez te has preguntado cómo las aplicaciones web manejan la carga de archivos de manera eficiente y segura? En el desarrollo backend con Node.js y Express, gestionar archivos subidos por los usuarios es una tarea común pero que requiere atención especial para evitar problemas de seguridad y rendimiento. En esta unidad, descubrirás cómo Multer, un middleware especializado para Express, facilita este proceso al ofrecer una configuración flexible para almacenar archivos y controlar sus características.

A lo largo de esta sesión, aprenderás a instalar Multer, configurar sus opciones de almacenamiento con `diskStorage`, establecer límites de tamaño para proteger tu aplicación y cómo integrarlo correctamente en rutas específicas usando middleware. Este conocimiento es fundamental para construir aplicaciones robustas que manejen archivos de forma profesional y segura, alineado con las mejores prácticas de la industria.

Al finalizar, estarás preparado para implementar Multer en tus proyectos, sentando las bases para la validación avanzada y gestión segura de archivos que veremos en la siguiente unidad.

### Multer: Middleware para la carga de archivos en Express

Multer es un middleware para Express que procesa solicitudes con contenido `multipart/form-data`, comúnmente usadas para subir archivos. Su función principal es facilitar la recepción y almacenamiento de archivos enviados desde formularios HTML.

### Instalación básica

Para comenzar a usar Multer, primero debes instalarlo en tu proyecto:

#### CÓDIGO

```
npm install multer
```

Luego, se importa en el archivo principal de tu servidor:

#### CÓDIGO

```
const multer = require('multer');
```

## Opciones de almacenamiento (storage)

Multer permite configurar dónde y cómo se almacenan los archivos subidos. La opción más común es `diskStorage`, que guarda los archivos en el sistema de archivos del servidor.

#### CÓDIGO

```
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'uploads/'); // Carpeta destino
  },
  filename: function (req, file, cb) {
    cb(null, Date.now() + '-' + file.originalname); // Nombre único
  }
});

const upload = multer({ storage: storage });
```

- `destination` : función que define la carpeta donde se guardan los archivos.
- `filename` : función que define el nombre con el que se guarda el archivo, evitando colisiones.

## Límites (limits)

Para proteger la aplicación, es importante limitar el tamaño y cantidad de archivos:

#### CÓDIGO

```
const upload = multer({
  storage: storage,
  limits: { fileSize: 1024 * 1024 * 5 } // 5MB máximo
});
```

Esto evita que se suban archivos demasiado grandes que puedan afectar el rendimiento o seguridad.

## Middleware para rutas específicas

Multer se integra como middleware en las rutas donde se reciben archivos:

- `upload.single('campo')` : para un solo archivo con nombre de campo específico.

- `upload.array('campo', maxCount)` : para múltiples archivos.
- `upload.fields([ { name: 'campo1', maxCount: 1 }, { name: 'campo2', maxCount: 3 } ])` : para múltiples campos con archivos.

Ejemplo:

#### CÓDIGO

```
app.post('/upload', upload.single('avatar'), (req, res) => {  
  res.send('Archivo recibido');  
});
```

#### Nota

Multer procesa el archivo antes de que llegue al controlador, agregando la información en `req.file` o `req.files`.

**Con este conocimiento, estás listo para configurar Multer en tus proyectos y controlar la carga de archivos de forma segura y eficiente.**

En el desarrollo backend, la gestión de archivos subidos es crítica para aplicaciones que manejan imágenes, documentos o cualquier tipo de contenido generado por usuarios. Multer se posiciona como la solución estándar en el ecosistema Express para esta tarea, ofreciendo flexibilidad y control.

Configurar correctamente Multer no solo implica instalarlo, sino también definir dónde se almacenan los archivos, cómo se nombran para evitar conflictos y establecer límites para proteger la aplicación. Además, integrarlo como middleware en rutas específicas permite modularizar la lógica y mantener el código limpio y mantenible.

Por ejemplo, en una aplicación de gestión de perfiles, podrías tener una ruta `/profile/avatar` que solo acepte un archivo de imagen para el avatar del usuario, mientras que otra ruta `/documents/upload` podría aceptar múltiples archivos PDF. Multer facilita esta diferenciación mediante su configuración middleware.

Esta unidad te prepara para implementar estas configuraciones básicas, que son la base para aplicar validaciones avanzadas y buenas prácticas de seguridad en la siguiente unidad del módulo.

## Validación y manejo de errores

En esta unidad aprenderás a validar el tipo y tamaño de archivos subidos con Multer, manejar errores comunes durante la carga y aplicar buenas prácticas para el almacenamiento seguro y la limpieza de archivos temporales. Esto te permitirá construir aplicaciones Express robustas y seguras que gestionen archivos de forma profesional.

¿Alguna vez has subido un archivo a una aplicación web y recibiste un error inesperado o el archivo no era el esperado? En el desarrollo backend con Node.js y Express, manejar la carga de archivos no solo



implica recibirlos, sino también validar que cumplan con los requisitos de tipo y tamaño, y gestionar adecuadamente los errores que puedan surgir. Esta unidad te guiará para implementar validaciones efectivas y manejar errores de forma profesional, asegurando que tu aplicación sea segura y confiable.

Partiremos de la configuración básica de Multer vista en la unidad anterior para introducir validaciones específicas y técnicas para manejar errores, además de reforzar buenas prácticas de seguridad en el almacenamiento y limpieza de archivos temporales. Al finalizar, estarás capacitado para proteger tu aplicación contra cargas indebidas y responder con mensajes claros y adecuados a los usuarios.

## Validación y manejo de errores en carga de archivos con Multer

### Validación de tipo MIME y extensión

Para garantizar que solo se acepten archivos permitidos, es fundamental validar el tipo MIME y la extensión del archivo. Multer permite implementar esta validación mediante la función `fileFilter`.

#### CÓDIGO

```
const multer = require('multer');

const fileFilter = (req, file, cb) => {
  const allowedMimeTypes = ['image/jpeg', 'image/png', 'application/pdf'];
  if (allowedMimeTypes.includes(file.mimetype)) {
    cb(null, true); // Aceptar archivo
  } else {
    cb(new Error('Tipo de archivo no permitido'), false); // Rechazar archivo
  }
};

const upload = multer({
  dest: 'uploads/',
  fileFilter: fileFilter
});
```

#### Nota

Validar solo la extensión del archivo no es suficiente, ya que puede ser manipulada. La validación del tipo MIME es más confiable.

### Validación de tamaño máximo

Para evitar cargas excesivas que puedan afectar el rendimiento o seguridad, Multer permite limitar el tamaño máximo del archivo con la opción `limits`.

#### CÓDIGO

```
const upload = multer({
  dest: 'uploads/',
  limits: { fileSize: 2 * 1024 * 1024 } // 2MB
});
```

Si un archivo supera este límite, Multer generará un error que debe ser manejado.

## Manejo de errores y respuestas al cliente

Los errores generados por Multer, como tipo no permitido o tamaño excedido, deben ser capturados y respondidos adecuadamente para mejorar la experiencia del usuario.

#### CÓDIGO

```
app.post('/upload', (req, res) => {
  upload.single('file')(req, res, (err) => {
    if (err instanceof multer.MulterError) {
      // Error específico de Multer
      return res.status(400).json({ error: err.message });
    } else if (err) {
      // Otro tipo de error
      return res.status(400).json({ error: err.message });
    }
    // Archivo cargado correctamente
    res.json({ message: 'Archivo subido con éxito' });
  });
});
```

## Seguridad y almacenamiento de archivos

### Renombrado seguro

Para evitar colisiones y posibles ataques, es recomendable renombrar los archivos subidos con nombres únicos, por ejemplo usando timestamps o UUIDs.

#### CÓDIGO

```
const path = require('path');
const { v4: uuidv4 } = require('uuid');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, uuidv4() + ext);
  }
});

const upload = multer({ storage: storage, fileFilter: fileFilter, limits:
{ fileSize: 2 * 1024 * 1024 } });
```

### Limpieza de archivos temporales

En caso de errores posteriores a la carga, es importante eliminar archivos temporales para evitar acumulación y posibles riesgos.

### Límites y prevención de abuso

Establecer límites en número de archivos, tamaño y frecuencia de carga ayuda a proteger la aplicación de ataques de denegación de servicio o uso indebido.

**Resumen:** La validación y manejo de errores en la carga de archivos es clave para construir aplicaciones backend robustas y seguras. Multer ofrece herramientas para validar, limitar y manejar errores, pero es responsabilidad del desarrollador implementar respuestas claras y prácticas de seguridad adecuadas.

En aplicaciones backend modernas, la carga de archivos es una funcionalidad común pero delicada. Sin validaciones adecuadas, un usuario malintencionado podría subir archivos maliciosos o demasiado grandes, afectando la estabilidad y seguridad del sistema.

Este conocimiento es esencial para desarrolladores backend que trabajan con Node.js y Express, ya que permite controlar el flujo de datos entrantes y proteger recursos del servidor. Además, manejar errores de forma clara mejora la experiencia del usuario y facilita el mantenimiento del código.

En la práctica, integrar estas validaciones y manejo de errores en middleware y rutas modulares garantiza un código limpio y escalable, alineado con buenas prácticas de desarrollo backend.

## Layouts y vistas dinámicas

En esta unidad, aprenderás a crear layouts que sirven como plantillas base para tus vistas en Express usando Handlebars. También descubrirás cómo usar partials para reutilizar fragmentos comunes de

HTML, evitando duplicación y facilitando el mantenimiento. Finalmente, verás cómo pasar datos dinámicos desde tus rutas para renderizar contenido variable en las vistas, todo con ejemplos prácticos y aplicables al desarrollo backend con Node.js y Express.

¿Alguna vez has querido mantener una estructura consistente en todas las páginas de tu aplicación web sin repetir el mismo código HTML una y otra vez? En el desarrollo backend con Node.js y Express, Handlebars nos ofrece una solución elegante: los layouts y partials. Estos conceptos permiten definir plantillas base y fragmentos reutilizables que facilitan la creación de vistas dinámicas y mantenibles.

En esta unidad, descubrirás cómo crear un layout que actúe como esqueleto para tus páginas, cómo insertar contenido variable en él usando el patrón `yield`, y cómo usar partials para incluir elementos comunes como cabeceras y pies de página. Además, aprenderás a pasar datos desde tus rutas para que tus vistas reflejen información dinámica, un aspecto fundamental para aplicaciones backend modernas.

Al finalizar, serás capaz de construir vistas limpias, modulares y dinámicas, mejorando la calidad y escalabilidad de tus proyectos Express con Handlebars.

## Layouts en Handlebars

Un *layout* es una plantilla base que envuelve otras vistas, proporcionando una estructura común para todas las páginas. En Handlebars, el layout define las secciones fijas (como `<head>`, menú, pie de página) y un espacio reservado para contenido variable.

### Patrón `{{body}}` o `{{yield}}`

El espacio donde se inserta el contenido específico de cada vista se denomina *yield* o *body*. En Handlebars, se usa la expresión triple llave `{{body}}` para indicar que aquí se renderizará la vista hija.

#### CÓDIGO

```
<!-- layout.hbs -->
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>{{title}}</title>
</head>
<body>
  {{> header}}
  {{{body}}}
  {{> footer}}
</body>
</html>
```

## Partials: Reutilización de fragmentos

Los *partials* son fragmentos de plantilla reutilizables, ideales para elementos comunes como cabeceras ( `header` ), pies de página ( `footer` ), menús, etc. Se definen en archivos separados y se incluyen con `{{> nombrePartial}}`.

### CÓDIGO

```
<!-- partials/header.hbs -->
<header>
  <h1>Mi Aplicación Express</h1>
  <nav>...</nav>
</header>
```

## Pasar datos dinámicos desde rutas

En Express, al usar `res.render('vista', datos)`, los datos se pasan a la plantilla para renderizar contenido variable.

### CÓDIGO

```
app.get('/usuario', (req, res) => {
  res.render('usuario', {
    title: 'Perfil de Usuario',
    nombre: 'Ana Pérez',
    edad: 28
  });
});
```

En la vista `usuario.hbs`:

### CÓDIGO

```
<h2>Bienvenida, {{nombre}}!</h2>
<p>Edad: {{edad}}</p>
```

### Nota

Los helpers permiten lógica mínima en plantillas, pero en esta unidad reforzaremos solo el uso de *partials* para mantener claridad y simplicidad.

En proyectos backend reales, mantener una estructura consistente y evitar duplicación de código HTML es clave para la escalabilidad y mantenimiento. Los layouts y *partials* permiten centralizar la estructura común y facilitar actualizaciones globales.

Por ejemplo, si decides cambiar el menú de navegación, solo modificarás el *partial* `header`, y el cambio se reflejará en todas las vistas que lo usen. Además, pasar datos dinámicos desde las rutas permite

personalizar cada página según la lógica de negocio, como mostrar el nombre del usuario autenticado o listar productos.

Este enfoque modular es estándar en la industria para aplicaciones Express con Handlebars, y dominarlo te prepara para construir aplicaciones robustas y profesionales.

## Repositorio con Aplicación Express

En esta unidad final, consolidarás todo lo aprendido construyendo y entregando un repositorio funcional que integre Handlebars, routers modularizados y Multer para la carga y validación de archivos. Aprenderás a organizar tu proyecto siguiendo buenas prácticas, implementar vistas dinámicas con layouts, partials y helpers, y asegurar el manejo seguro de archivos. Además, prepararás un README claro con instrucciones para instalar, ejecutar y probar la aplicación, demostrando tu dominio en desarrollo backend con Node.js y Express.

### Construcción de tu Aplicación Express

Imagina que eres un desarrollador backend en una empresa que necesita una aplicación web robusta, organizada y segura. Tu tarea no es entregar un repositorio, sino practicar y demostrarte a tí mismo tu capacidad para integrar vistas dinámicas con Handlebars, modularizar rutas con Express Routers y manejar la carga de archivos con Multer, aplicando validaciones y buenas prácticas de seguridad.

En esta unidad, consolidarás todo lo aprendido en el curso para construir una aplicación completa y funcional. No solo se trata de escribir código, sino de organizarlo profesionalmente, documentarlo claramente y asegurar que tu aplicación sea segura y fácil de mantener.

Al finalizar, serás capaz de crear un repositorio listo para producción que refleje tu dominio en desarrollo backend con Node.js y Express, cumpliendo con estándares de calidad y seguridad. ¡Vamos a comenzar!

## Análisis Profundo de Conceptos Clave

### Motores de Plantillas (Template Engines) y Handlebars

Los motores de plantillas permiten generar HTML dinámico en el servidor combinando plantillas con datos. **Handlebars** es un motor logic-less que facilita esta tarea usando una sintaxis sencilla con expresiones `{{}}` para incrustar datos y controlar la presentación.

- **Sintaxis básica:** Las expresiones `{{variable}}` insertan datos escapados para evitar vulnerabilidades XSS.
- **Partials:** Fragmentos reutilizables de plantilla que evitan duplicación y mejoran mantenimiento.
- **Helpers:** Funciones auxiliares que permiten lógica mínima dentro de las plantillas, como formateo o cálculos simples.
- **Layouts:** Plantillas base que envuelven vistas específicas, permitiendo una estructura común (header, footer) y contenido dinámico por ruta.

## Integración de Handlebars con Express

Express permite configurar Handlebars como motor de vistas mediante `app.set('view engine', 'hbs')` y definir la carpeta de vistas. Se pueden registrar helpers y partials para extender funcionalidades.

### Renderizado Dinámico y Control de Flujo en Plantillas

- **Condicionales ( `if` ):** Permiten mostrar u ocultar bloques según condiciones.
- **Iteraciones ( `each` ):** Recorren arreglos para generar listas dinámicas.
- **Renderizado de listas:** Incluye manejo de casos vacíos y claves únicas para optimizar el DOM.

### Routers y Modularización en Express

Para mantener el código organizado y escalable, se separan rutas en módulos usando `express.Router()`. Cada router puede manejar rutas específicas, middlewares locales y parámetros, y renderizar vistas con datos propios.

### Multer: Configuración, Validación y Seguridad

Multer es un middleware para manejar `multipart/form-data` y subir archivos.

- **Configuración:** Definir almacenamiento ( `diskStorage` ), límites de tamaño y filtros de tipo de archivo.
- **Validación:** Verificar tipo MIME y tamaño para evitar archivos maliciosos o demasiado grandes.
- **Manejo de errores:** Capturar errores de subida y responder adecuadamente al cliente.
- **Seguridad:** Usar nombres únicos para archivos, limpiar archivos temporales y limitar la cantidad y tamaño para prevenir abusos.

#### Nota

La correcta integración y configuración de estos componentes es clave para una aplicación backend profesional y segura.

## Aplicación Práctica

Para esta práctica, debes construir un repositorio Git que contenga tu aplicación Express con las siguientes características:

- **Estructura clara y modular:** Carpeta `routes/` con routers organizados, carpeta `views/` con layouts, partials y vistas, y carpeta `public/` para recursos estáticos.
- **Configuración completa de Handlebars:** Registro de helpers y partials, uso de layouts para vistas dinámicas.
- **Routers que renderizan vistas:** Cada router debe manejar rutas específicas y pasar datos dinámicos a las vistas.
- **Endpoints para carga de archivos:** Usando Multer con validaciones estrictas y manejo de errores.
- **Manejo de errores global y local:** Para capturar y responder adecuadamente a fallos.

- **README detallado:** Instrucciones claras para instalar dependencias, ejecutar la aplicación y probar funcionalidades, incluyendo ejemplos de pruebas manuales.

Esta estructura no solo facilita la mantenibilidad, sino que también refleja las buenas prácticas esperadas en la industria del desarrollo backend.



# Desarrollo de Aplicaciones en Tiempo Real con Websockets

Inicia tu camino en la comunicación en tiempo real comprendiendo qué son los WebSockets y cómo se diferencian del tradicional protocolo HTTP. Este módulo introduce la arquitectura básica cliente-servidor para aplicaciones que requieren interacción instantánea, y te guía a través del proceso fundamental de conexión (handshake) que establece un canal persistente y bidireccional. Con ejemplos claros y análisis prácticos, entenderás cuándo y por qué usar WebSockets, preparando el terreno para implementar tu propio servidor y cliente en el siguiente módulo.

## Preparación del entorno y estructura del proyecto

En esta unidad aprenderás a configurar el entorno de desarrollo para un chat en tiempo real usando Node.js y Socket.IO. Veremos cómo inicializar el proyecto, instalar las dependencias necesarias y organizar la estructura básica de archivos para facilitar el desarrollo y mantenimiento del chat.

¿Alguna vez te has preguntado cómo se prepara un proyecto para construir una aplicación de chat en tiempo real? En esta unidad, daremos los primeros pasos esenciales para configurar un entorno de desarrollo con Node.js y Socket.IO, herramientas clave para lograr comunicaciones instantáneas en la web. Partiremos desde cero, creando el proyecto, instalando las dependencias necesarias y organizando una estructura de archivos clara y funcional. Esto te permitirá enfocarte en la lógica del chat sin perder tiempo en configuraciones complejas. Al finalizar, estarás listo para implementar el servidor y el cliente que intercambiarán mensajes en tiempo real, avanzando hacia la construcción de tu propia aplicación interactiva.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Fundamentos de Socket.IO y configuración inicial del proyecto

Socket.IO es una biblioteca que facilita la comunicación en tiempo real entre clientes y servidores web. Sobre la base de WebSockets, Socket.IO añade características importantes como:

- **API basada en eventos:** Permite enviar y recibir mensajes mediante eventos personalizados.
- **Manejo automático de reconexiones:** Si la conexión se pierde, Socket.IO intenta reconectar automáticamente.
- **Compatibilidad con navegadores antiguos:** Usa técnicas de fallback cuando WebSockets no están disponibles.
- **Rooms y namespaces:** Facilitan la organización de canales de comunicación y la segmentación de usuarios.

Para aprovechar estas ventajas, primero debemos preparar nuestro entorno de desarrollo.

## Inicialización del proyecto Node.js

1. **Crear carpeta del proyecto:** Por ejemplo, `chat-realtime`.
2. **Inicializar npm:** Ejecuta `npm init -y` para crear un `package.json` con configuración básica.
3. **Instalar dependencias:**

### CÓDIGO

```
npm install express socket.io
```

- `express` : Servidor web ligero para servir archivos estáticos y manejar rutas.
- `socket.io` : Biblioteca para comunicación en tiempo real.

## Estructura recomendada del proyecto

Organizar el proyecto facilita el desarrollo y mantenimiento. Una estructura mínima para nuestro chat podría ser:

### CÓDIGO

```
chat-realtime/  
├── server/  
│   └── index.js # Código del servidor Node.js  
├── public/  
│   ├── index.html # Interfaz del cliente  
│   └── js/  
│       └── client.js # Lógica cliente Socket.IO  
└── package.json
```

- La carpeta `server/` contendrá el código backend.
- `public/` alojará los archivos estáticos que el cliente descargará.

## Configuración básica del servidor

En `server/index.js` se crea un servidor Express que sirve la carpeta `public` y se integra Socket.IO para manejar conexiones:

**CÓDIGO**

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

app.use(express.static('public'));

io.on('connection', (socket) => {
  console.log('Usuario conectado:', socket.id);
  // Aquí se manejarán eventos de mensajes
});

server.listen(3000, () => {
  console.log('Servidor escuchando en http://localhost:3000');
});
```

Este código inicial es la base para construir la comunicación en tiempo real que veremos en las siguientes unidades.

**Nota**

En esta unidad reforzamos los fundamentos de Socket.IO y la importancia de una estructura clara para proyectos de tiempo real, preparando el terreno para implementar funcionalidades más complejas.

En el desarrollo web actual, las aplicaciones en tiempo real son cada vez más demandadas, especialmente para funcionalidades como chats, notificaciones o colaboraciones en vivo. Socket.IO se ha convertido en una herramienta estándar para lograr estas comunicaciones de forma sencilla y robusta.

Configurar correctamente el entorno y la estructura del proyecto es fundamental para evitar problemas futuros y facilitar la escalabilidad. Por ejemplo, separar el código del servidor y del cliente permite trabajar de forma modular y mantener el código organizado.

Además, usar Express para servir archivos estáticos es una práctica común que simplifica la integración con Socket.IO, ya que ambos pueden compartir el mismo servidor HTTP.

Esta preparación inicial te permitirá centrarte en la lógica de la aplicación y en la experiencia del usuario, sin preocuparte por configuraciones complejas o desorden en el proyecto.

## ¿Qué son los WebSockets y para qué se utilizan?

Introducción al protocolo WebSocket, sus características principales y los escenarios donde su uso aporta valor frente a HTTP, con ejemplos prácticos en desarrollo web.

Imagina que estás desarrollando una aplicación web donde los usuarios necesitan recibir actualizaciones en tiempo real, como un chat o un panel de control con datos que cambian constantemente. ¿Cómo lograr que el servidor y el navegador se comuniquen de forma eficiente y continua sin tener que recargar la página o hacer múltiples peticiones HTTP?

En esta unidad exploraremos **qué son los WebSockets**, un protocolo que permite una comunicación bidireccional persistente entre cliente y servidor. Entenderemos sus características principales y veremos en qué casos su uso es más adecuado que el tradicional HTTP.

Al finalizar, podrás explicar qué son los WebSockets y reconocerás escenarios reales donde su implementación mejora la experiencia y funcionalidad de aplicaciones web modernas.

### ¿Qué son los WebSockets?

Los **WebSockets** son un protocolo de comunicación que permite establecer un canal de comunicación **bidireccional y persistente** entre un cliente (como un navegador) y un servidor, utilizando una única conexión TCP.

Según el RFC 6455, el protocolo WebSocket proporciona canales de comunicación full-duplex sobre una única conexión TCP.

Esto significa que, a diferencia del protocolo HTTP, donde el cliente debe enviar una petición para recibir una respuesta, con WebSockets tanto el cliente como el servidor pueden enviar mensajes en cualquier momento, sin necesidad de esperar una solicitud previa.

#### Características principales de WebSockets:

- **Comunicación bidireccional:** Cliente y servidor pueden enviar y recibir datos de forma independiente y simultánea.
- **Conexión persistente:** Una vez establecida, la conexión se mantiene abierta, evitando la sobrecarga de abrir y cerrar conexiones repetidamente.
- **Bajo overhead:** El protocolo reduce la cantidad de datos adicionales necesarios para mantener la comunicación, mejorando la eficiencia.

#### Comparación conceptual con HTTP

| Característica         | HTTP                                 | WebSockets                    |
|------------------------|--------------------------------------|-------------------------------|
| Modelo de comunicación | Solicitud-respuesta (unidireccional) | Full-duplex (bidireccional)   |
| Conexión               | Se abre y cierra por cada petición   | Persistente durante la sesión |
| Uso típico             | Carga de páginas, APIs REST          | Aplicaciones en tiempo real   |

## Beneficios de la comunicación full-duplex

La comunicación full-duplex permite que tanto el cliente como el servidor envíen datos en cualquier momento, lo que es ideal para:

- Chats en línea
- Juegos multijugador
- Colaboración en documentos en tiempo real
- Actualización dinámica de dashboards y paneles de control

## Ejemplos de aplicación en la industria

- **Slack:** Utiliza WebSockets para actualizar mensajes y notificaciones en tiempo real.
- **Trading financiero:** Plataformas que muestran cotizaciones y operaciones en vivo.
- **Juegos en línea:** Comunicación rápida y continua entre jugadores y servidores.

### Nota

Aunque HTTP sigue siendo fundamental para muchas aplicaciones, WebSockets es la opción preferida cuando se requiere interacción continua y en tiempo real.

## Contexto de aplicación

En el desarrollo web actual, la experiencia del usuario es clave. Aplicaciones como chats, sistemas de notificaciones, o paneles de control con datos en vivo necesitan una comunicación eficiente y rápida entre cliente y servidor.

Los WebSockets permiten implementar estas funcionalidades sin la latencia y el consumo excesivo de recursos que implicaría hacer múltiples peticiones HTTP.

Este conocimiento es fundamental para avanzar en la creación de aplicaciones modernas con Node.js y tecnologías de tiempo real, que veremos en los siguientes módulos.

Antes de implementar, es importante entender cuándo es adecuado usar WebSockets y cuándo HTTP es suficiente, para elegir la solución más eficiente y escalable.

## Diferencias entre HTTP y WebSockets

Explora las diferencias fundamentales entre el modelo request/response de HTTP y el canal persistente y full-duplex de WebSockets. Analiza aspectos clave como latencia, overhead, escalabilidad y seguridad para entender cuándo es más adecuado usar cada tecnología en aplicaciones web en tiempo real.

¿Alguna vez te has preguntado por qué las aplicaciones modernas como chats en línea o juegos multijugador requieren tecnologías diferentes a las tradicionales páginas web? La respuesta radica en cómo se comunican el cliente y el servidor. En esta unidad, exploraremos las diferencias entre HTTP, el

protocolo clásico de la web, y WebSockets, una tecnología que permite comunicación en tiempo real y bidireccional.

Al finalizar, podrás comparar claramente el modelo de comunicación request/response de HTTP con el canal persistente y full-duplex que ofrece WebSockets, y evaluar cuándo es más eficiente usar uno u otro según el contexto de tu aplicación. Además, revisaremos aspectos de seguridad esenciales para proteger estas conexiones, preparándote para implementar soluciones robustas y seguras.

## Modelos de Comunicación: HTTP vs WebSockets

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### HTTP: Modelo Request/Response

HTTP (HyperText Transfer Protocol) es el protocolo base de la web, basado en un modelo **request/response**. Esto significa que el cliente (por ejemplo, un navegador) envía una solicitud al servidor, y este responde con los datos solicitados. Cada interacción es independiente y se cierra tras la respuesta.

- **Ventajas:** Simplicidad, amplio soporte, fácil de escalar horizontalmente.
- **Limitaciones:** No es eficiente para comunicación en tiempo real, ya que requiere múltiples solicitudes para actualizar datos.

### WebSockets: Canal Persistente y Full-Duplex

WebSockets es un protocolo que establece un canal de comunicación **persistente** y **bidireccional** (full-duplex) entre cliente y servidor sobre una única conexión TCP. Esto permite que ambos puedan enviar y recibir datos en cualquier momento sin necesidad de abrir nuevas conexiones.

- **Ventajas:** Baja latencia, comunicación en tiempo real, menor overhead tras la conexión inicial.
- **Limitaciones:** Más complejo de escalar, requiere manejo cuidadoso de seguridad y recursos.

## Comparación Clave

| Característica | HTTP (Request/Response)            | WebSockets (Canal Persistente)              |
|----------------|------------------------------------|---------------------------------------------|
| Comunicación   | Unidireccional por solicitud       | Bidireccional simultánea                    |
| Latencia       | Mayor (por cada solicitud)         | Baja (conexión abierta)                     |
| Overhead       | Alto (cabeceras en cada solicitud) | Bajo (solo en handshake inicial)            |
| Escalabilidad  | Fácil con balanceadores            | Más complejo, requiere manejo de conexiones |

## Escenarios de Uso

- **HTTP:** Ideal para páginas web estáticas, APIs REST, cargas puntuales de datos.

- **WebSockets:** Aplicaciones que requieren actualización constante y rápida, como chats, juegos en línea, dashboards en tiempo real.

## Consideraciones de Seguridad

Al usar WebSockets, es fundamental reforzar la seguridad debido a la conexión persistente:

- **CORS:** Controla qué orígenes pueden establecer conexiones.
- **TLS (wss://):** Encriptación para proteger datos en tránsito.
- **Validación de origen:** Verificar que las conexiones provengan de fuentes confiables.
- **Sanearamiento de mensajes:** Prevenir inyección de código o datos maliciosos.
- **Límites de conexión:** Evitar abuso o ataques DoS mediante límites en número y duración de conexiones.

### Nota

Estas prácticas complementan el handshake inicial que establece la conexión WebSocket, asegurando que solo clientes autorizados puedan comunicarse.

En el desarrollo web actual, elegir entre HTTP y WebSockets impacta directamente en la experiencia del usuario y la eficiencia del sistema. Por ejemplo, un chat en tiempo real no puede depender de HTTP puro sin generar retrasos y sobrecarga innecesaria, mientras que una página informativa puede funcionar perfectamente con HTTP.

Comprender estas diferencias te permitirá diseñar arquitecturas más adecuadas y optimizadas para tus proyectos. Además, al conocer las consideraciones de seguridad, podrás proteger mejor tus aplicaciones frente a amenazas comunes en conexiones persistentes.

Esta base teórica es esencial para el siguiente módulo, donde implementaremos un servidor y cliente usando Socket.IO para aprovechar las ventajas de WebSockets en la práctica.

## Creación del servidor WebSocket con Node.js y Socket.io

En esta unidad aprenderás a configurar un servidor WebSocket usando Node.js y Socket.IO, manejando eventos clave como la conexión y desconexión de clientes. Además, revisaremos consideraciones básicas de seguridad para un entorno de desarrollo local, preparando el terreno para una comunicación en tiempo real robusta y segura.

Imagina que estás construyendo una aplicación de chat en tiempo real donde múltiples usuarios pueden enviar y recibir mensajes instantáneamente. Para lograr esta experiencia fluida, necesitamos un servidor que mantenga conexiones abiertas y maneje la comunicación bidireccional con los clientes. Aquí es donde entra en juego **Node.js** junto con **Socket.IO**, una biblioteca que facilita la implementación de WebSockets.

En esta unidad, reforzaremos cómo configurar un servidor WebSocket con Node.js y Socket.IO, entenderemos el flujo básico de conexión o *handshake* que establece el canal de comunicación, y abordaremos las primeras consideraciones de seguridad para un entorno de desarrollo local. Al finalizar, estarás listo para manejar conexiones y desconexiones de clientes, y preparar tu servidor para un despliegue más seguro.

Este conocimiento es fundamental para construir aplicaciones web interactivas y en tiempo real, y será la base para las siguientes unidades donde implementaremos el cliente y la lógica de mensajes.

## Servidor WebSocket con Node.js y Socket.IO

### ¿Qué es un servidor WebSocket?

Un servidor WebSocket es una aplicación que mantiene conexiones persistentes con múltiples clientes para permitir comunicación bidireccional en tiempo real. A diferencia de HTTP tradicional, que es un protocolo sin estado y basado en peticiones-respuestas, WebSocket permite un canal abierto y continuo.

### Instalación e inicialización de Socket.IO

Para crear un servidor WebSocket en Node.js, utilizamos Socket.IO, que abstrae la complejidad del protocolo WebSocket y ofrece una API sencilla.

#### CÓDIGO

```
npm install socket.io
```

Luego, en tu archivo principal del servidor:

#### CÓDIGO

```
const http = require('http');
const { Server } = require('socket.io');

const server = http.createServer();
const io = new Server(server, {
  cors: {
    origin: '*', // Para desarrollo local, permite todas las conexiones
  }
});

server.listen(3000, () => {
  console.log('Servidor escuchando en puerto 3000');
});
```



## Manejo de eventos básicos: connection y disconnect

Socket.IO emite eventos que podemos escuchar para manejar la lógica del servidor.

- `connection` : Se dispara cuando un cliente establece conexión.
- `disconnect` : Se dispara cuando un cliente se desconecta.

Ejemplo:

### CÓDIGO

```
io.on('connection', (socket) => {  
  console.log(`Cliente conectado: ${socket.id}`);  
  
  socket.on('disconnect', () => {  
    console.log(`Cliente desconectado: ${socket.id}`);  
  });  
});
```

## Flujo básico de conexión (handshake)

El *handshake* es el proceso inicial donde el cliente solicita actualizar la conexión HTTP a WebSocket. Este intercambio incluye:

- Solicitud HTTP con encabezado `Upgrade: websocket`.
- Respuesta del servidor aceptando la actualización.
- Establecimiento del canal WebSocket para comunicación continua.

Socket.IO maneja este proceso internamente, pero es importante entender que es la base para mantener la conexión abierta.

## Consideraciones de seguridad básicas

Aunque en desarrollo local podemos relajar algunas configuraciones, es recomendable:

- Configurar CORS para limitar orígenes permitidos.
- Considerar el uso de TLS (`wss://`) para cifrar la comunicación en producción.
- Validar el origen de las conexiones para evitar accesos no autorizados.
- Implementar límites y saneamiento en los mensajes para prevenir abusos.

Estas prácticas son esenciales para proteger aplicaciones en entornos reales.

## Aplicación práctica y contexto

En el desarrollo web actual, las aplicaciones en tiempo real son cada vez más comunes: chats, notificaciones, juegos multijugador, entre otros. Node.js, con su modelo de I/O no bloqueante, es ideal para manejar múltiples conexiones simultáneas sin bloquear el servidor.

Socket.IO simplifica la implementación de WebSockets, permitiendo que los desarrolladores se enfoquen en la lógica de la aplicación más que en los detalles del protocolo.

En esta unidad, el código de ejemplo que implementaremos será la base para construir un chat funcional. Entender cómo manejar eventos de conexión y desconexión es crucial para mantener el estado de los usuarios y gestionar recursos.

Además, aunque el enfoque es en desarrollo local, introducir conceptos de seguridad desde el inicio ayuda a formar buenas prácticas que serán vitales en despliegues reales.

Esta unidad conecta directamente con la siguiente, donde implementaremos el cliente que interactuará con este servidor, enviando y recibiendo mensajes en tiempo real.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Implementación del cliente WebSocket y manejo de mensajes

En esta unidad aprenderás a construir el cliente frontend que se conecta a un servidor WebSocket usando Socket.IO, gestionando eventos para enviar y recibir mensajes en tiempo real. Además, abordaremos cómo manejar casos comunes como reconexiones automáticas, errores y validación básica de los mensajes recibidos para asegurar una comunicación robusta y fluida en aplicaciones web.

¿Alguna vez te has preguntado cómo las aplicaciones de chat mantienen la conversación en tiempo real sin necesidad de recargar la página? La respuesta está en la comunicación WebSocket, que permite un canal abierto y bidireccional entre cliente y servidor. En esta unidad, reforzaremos cómo implementar el cliente WebSocket usando la librería `socket.io-client` para conectar con nuestro servidor Node.js, enviar y recibir mensajes, y actualizar la interfaz de usuario de forma dinámica.

Además, abordaremos cómo manejar situaciones comunes en aplicaciones reales, como la reconexión automática cuando la conexión se pierde, la gestión de errores y la validación básica de los mensajes que recibimos para evitar problemas en la comunicación. Al finalizar, estarás listo para construir un cliente robusto que soporte la interacción en tiempo real, base fundamental para cualquier chat moderno o aplicación colaborativa.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### Implementación del cliente WebSocket con Socket.IO

Para establecer una comunicación en tiempo real, el cliente debe conectarse al servidor WebSocket. Usaremos la librería `socket.io-client`, que simplifica la conexión y gestión de eventos.

#### CÓDIGO

```
import { io } from 'socket.io-client';

// Establecer conexión con el servidor
const socket = io('http://localhost:3000');
```

#### Nota

`io()` crea una conexión persistente que permite enviar y recibir mensajes sin necesidad de nuevas solicitudes HTTP.

## Gestión de eventos

Socket.IO utiliza eventos nombrados para organizar la comunicación:

- `connect` : se dispara cuando la conexión se establece.
- `disconnect` : cuando se pierde la conexión.
- Eventos personalizados, por ejemplo, `message` para recibir mensajes.

#### CÓDIGO

```
socket.on('connect', () => {
  console.log('Conectado al servidor con id:', socket.id);
});

socket.on('message', (payload) => {
  console.log('Mensaje recibido:', payload);
  // Aquí se procesará el mensaje
});
```

## Envío de mensajes

Para enviar datos al servidor, usamos `socket.emit` con un evento nombrado y un payload:

#### CÓDIGO

```
const mensaje = { usuario: 'Ana', texto: 'Hola a todos!' };
socket.emit('message', mensaje);
```

## Reconexión y manejo de errores

Socket.IO maneja automáticamente la reconexión, pero podemos escuchar eventos para informar al usuario:

**CÓDIGO**

```
socket.on('disconnect', () => {
  console.warn('Conexión perdida. Intentando reconectar...');
});

socket.io.on('reconnect', (intento) => {
  console.log(`Reconectado tras ${intento} intentos`);
});

socket.on('connect_error', (error) => {
  console.error('Error de conexión:', error.message);
});
```

## Validación básica de mensajes

Es importante validar que los mensajes recibidos tengan la estructura esperada para evitar errores:

**CÓDIGO**

```
socket.on('message', (payload) => {
  if (payload && typeof payload.usuario === 'string' && typeof payload.texto === 'string') {
    // Procesar mensaje
  } else {
    console.warn('Mensaje inválido recibido:', payload);
  }
});
```

**Importante**

La validación protege la aplicación de datos inesperados o malformados, mejorando la estabilidad y seguridad.

En aplicaciones web modernas, el cliente es responsable de mantener una conexión activa con el servidor para recibir actualizaciones en tiempo real, como mensajes de chat, notificaciones o datos colaborativos. La implementación correcta del cliente WebSocket es crucial para garantizar una experiencia fluida y reactiva.

En este contexto, Socket.IO ofrece una API sencilla para manejar conexiones, eventos y reconexiones automáticas, facilitando la construcción de aplicaciones robustas sin necesidad de gestionar manualmente los detalles del protocolo WebSocket.

Además, el manejo adecuado de eventos y la validación de mensajes permiten que la interfaz de usuario se actualice dinámicamente y que la aplicación maneje errores o desconexiones sin interrumpir la experiencia del usuario.

Esta unidad te prepara para implementar el cliente que interactúa con el servidor Node.js que configuraste previamente, sentando las bases para construir un chat funcional y confiable.

## Casos de uso y arquitectura cliente-servidor

Explora los casos de uso típicos de aplicaciones en tiempo real y aprende los fundamentos de la arquitectura cliente-servidor, incluyendo patrones de diseño, topologías de despliegue y estrategias básicas de escalado para garantizar rendimiento y confiabilidad.

¿Alguna vez te has preguntado cómo aplicaciones como chats en línea, notificaciones en tiempo real o juegos multijugador logran mantener a todos los usuarios sincronizados al instante? Estas aplicaciones dependen de una arquitectura especial que permite la comunicación bidireccional y continua entre clientes y servidores. En esta unidad, descubrirás los casos de uso más comunes para aplicaciones en tiempo real y cómo se estructura su arquitectura cliente-servidor para soportar estas funcionalidades. Además, exploraremos las estrategias básicas para escalar estas aplicaciones y mantener su rendimiento a medida que crecen. Al finalizar, estarás preparado para diseñar mentalmente la arquitectura de una aplicación en tiempo real y comprender los patrones y trade-offs involucrados.

### Casos de uso típicos en aplicaciones en tiempo real

Las aplicaciones en tiempo real permiten la comunicación instantánea y continua entre usuarios y sistemas. Algunos ejemplos comunes incluyen:

- **Chats y mensajería instantánea:** Permiten enviar y recibir mensajes al instante.
- **Notificaciones en vivo:** Alertas de eventos, actualizaciones o cambios en sistemas.
- **Juegos multijugador:** Sincronización constante del estado del juego entre jugadores.
- **Colaboración en documentos:** Edición simultánea y sincronizada de archivos.

Estos casos requieren una arquitectura que soporte conexiones persistentes y comunicación bidireccional.

### Arquitectura cliente-servidor para aplicaciones en tiempo real

La arquitectura básica consta de:

- **Clientes:** Aplicaciones o navegadores que mantienen una conexión activa con el servidor.
- **Servidor:** Gestiona conexiones, procesa eventos y distribuye mensajes.
- **Brokers o intermediarios (opcional):** Sistemas que facilitan la comunicación y escalado, como Redis pub/sub.

### Patrones y topologías comunes

- **Rooms o canales:** Agrupan clientes para enviar mensajes específicos a un subconjunto.
- **Proxies y balanceadores (e.g., NGINX):** Distribuyen la carga entre múltiples servidores.
- **Sticky sessions:** Mantienen la conexión de un cliente con el mismo servidor para preservar estado.

### Nota

La comunicación se basa en mantener conexiones abiertas (sockets) que permiten enviar y recibir datos en tiempo real, a diferencia del modelo tradicional HTTP que es unidireccional y basado en peticiones.

## Estrategias de escalado

Para soportar muchos usuarios simultáneos y mantener la calidad del servicio, se aplican varias técnicas:

| Estrategia                             | Descripción                                                                                                             |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Sticky sessions</b>                 | Asignan un cliente siempre al mismo servidor para mantener estado y evitar pérdida de datos.                            |
| <b>Brokers (Redis pub/sub)</b>         | Facilitan la comunicación entre múltiples instancias de servidor, permitiendo distribuir mensajes y sincronizar estado. |
| <b>Clustering</b>                      | Ejecutar múltiples procesos o servidores para balancear la carga.                                                       |
| <b>Separación de responsabilidades</b> | Dividir funciones (autenticación, mensajería, almacenamiento) en servicios especializados.                              |

Cada estrategia tiene ventajas y limitaciones que deben evaluarse según el caso de uso y recursos disponibles.

## Trade-offs y consideraciones

- **Complejidad vs. rendimiento:** Más escalado implica mayor complejidad en la arquitectura.
- **Consistencia vs. disponibilidad:** Mantener todos los clientes sincronizados puede afectar la disponibilidad en caso de fallos.
- **Costos:** Infraestructura y mantenimiento aumentan con la escala.

**Reflexión:** Diseñar una arquitectura en tiempo real requiere balancear estos factores para lograr una solución eficiente y escalable.

En el desarrollo web moderno, las aplicaciones en tiempo real son cada vez más demandadas para mejorar la experiencia del usuario. Entender la arquitectura cliente-servidor y las estrategias de escalado es fundamental para diseñar sistemas robustos y eficientes. Por ejemplo, un chat básico puede funcionar con un solo servidor, pero al crecer la base de usuarios, será necesario implementar balanceo de carga y brokers para mantener la calidad del servicio. Este conocimiento prepara el terreno para implementar soluciones prácticas con Node.js y WebSockets, que veremos en las siguientes unidades.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

# CRUD y Manejo de Bases de Datos con MongoDB

Inicia tu camino en bases de datos NoSQL con un enfoque práctico en MongoDB. Este módulo te introduce a los conceptos esenciales del modelo de documentos y la estructura jerárquica de bases de datos, colecciones y documentos. Aprenderás a usar la CLI de MongoDB para insertar y consultar datos mediante comandos básicos y filtros simples. Al dominar estas habilidades, estarás preparado para avanzar hacia operaciones más complejas y optimización en módulos posteriores, fortaleciendo tu capacidad para desarrollar aplicaciones backend eficientes con MongoDB.

## Bases de datos NoSQL y modelo de documentos en MongoDB

Introducción clara y práctica a las bases de datos NoSQL y al modelo de documentos que utiliza MongoDB, enfocada en desarrolladores principiantes para comprender diferencias clave con bases relacionales y la estructura flexible de BSON/JSON.

¿Alguna vez te has preguntado por qué las bases de datos tradicionales no siempre son la mejor opción para aplicaciones modernas? En este módulo, exploraremos las bases de datos NoSQL, una alternativa flexible y escalable que está revolucionando el desarrollo backend. Aprenderás qué son, por qué son útiles y cómo MongoDB utiliza un modelo de documentos basado en BSON/JSON para almacenar datos de forma eficiente y adaptable. Al finalizar, podrás distinguir claramente entre bases de datos relacionales y NoSQL, y entenderás cómo MongoDB maneja la información para ajustarse a las necesidades dinámicas de las aplicaciones web actuales.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Bases de datos NoSQL: Una introducción práctica

Las bases de datos **NoSQL** son sistemas de almacenamiento de datos que no siguen el modelo relacional tradicional basado en tablas y esquemas rígidos. En lugar de eso, ofrecen modelos más flexibles y escalables, ideales para manejar grandes volúmenes de datos y estructuras variables.

## Modelos comunes de bases NoSQL

- **Orientadas a documentos:** almacenan datos en documentos JSON o BSON, como MongoDB.
- **Clave-valor:** guardan pares clave-valor simples, como Redis.
- **Grafos:** representan relaciones complejas, como Neo4j.
- **Columnas:** almacenan datos en columnas, como Cassandra.

¿Por qué NoSQL? - Escalabilidad horizontal sencilla. - Flexibilidad para datos no estructurados o semiestructurados. - Mejor rendimiento en ciertos casos de uso, como grandes volúmenes o datos en tiempo real.

## Diferencias clave con bases de datos relacionales

| Característica  | Bases Relacionales              | Bases NoSQL                               |
|-----------------|---------------------------------|-------------------------------------------|
| Modelo de datos | Tablas con filas y columnas     | Documentos, clave-valor, grafos, columnas |
| Esquema         | Rígido, definido previamente    | Flexible, dinámico                        |
| Escalabilidad   | Vertical (más potente hardware) | Horizontal (más servidores)               |
| Transacciones   | ACID completo                   | Varía, a menudo eventual                  |

## Modelo de documentos en MongoDB

MongoDB utiliza un modelo basado en **documentos BSON** (Binary JSON), que es una extensión binaria del formato JSON. Cada documento es un registro independiente que puede contener campos anidados, arrays y tipos de datos variados.

### Características del modelo BSON/JSON

- **Flexible:** no requiere un esquema fijo, permitiendo que documentos en la misma colección tengan estructuras diferentes.
- **Anidamiento:** soporta objetos y arrays dentro de documentos.
- **Tipos de datos ricos:** incluye fechas, enteros, cadenas, booleanos, y más.

### Ejemplo de documento MongoDB

#### CÓDIGO

```
{
  "_id": "507f1f77bcf86cd799439011",
  "nombre": "Juan Pérez",
  "edad": 30,
  "contacto": {
    "email": "juan.perez@example.com",
    "tel": "555-1234"
  },
  "intereses": ["programación", "música", "deportes"]
}
```

Este documento representa un usuario con campos simples y anidados, mostrando la flexibilidad del modelo.



## Operaciones básicas de actualización y eliminación (introducción)

Aunque profundizaremos en operaciones CRUD avanzadas en unidades posteriores, es importante conocer que MongoDB permite actualizar y eliminar documentos usando comandos como `updateOne`, `updateMany` y `deleteOne`. Estos comandos utilizan operadores especiales como `$set` para modificar campos, `$inc` para incrementar valores numéricos, y `$unset` para eliminar campos.

Estos conceptos serán reforzados y aplicados en unidades futuras para que puedas manipular datos de forma eficiente y segura.

En el desarrollo backend moderno, elegir la base de datos adecuada es crucial para el rendimiento y la escalabilidad de tus aplicaciones. MongoDB, con su modelo de documentos flexible, es especialmente útil cuando trabajas con datos que cambian con frecuencia o que no encajan bien en tablas rígidas.

Por ejemplo, en una aplicación de comercio electrónico, los productos pueden tener atributos muy variados según la categoría (ropa, electrónica, libros), y el modelo de documentos permite almacenar esta diversidad sin complicaciones.

Además, la estructura BSON/JSON facilita la integración con aplicaciones JavaScript y Node.js, ya que los datos se manejan en formatos muy similares a los objetos nativos del lenguaje.

Este conocimiento te prepara para crear bases de datos y colecciones en MongoDB, y para realizar consultas básicas que exploraremos en las siguientes unidades.

## Estructura de bases de datos, colecciones y documentos

En esta unidad, aprenderás a manejar la jerarquía fundamental de MongoDB: bases de datos, colecciones y documentos. Además, reforzaremos el uso de la herramienta de línea de comandos `mongosh` para crear, listar y navegar entre bases de datos y colecciones. Al finalizar, podrás diferenciar claramente estos conceptos y aplicar comandos básicos para gestionar tu entorno MongoDB desde la CLI.

Entender la estructura básica de bases de datos, colecciones y documentos es clave para trabajar eficazmente con MongoDB. En esta unidad, exploraremos cómo estas piezas encajan y cómo usar la herramienta `mongosh` para crear y navegar por ellas desde la línea de comandos. Esto te permitirá gestionar tus datos de forma clara y ordenada, sentando las bases para consultas y operaciones más avanzadas que veremos en unidades posteriores.

### Jerarquía en MongoDB: Bases de datos, colecciones y documentos

MongoDB es una base de datos NoSQL orientada a documentos que almacena datos en formato BSON, una extensión binaria de JSON. La organización de los datos sigue una jerarquía clara:

- **Base de datos:** Es el contenedor principal que agrupa colecciones relacionadas. Piensa en ella como una carpeta que contiene varios archivos.
- **Colección:** Dentro de una base de datos, las colecciones son grupos de documentos. Son equivalentes a las tablas en bases de datos relacionales, pero sin esquema fijo.

- **Documento:** Es la unidad básica de datos, almacenada en formato JSON/BSON. Cada documento es un objeto con pares clave-valor que representan la información.

#### Ejemplo

Una base de datos llamada tienda puede contener una colección productos, y cada documento dentro de productos representa un artículo con campos como nombre, precio y categoría.

## Uso básico de mongosh para gestionar bases de datos y colecciones

`mongosh` es la herramienta de línea de comandos oficial para interactuar con MongoDB. Permite ejecutar comandos para crear, listar y seleccionar bases de datos y colecciones.

- **Listar bases de datos:**

#### CÓDIGO

```
show dbs
```

- **Seleccionar o crear una base de datos:**

#### CÓDIGO

```
use nombre_de_la_base
```

Si la base de datos no existe, `use` la crea en el momento en que se inserte el primer documento.

- **Listar colecciones dentro de la base de datos actual:**

#### CÓDIGO

```
show collections
```

- **Crear una colección explícitamente:**

#### CÓDIGO

```
db.createCollection('nombre_coleccion')
```

#### Nota

En MongoDB, las colecciones se crean automáticamente al insertar el primer documento, por lo que crear colecciones explícitamente no es obligatorio.

## Buenas prácticas en el modelado de documentos

- Mantén los documentos **simples y coherentes** para facilitar consultas y mantenimiento.
- Evita anidar demasiados niveles de documentos para no complicar las operaciones.

- Usa tipos de datos adecuados para cada campo (números, cadenas, fechas).
- Aprovecha la flexibilidad de MongoDB para adaptar el esquema según las necesidades, pero con control.

**Ejemplo de documento simple en la colección `productos` :**

#### CÓDIGO

```
{
  "nombre": "Camiseta",
  "precio": 19.99,
  "categoría": "Ropa",
  "en_stock": true
}
```

## Resumen

- MongoDB organiza los datos en bases de datos, colecciones y documentos.
- `mongosh` es la CLI para gestionar estas estructuras con comandos simples.
- Entender esta jerarquía es fundamental para manipular datos y preparar consultas.

Esta base te permitirá avanzar con confianza hacia operaciones CRUD y consultas más complejas en las siguientes unidades.

En el desarrollo backend con MongoDB, manejar correctamente la estructura de bases de datos, colecciones y documentos es esencial para construir aplicaciones eficientes y escalables. Por ejemplo, al diseñar una API para una tienda en línea, saber cómo organizar los productos, usuarios y pedidos en colecciones separadas dentro de una base de datos facilita la gestión y consulta de datos.

Además, el uso de `mongosh` permite a los desarrolladores interactuar directamente con la base de datos para tareas de administración, pruebas rápidas y depuración, sin necesidad de interfaces gráficas. Esto es especialmente útil en entornos de desarrollo y producción donde el acceso rápido y seguro es clave.

Esta unidad te prepara para usar estas herramientas y conceptos en escenarios reales, como crear bases de datos para diferentes proyectos, listar colecciones existentes y entender cómo se almacenan los datos que tu aplicación consumirá y modificará.

**IMAGEN** *Imagen de referencia del material original (no disponible en esta versión de lectura).*

## Inserción y lectura de documentos en MongoDB

Aprende a insertar documentos en colecciones usando los comandos `insertOne` e `insertMany` desde la CLI de MongoDB, y a recuperar datos con consultas básicas usando `find()`. Además, explorare-

mos cómo mejorar la visualización con `pretty()`, limitar resultados con `limit()`, y seleccionar campos específicos y ordenar resultados mediante proyección y ordenamiento.

En esta unidad, descubrirás cómo insertar documentos en colecciones usando comandos simples desde la CLI de MongoDB, y cómo realizar consultas básicas para leer esos datos. Aprenderás a usar `insertOne` e `insertMany` para agregar información, y a recuperar documentos con `find()`, mejorando la visualización con `pretty()` y controlando la cantidad de resultados con `limit()`. Además, exploraremos cómo seleccionar solo los campos que necesitas y ordenar los resultados para facilitar su análisis. Estos conocimientos son fundamentales para manejar datos en aplicaciones backend modernas y te prepararán para operaciones más avanzadas que veremos en unidades posteriores.

**IMAGEN** *Imagen de referencia del material original (no disponible en esta versión de lectura).*

## Inserción de documentos en MongoDB

MongoDB almacena datos en **documentos JSON** dentro de colecciones. Para agregar documentos, usamos los comandos:

- `insertOne(documento)` : inserta un solo documento.
- `insertMany([documento1, documento2, ...])` : inserta múltiples documentos a la vez.

**Ejemplo:**

CÓDIGO

```
db.usuarios.insertOne({ nombre: "Ana", edad: 28, ciudad: "Madrid" })
```

Esto agrega un documento con los campos `nombre`, `edad` y `ciudad` a la colección `usuarios`.

## Lectura básica con find()

Para recuperar documentos usamos `find()`, que devuelve un cursor para iterar sobre los resultados.

- `db.coleccion.find()` : devuelve todos los documentos.
- `db.coleccion.find(filtro)` : devuelve documentos que cumplen el filtro.

**Ejemplo:**

CÓDIGO

```
db.usuarios.find({ ciudad: "Madrid" })
```

Recupera todos los usuarios que viven en Madrid.

## Mejoras en la visualización

- `pretty()` : formatea la salida para que sea más legible.

#### CÓDIGO

```
db.usuarios.find().pretty()
```

- `limit(n)` : limita el número de documentos devueltos.

#### CÓDIGO

```
db.usuarios.find().limit(5)
```

## Proyección y ordenamiento

- **Proyección:** seleccionar solo ciertos campos para mostrar.

#### CÓDIGO

```
db.usuarios.find({}, { nombre: 1, ciudad: 1, _id: 0 })
```

Muestra solo los campos `nombre` y `ciudad` , ocultando `_id` .

- **Ordenamiento:** ordenar resultados con `sort()` .

#### CÓDIGO

```
db.usuarios.find().sort({ edad: -1 })
```

Ordena usuarios por edad descendente.

#### Nota

El filtro en `find()` es un objeto que define condiciones para seleccionar documentos, mientras que la proyección define qué campos mostrar.

**Con estos comandos, puedes insertar y consultar datos básicos en MongoDB, una habilidad esencial para cualquier desarrollador backend que trabaje con bases de datos NoSQL.**

En el desarrollo backend, manejar datos de forma eficiente es clave para construir aplicaciones rápidas y escalables. MongoDB, como base de datos NoSQL orientada a documentos, permite almacenar información flexible y realizar consultas potentes sin la rigidez de esquemas relacionales.

Insertar documentos con `insertOne` e `insertMany` es el primer paso para alimentar tu base de datos con información real, ya sea usuarios, productos o registros de eventos. Luego, recuperar esos datos con `find()` te permite mostrarlos en tu aplicación, analizarlos o procesarlos.

El uso de proyección y ordenamiento optimiza la cantidad de datos transferidos y mejora la experiencia del usuario al mostrar solo lo necesario y en el orden adecuado. Por ejemplo, en una aplicación de comercio electrónico, podrías mostrar solo el nombre y precio de productos ordenados por popularidad.

Estos conceptos son la base para operaciones CRUD más avanzadas y consultas complejas que exploraremos en las siguientes unidades, donde aprenderás a actualizar, eliminar y optimizar tus consultas para aplicaciones reales.

## Consultas avanzadas con operadores y condiciones lógicas

En esta unidad, profundizaremos en cómo realizar consultas avanzadas en MongoDB utilizando operadores especializados como `$elemMatch`, `$regex` y `$exists`, además de combinar condiciones lógicas para manejar estructuras de datos complejas, incluyendo arrays y documentos anidados. Aprenderás a optimizar tus consultas para evitar operaciones costosas y mejorar el rendimiento de tus aplicaciones backend.

¿Alguna vez te has enfrentado a la necesidad de buscar datos específicos dentro de estructuras complejas en MongoDB, como arrays o documentos anidados? En esta unidad, exploraremos cómo aprovechar operadores avanzados y condiciones lógicas para realizar consultas precisas y eficientes. Esto te permitirá manejar datos reales de aplicaciones web backend con mayor control y optimización, mejorando la experiencia de usuario y el rendimiento de tus sistemas.

### Jerarquía básica en MongoDB

MongoDB organiza la información en una jerarquía clara: una **base de datos** contiene varias **colecciones**, y cada colección almacena múltiples **documentos**. Los documentos son objetos en formato BSON (similar a JSON) que pueden incluir campos simples, arrays y documentos anidados.

### Operadores avanzados y condiciones lógicas

Para consultas complejas, MongoDB ofrece operadores especializados:

- **\$elemMatch**: Permite buscar documentos donde un array contiene al menos un elemento que cumple ciertas condiciones.
- **\$regex**: Realiza búsquedas por patrones en campos de texto, similar a expresiones regulares.
- **\$exists**: Verifica la existencia o ausencia de un campo en un documento.

Además, puedes combinar condiciones usando operadores lógicos como `$and`, `$or`, `$not` y `$nor` para construir consultas sofisticadas.

### Ejemplo de \$elemMatch

#### CÓDIGO

```
// Buscar documentos donde el array 'comentarios' tiene al menos un elemento
// con 'autor' igual a 'Ana' y 'likes' mayor a 10
{
  comentarios: { $elemMatch: { autor: 'Ana', likes: { $gt: 10 } } }
}
```

## Ejemplo de \$regex

### CÓDIGO

```
// Buscar documentos donde el campo 'titulo' contiene la palabra 'MongoDB' (sin
importar mayúsculas)
{
  titulo: { $regex: /mongodb/i }
}
```

## Combinación de condiciones lógicas

### CÓDIGO

```
// Buscar documentos que tengan campo 'activo' true y que el campo 'categoria'
sea 'tecnología' o 'educación'
{
  $and: [
    { activo: true },
    { $or: [ { categoria: 'tecnología' }, { categoria: 'educación' } ] }
  ]
}
```

### Nota

Usar estos operadores correctamente mejora la precisión y eficiencia de tus consultas, especialmente en estructuras anidadas y arrays.

En aplicaciones backend, es común manejar datos con estructuras complejas, como listas de comentarios, etiquetas o configuraciones anidadas. Por ejemplo, una plataforma de blogs puede almacenar múltiples comentarios en un array dentro de cada documento de artículo. Saber cómo consultar estos datos con operadores avanzados permite filtrar información relevante sin necesidad de cargar todos los datos y procesarlos en la aplicación, lo que optimiza recursos y tiempos de respuesta.

Además, el uso de expresiones regulares con `$regex` facilita búsquedas flexibles en campos de texto, muy útil para implementar funcionalidades de búsqueda o filtros dinámicos.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

Finalmente, combinar condiciones lógicas permite construir consultas que reflejen reglas de negocio complejas, como filtrar usuarios activos que cumplan múltiples criterios simultáneamente.

## Proyecciones y ordenamiento de resultados

Aprende a limitar los campos que devuelve una consulta en MongoDB usando proyecciones, y a ordenar los resultados eficientemente con `sort()`, comprendiendo su impacto en el rendimiento y la relación con los índices.

¿Alguna vez has recibido más información de la que realmente necesitas al consultar una base de datos? En MongoDB, esto puede afectar el rendimiento y la claridad de tus resultados. En esta unidad, exploraremos cómo usar **proyecciones** para seleccionar solo los campos que te interesan y cómo ordenar los resultados con `sort()` para obtener datos organizados y útiles. Además, entenderemos por qué el uso adecuado de índices es clave para que estas operaciones sean eficientes, especialmente en aplicaciones backend donde la velocidad y la optimización son cruciales.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### Repaso rápido: Insertar y leer documentos

Antes de profundizar, recordemos que para agregar documentos usamos `insertOne()` o `insertMany()`, y para leerlos, el método básico es `find()`. Por ejemplo:

#### CÓDIGO

```
// Insertar un documento
db.usuarios.insertOne({ nombre: "Ana", edad: 28, ciudad: "Madrid" });

// Leer todos los documentos
db.usuarios.find();
```

### Proyección: Seleccionando solo lo necesario

La **proyección** permite limitar los campos que devuelve una consulta, mejorando el rendimiento y la claridad. Se especifica como segundo parámetro en `find()`, usando inclusión (`1`) o exclusión (`0`).

- **Inclusión:** Solo los campos con valor `1` se devuelven.
- **Exclusión:** Se omiten los campos con valor `0`.

Ejemplo de inclusión:

#### CÓDIGO

```
db.usuarios.find({}, { nombre: 1, ciudad: 1, _id: 0 });
```

Devuelve solo los campos `nombre` y `ciudad`, excluyendo `_id`.

#### Nota

No se puede mezclar inclusión y exclusión en la misma proyección, excepto para `_id`.



## Ordenamiento con sort()

Para ordenar los resultados, usamos `sort()`, que recibe un objeto con campos y dirección:

- `1` para orden ascendente
- `-1` para orden descendente

Ejemplo:

### CÓDIGO

```
db.usuarios.find().sort({ edad: -1 });
```

Ordena los usuarios de mayor a menor edad.

## Relación entre sort() e índices

El uso de `sort()` sin un índice adecuado puede causar que MongoDB realice un **ordenamiento en memoria**, lo que impacta negativamente el rendimiento, especialmente con grandes volúmenes de datos.

Para optimizar:

- Crea índices en los campos que usas para ordenar.
- MongoDB puede usar el índice para devolver resultados ya ordenados, evitando el costo extra.

Ejemplo de creación de índice:

### CÓDIGO

```
db.usuarios.createIndex({ edad: 1 });
```

## Buenas prácticas

- Siempre proyecta solo los campos necesarios para reducir la carga de datos.
- Usa índices para campos frecuentes en ordenamientos.
- Evita ordenar por campos sin índice en colecciones grandes.

## ¿Listo para aplicar estas técnicas en tus consultas?

En el desarrollo backend con MongoDB, es común que las APIs necesiten devolver solo ciertos campos para optimizar la transferencia de datos y mejorar la experiencia del usuario. Por ejemplo, una API que lista usuarios puede no necesitar enviar toda la información personal, solo el nombre y la ciudad.

Además, ordenar resultados es fundamental para mostrar datos relevantes, como los productos más recientes o los usuarios con mayor puntuación. Sin embargo, sin índices adecuados, estas operaciones pueden ralentizar la aplicación.

Comprender y aplicar proyecciones y ordenamientos eficientes es clave para construir aplicaciones escalables y rápidas, especialmente cuando trabajas con grandes volúmenes de datos en entornos de producción.

## Consultas con filtros y operadores básicos

Aprende a construir consultas efectivas en MongoDB utilizando operadores de comparación y lógicos básicos. Descubre cómo combinar filtros para recuperar datos específicos y optimizar tus consultas con técnicas de paginación e índices.

¿Alguna vez has necesitado encontrar datos específicos en una base de datos pero no sabías cómo filtrar la información correctamente? En MongoDB, las consultas con filtros y operadores básicos son la clave para extraer exactamente lo que necesitas de tus colecciones. En esta unidad, aprenderás a usar operadores como `$eq`, `$gt`, `$lt`, `$in`, `$and` y `$or` para construir consultas precisas y eficientes. Además, exploraremos técnicas de paginación para manejar grandes volúmenes de datos y una introducción a la optimización mediante índices, preparando el camino para consultas más rápidas y efectivas en tus aplicaciones backend.

### Operadores de comparación y lógicos en MongoDB

MongoDB utiliza operadores especiales para construir filtros que permiten seleccionar documentos según condiciones específicas. Estos operadores se usan dentro de objetos JSON en las consultas.

#### Operadores de comparación básicos

- `$eq` : Igualdad. Selecciona documentos donde el valor del campo es igual al especificado.
- `$gt` : Mayor que. Selecciona documentos donde el valor del campo es mayor que el especificado.
- `$lt` : Menor que. Selecciona documentos donde el valor del campo es menor que el especificado.
- `$in` : Pertenece a un conjunto. Selecciona documentos donde el valor del campo está dentro de un arreglo de valores.

#### Ejemplo básico con `$eq` y `$gt`

##### CÓDIGO

```
// Encuentra usuarios con edad igual a 30
db.usuarios.find({ edad: { $eq: 30 } })

// Encuentra productos con precio mayor a 100
db.productos.find({ precio: { $gt: 100 } })
```

#### Operadores lógicos

- `$and` : Combina múltiples condiciones y todas deben cumplirse.
- `$or` : Combina condiciones y al menos una debe cumplirse.

## Ejemplo con \$and y \$or

### CÓDIGO

```
// Usuarios con edad mayor a 25 y ciudad 'Madrid'
db.usuarios.find({ $and: [ { edad: { $gt: 25 } }, { ciudad: 'Madrid' } ] })

// Productos con precio menor a 50 o categoría 'libros'
db.productos.find({ $or: [ { precio: { $lt: 50 } }, { categoria: 'libros' } ] })
```

### Nota

MongoDB permite simplificar consultas con \$and usando múltiples pares clave-valor directamente.

## Consultas anidadas y combinación de filtros

Puedes combinar operadores para consultas más complejas, por ejemplo:

### CÓDIGO

```
// Usuarios con edad entre 20 y 30 y que vivan en 'Barcelona' o 'Valencia'
db.usuarios.find({
  edad: { $gt: 20, $lt: 30 },
  $or: [ { ciudad: 'Barcelona' }, { ciudad: 'Valencia' } ]
})
```

## Técnicas de paginación

Cuando trabajas con grandes conjuntos de datos, es común mostrar resultados en páginas para mejorar la experiencia y rendimiento.

- **skip(n)** : Omite los primeros **n** documentos.
- **limit(m)** : Limita la cantidad de documentos devueltos a **m** .

## Ejemplo de paginación

### CÓDIGO

```
// Obtener la segunda página con 10 documentos por página
db.productos.find().skip(10).limit(10)
```

## Introducción a la optimización de consultas e índices

Los índices aceleran las consultas al permitir búsquedas rápidas en campos específicos. MongoDB crea índices automáticamente en el campo `_id` , pero puedes crear índices personalizados para mejorar el rendimiento.

En esta unidad solo introducimos el concepto; profundizaremos en índices y análisis de planes de ejecución en unidades posteriores.

En el desarrollo backend, filtrar datos de forma eficiente es fundamental para construir APIs rápidas y escalables. Por ejemplo, una aplicación de comercio electrónico debe mostrar productos según criterios como precio, categoría o disponibilidad, y paginar resultados para no saturar al usuario ni al servidor.

MongoDB facilita estas tareas con operadores de consulta flexibles y técnicas de paginación que permiten manejar grandes volúmenes de datos sin perder rendimiento. Además, entender la optimización mediante índices es clave para aplicaciones en producción que requieren respuestas rápidas.

En esta unidad, te prepararemos para construir consultas que extraen datos precisos y para manejar resultados paginados, habilidades esenciales para cualquier desarrollador backend que trabaje con MongoDB.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Paginación y optimización de consultas

En esta unidad, exploraremos técnicas avanzadas para paginar resultados en MongoDB y optimizar consultas mediante índices y análisis de planes de ejecución. Aprenderás a aplicar métodos de paginación eficientes y a crear índices que mejoren el rendimiento de tus consultas, preparándote para manejar grandes volúmenes de datos en aplicaciones reales.

¿Alguna vez has intentado mostrar grandes cantidades de datos en una aplicación web y notaste que la carga se vuelve lenta o poco eficiente? La paginación y la optimización de consultas son técnicas clave para mejorar el rendimiento y la experiencia del usuario en aplicaciones que manejan bases de datos NoSQL como MongoDB.

En esta unidad, reforzaremos cómo construir consultas condicionales usando filtros y operadores básicos, y profundizaremos en dos métodos comunes de paginación: `skip/limit` y paginación basada en cursores o rangos de ordenamiento. Además, introduciremos el concepto de índices en MongoDB, cómo crearlos y cómo analizar su impacto usando la función `explain()`. Estas habilidades te permitirán manejar datos de forma eficiente y preparar tus aplicaciones para escalar.

Al finalizar, podrás aplicar técnicas de paginación adecuadas según el contexto y optimizar tus consultas para mejorar el rendimiento, habilidades esenciales para cualquier desarrollador backend que trabaje con MongoDB.

### Filtros y operadores básicos para consultas condicionales

Para construir consultas efectivas en MongoDB, es fundamental entender los operadores básicos que permiten filtrar documentos según condiciones específicas. Algunos operadores clave son:

- `$eq` : Igualdad exacta.

- `$gt` y `$lt` : Mayor que y menor que, respectivamente.
- `$in` : Coincidencia con cualquiera de los valores en un arreglo.
- `$and` y `$or` : Combinación lógica de condiciones.

#### Ejemplo:

##### CÓDIGO

```
// Buscar usuarios con edad mayor a 25 y que vivan en 'Madrid' o 'Barcelona'
db.usuarios.find({
  $and: [
    { edad: { $gt: 25 } },
    { ciudad: { $in: ['Madrid', 'Barcelona'] } }
  ]
})
```

## Técnicas de paginación en MongoDB

La paginación permite dividir grandes conjuntos de datos en partes manejables para mostrar en interfaces de usuario.

### 1. Paginación con `skip` y `limit`

- `skip(n)` : Omite los primeros `n` documentos.
- `limit(m)` : Limita la cantidad de documentos devueltos a `m`.

**Ventajas:** Fácil de implementar.

**Desventajas:** Ineficiente para páginas muy avanzadas, ya que MongoDB debe recorrer y descartar documentos.

#### Ejemplo:

##### CÓDIGO

```
db.productos.find().skip(20).limit(10)
```

### 2. Paginación basada en cursores y rangos de ordenamiento

En lugar de saltar documentos, se usa un campo ordenado (como `_id` o fecha) para obtener documentos "a partir de" un valor específico.

**Ventajas:** Más eficiente en grandes colecciones.

#### Ejemplo:

#### CÓDIGO

```
// Obtener productos con _id mayor al último _id visto
const lastId = ObjectId("507f1f77bcf86cd799439011");
db.productos.find({ _id: { $gt: lastId } }).limit(10)
```

## Optimización de consultas e índices

Los índices son estructuras que aceleran la búsqueda de documentos en una colección.

### Tipos de índices comunes

- **Índice simple:** En un solo campo.
- **Índice compuesto:** En múltiples campos, útil para consultas que filtran por varios criterios.

#### Creación de índice simple:

#### CÓDIGO

```
db.usuarios.createIndex({ email: 1 }) // índice ascendente en email
```

#### Creación de índice compuesto:

#### CÓDIGO

```
db.usuarios.createIndex({ ciudad: 1, edad: -1 })
```

## Uso de `explain()` para analizar consultas

`explain()` muestra el plan de ejecución de una consulta, ayudando a identificar si se usan índices y cómo.

#### Ejemplo:

#### CÓDIGO

```
db.usuarios.find({ ciudad: 'Madrid' }).explain('executionStats')
```

#### Nota

Un plan eficiente usa índices y tiene un bajo número de documentos examinados.

**Con estos conceptos claros, estás listo para aplicar técnicas de paginación y optimización en tus proyectos con MongoDB.**

En aplicaciones web y backend, manejar grandes volúmenes de datos es común. Mostrar todos los registros a la vez no es viable ni eficiente, por lo que la paginación es esencial para mejorar la experiencia del usuario y reducir la carga en el servidor.

Sin embargo, no todas las técnicas de paginación son iguales. Usar `skip` y `limit` puede ser suficiente para conjuntos pequeños, pero a medida que la base de datos crece, esta técnica se vuelve lenta y consume más recursos.

La paginación basada en cursores o rangos de ordenamiento es una solución más escalable, especialmente cuando se combinan con índices adecuados que aceleran la búsqueda.

Los índices, por su parte, son fundamentales para optimizar consultas frecuentes. Crear índices en los campos que se usan para filtrar o ordenar puede reducir drásticamente el tiempo de respuesta.

Finalmente, la función `explain()` es una herramienta práctica para diagnosticar y mejorar el rendimiento de tus consultas, permitiéndote validar que tus índices están siendo utilizados correctamente.

En la siguiente sección, pondremos en práctica estas técnicas para que puedas aplicarlas directamente en tus proyectos.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Actualización y eliminación de documentos

En esta unidad, reforzaremos los patrones clave para actualizar y eliminar documentos en MongoDB, explorando operadores esenciales como `$set` y `$inc`, y la técnica de `upsert` para inserciones condicionales. También abordaremos consideraciones prácticas para borrados masivos y la importancia de respaldos previos, todo con un enfoque claro y aplicado para desarrolladores backend principiantes.

En esta unidad, exploraremos cómo actualizar y eliminar documentos usando comandos clave como `updateOne`, `updateMany` y `deleteOne`. Además, aprenderás a usar operadores como `$set` y `$inc` para modificar campos específicos, y la técnica de `upsert` para insertar documentos cuando no existen. También discutiremos buenas prácticas para borrados masivos y la importancia de realizar respaldos antes de operaciones críticas. Este conocimiento es fundamental para manejar datos en aplicaciones backend modernas con MongoDB, asegurando integridad y eficiencia.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Fundamentos de Bases de Datos NoSQL y Modelo de Documentos

Antes de profundizar en las operaciones de actualización y eliminación, recordemos que MongoDB es una base de datos NoSQL orientada a documentos. Esto significa que los datos se almacenan en documentos flexibles en formato BSON (una extensión binaria de JSON), permitiendo estructuras anidadas y sin esquema rígido.

## Patrones de Actualización en MongoDB

MongoDB ofrece métodos para modificar documentos existentes:

- `updateOne(filter, update, options)` : Actualiza el primer documento que cumple el filtro.
- `updateMany(filter, update, options)` : Actualiza todos los documentos que cumplen el filtro.

Los objetos `update` usan operadores para definir cambios específicos:

- `$set` : Establece el valor de un campo.
- `$inc` : Incrementa o decrementa un valor numérico.
- `$unset` : Elimina un campo del documento.

### Ejemplo

#### CÓDIGO

```
db.usuarios.updateOne(
  { _id: 1 },
  { $set: { nombre: "Ana" }, $inc: { edad: 1 } }
)
```

Este comando cambia el nombre y aumenta la edad en 1.

## Upsert: Actualizar o Insertar

La opción `upsert: true` permite que si no existe un documento que cumpla el filtro, se cree uno nuevo con los datos indicados.

#### CÓDIGO

```
db.productos.updateOne(
  { sku: "12345" },
  { $set: { precio: 100 } },
  { upsert: true }
)
```

Si no hay producto con sku "12345", se inserta uno nuevo.

## Eliminación de Documentos

Para eliminar documentos:

- `deleteOne(filter)` : Elimina el primer documento que cumple el filtro.
- `deleteMany(filter)` : Elimina todos los documentos que cumplen el filtro.

**Precaución:** Las eliminaciones masivas pueden afectar muchos datos; siempre verifica el filtro y considera respaldos.



## Atomicidad a Nivel de Documento

Las operaciones de actualización y eliminación en MongoDB son atómicas a nivel de documento, lo que significa que cada documento se modifica o elimina completamente o no se modifica en absoluto, garantizando consistencia.

**Con este repaso, reforzamos los conceptos clave para manipular datos de forma segura y eficiente en MongoDB.**

En el desarrollo backend, actualizar y eliminar datos correctamente es crucial para mantener la integridad y el rendimiento de la aplicación. Por ejemplo, en una aplicación de comercio electrónico, actualizar el stock de productos con `$inc` tras una compra evita inconsistencias. Usar `upsert` facilita sincronizar datos externos sin duplicados.

Además, eliminar documentos debe hacerse con cuidado para evitar pérdida accidental de información. Por eso, antes de borrados masivos, es recomendable realizar respaldos y validar filtros.

Este conocimiento prepara al desarrollador para manejar datos dinámicos y operaciones complejas, base para optimizaciones y uso avanzado de MongoDB que veremos en unidades posteriores.

# Desarrollo Avanzado de Backend con Node.js y MongoDB

Inicia tu camino en el desarrollo backend aprendiendo a conectar aplicaciones Node.js con MongoDB Atlas mediante Mongoose. Este módulo introduce el patrón ODM, las ventajas de MongoDB como servicio en la nube y la configuración básica de un proyecto con Express. Aprenderás a instalar y configurar Mongoose, gestionar variables de entorno para conexiones seguras y establecer una conexión estable y manejable con MongoDB Atlas. Al finalizar, estarás preparado para modelar datos y construir operaciones CRUD en el siguiente módulo, asegurando una base sólida para tus aplicaciones backend.

## Schemas, modelos y validaciones en Mongoose

Introducción a la definición de schemas y modelos en Mongoose, con enfoque en tipos de datos y validaciones para garantizar la integridad de los documentos en MongoDB. Se presentan ejemplos prácticos para crear schemas con validaciones básicas y la relación entre schema, modelo y colección.

En el desarrollo backend con Node.js y MongoDB, Mongoose se presenta como una herramienta esencial para definir la estructura y las validaciones de tus datos mediante schemas y modelos. En esta unidad, descubrirás cómo crear schemas que especifican tipos de datos y reglas de validación, y cómo estos se transforman en modelos que interactúan con las colecciones de MongoDB. Este conocimiento es fundamental para construir APIs robustas y confiables, y te preparará para implementar operaciones CRUD efectivas en las siguientes unidades.

### Schemas y Modelos en Mongoose

Mongoose es un ODM (*Object Data Modeling*) que facilita la interacción con MongoDB desde Node.js, permitiendo definir la estructura de los documentos mediante **schemas** y operar sobre ellos con **modelos**.

### ¿Qué es un Schema?

Un **schema** en Mongoose es un objeto que define la estructura de los documentos en una colección, especificando los **tipos de datos** y las **reglas de validación** para cada campo.

#### Tipos de datos básicos en schemas

- **String** : texto
- **Number** : números
- **Date** : fechas
- **Array** : listas de valores

Ejemplo básico de schema:

#### CÓDIGO

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  email: String,
  createdAt: Date
});
```

## Opciones comunes en schemas

Puedes configurar opciones para controlar el comportamiento del schema, como:

- `required` : campo obligatorio
- `default` : valor por defecto
- `unique` : valor único en la colección

## Validaciones en Mongoose

Las validaciones aseguran que los datos cumplan ciertas condiciones antes de guardarse.

### Validaciones integradas

- `required` : asegura que el campo no esté vacío
- `match` : valida con una expresión regular
- `min` y `max` : para valores numéricos o fechas

Ejemplo con validaciones:

#### CÓDIGO

```
const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, min: 0 },
  category: { type: String, match: /^[A-Za-z]+$ / }
});
```

## Validadores personalizados (custom validators)

Puedes definir funciones que validen condiciones específicas:

#### CÓDIGO

```
const userSchema = new mongoose.Schema({
  age: {
    type: Number,
    validate: {
      validator: function(v) { return v >= 18; },
      message: props => `${props.value} no es una edad válida. Debe ser mayor o igual a 18.`
    }
  }
});
```

## ¿Qué es un Modelo?

Un `model` es una clase construida a partir de un schema que permite crear y consultar documentos en la colección correspondiente.

#### CÓDIGO

```
const User = mongoose.model('User', userSchema);
```

Aquí, `User` es el modelo que representa la colección `users` en MongoDB (Mongoose pluraliza el nombre).

#### Nota

El modelo es la interfaz para interactuar con la base de datos, mientras que el schema define la estructura y reglas de los documentos.

**Con este conocimiento, estás listo para definir estructuras de datos claras y seguras en tus aplicaciones backend.**

En el desarrollo backend, mantener la integridad y consistencia de los datos es crucial para evitar errores y comportamientos inesperados. Mongoose facilita esta tarea al permitirte definir schemas con tipos y validaciones que se aplican automáticamente antes de guardar los datos en MongoDB.

Por ejemplo, si estás desarrollando una API para gestionar usuarios, puedes asegurarte de que cada usuario tenga un correo electrónico válido y una edad mínima, evitando así datos erróneos que podrían afectar la lógica de negocio o la experiencia del usuario.

Además, la relación entre schema y modelo te permite trabajar con objetos JavaScript que representan documentos en la base de datos, simplificando la manipulación y consulta de datos.

Esta unidad sienta las bases para implementar operaciones CRUD seguras y eficientes, que exploraremos en las siguientes unidades, donde aplicarás estos conceptos para crear, leer, actualizar y eliminar documentos usando Mongoose y Express.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Conceptos clave de Mongoose y ODM en MongoDB

Introducción a Mongoose como una herramienta ODM para MongoDB, explicando qué es un ODM, sus ventajas frente al driver nativo y el flujo general de trabajo con schemas, modelos y consultas. Se incluyen ejemplos conceptuales para facilitar la comprensión y preparar al estudiante para la definición de schemas y modelos en unidades posteriores.

¿Alguna vez te has preguntado cómo manejar de forma eficiente los datos en una aplicación Node.js que utiliza MongoDB? Mongoose es la respuesta para muchos desarrolladores backend que buscan una forma estructurada y segura de interactuar con sus bases de datos. En esta unidad, descubrirás qué es un ODM (Object Data Modeling), por qué Mongoose es una herramienta esencial para trabajar con MongoDB y cómo facilita la gestión de datos mediante schemas y modelos. Al finalizar, comprenderás la relación entre los documentos almacenados en MongoDB y los modelos que defines en tu aplicación, sentando las bases para implementar operaciones CRUD robustas y escalables.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### ¿Qué es un ODM (Object Data Modeling)?

Un **ODM** es un patrón de diseño que permite mapear documentos de una base de datos NoSQL, como MongoDB, a objetos en el código de una aplicación. Esto significa que en lugar de trabajar directamente con consultas y documentos JSON, puedes manipular objetos con propiedades y métodos, lo que facilita la escritura, lectura y validación de datos.

#### Importante

El ODM abstrae la complejidad de las operaciones con la base de datos, proporcionando una interfaz más amigable y estructurada.

### Mongoose: El ODM para MongoDB en Node.js

Mongoose es una librería para Node.js que implementa este patrón ODM específicamente para MongoDB. Permite:

- Definir **schemas** que describen la estructura y validaciones de los documentos.
- Crear **modelos** basados en esos schemas para interactuar con la base de datos.
- Realizar consultas, actualizaciones y eliminaciones de forma sencilla y segura.

## Ventajas de usar Mongoose frente al driver nativo de MongoDB

| Característica                 | Driver Nativo MongoDB | Mongoose (ODM)                  |
|--------------------------------|-----------------------|---------------------------------|
| Definición de estructura       | No                    | Sí, mediante schemas            |
| Validaciones integradas        | No                    | Sí, en schemas                  |
| Métodos y hooks personalizados | Limitados             | Sí, permite lógica en modelos   |
| Abstracción de consultas       | Baja                  | Alta, con métodos simplificados |

## Flujo general de trabajo con Mongoose

1. **Definir un schema:** Describe la forma y reglas de los datos.
2. **Crear un modelo:** Basado en el schema, representa una colección en MongoDB.
3. **Usar el modelo:** Para crear, leer, actualizar y eliminar documentos.

Por ejemplo, un schema para usuarios puede definir campos como `nombre`, `email` y `edad` con sus tipos y validaciones.

## Ejemplo conceptual

### CÓDIGO

```
const mongoose = require('mongoose');

// Definición del schema
const usuarioSchema = new mongoose.Schema({
  nombre: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  edad: { type: Number, min: 0 }
});

// Creación del modelo
const Usuario = mongoose.model('Usuario', usuarioSchema);

// Uso del modelo para crear un nuevo usuario
const nuevoUsuario = new Usuario({ nombre: 'Ana', email: 'ana@example.com',
  edad: 28 });
nuevoUsuario.save()
  .then(() => console.log('Usuario guardado'))
  .catch(err => console.error('Error:', err));
```

Este ejemplo muestra cómo Mongoose facilita la definición y manipulación de datos, asegurando que los documentos cumplan con las reglas establecidas en el schema.

En el desarrollo backend con Node.js, manejar datos de forma eficiente y segura es fundamental para construir aplicaciones robustas. MongoDB, como base de datos NoSQL, almacena datos en documentos flexibles, pero trabajar directamente con ellos puede ser propenso a errores y falta de estructura.

Mongoose actúa como un puente entre la base de datos y la aplicación, proporcionando una capa de abstracción que permite definir claramente cómo deben ser los datos y cómo interactuar con ellos. Esto no solo mejora la calidad del código, sino que también facilita el mantenimiento y escalabilidad del proyecto.

En proyectos reales, usar Mongoose ayuda a evitar inconsistencias en los datos y simplifica la implementación de funcionalidades complejas, como validaciones personalizadas y middleware para operaciones antes o después de guardar datos.

Esta unidad prepara el terreno para que en las siguientes unidades puedas definir schemas y modelos con confianza, y así implementar un CRUD completo y funcional con Express y MongoDB.

## Operaciones CRUD: Create y Read con Mongoose

En esta unidad aprenderás a implementar las operaciones básicas de creación y lectura de documentos en MongoDB utilizando Mongoose. Veremos cómo usar métodos como `Model.create`, `Model.find` y `Model.findById` para manejar datos de forma eficiente, incluyendo técnicas de paginación y manejo de errores, todo orientado a aplicaciones backend con Node.js y Express.

¿Alguna vez te has preguntado cómo las aplicaciones web almacenan y recuperan datos de forma rápida y segura? En el desarrollo backend con Node.js, Mongoose se convierte en una herramienta clave para interactuar con MongoDB de manera sencilla y estructurada. En esta unidad, nos enfocaremos en las operaciones básicas de creación ( `Create` ) y lectura ( `Read` ) de documentos, fundamentales para cualquier API RESTful.

A través de ejemplos prácticos, aprenderás a insertar nuevos registros en la base de datos y a consultarlos usando filtros y paginación, todo ello con un enfoque orientado a proyectos reales. Este conocimiento te permitirá construir endpoints robustos y eficientes, sentando las bases para las operaciones de actualización y eliminación que veremos en unidades posteriores.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### Operaciones CRUD con Mongoose: Create y Read

Mongoose es un ODM (Object Data Modeling) que facilita la interacción con MongoDB desde Node.js, permitiendo definir **schemas** y **modelos** para estructurar y validar datos.

#### Creación de documentos con `Model.create`

El método `Model.create()` permite insertar uno o varios documentos en la colección asociada al modelo.

#### CÓDIGO

```
// Ejemplo básico de creación
const nuevoUsuario = await Usuario.create({
  nombre: 'Ana',
  email: 'ana@example.com',
  edad: 28
});
```

Este método valida el documento según el schema definido y lanza errores si hay inconsistencias.

### Lectura de documentos con `Model.find` y `Model.findById`

- `Model.find(filter)` devuelve un array con todos los documentos que cumplen el filtro.
- `Model.findById(id)` busca un documento por su `_id`.

#### CÓDIGO

```
// Encontrar todos los usuarios mayores de 18 años
const adultos = await Usuario.find({ edad: { $gte: 18 } });

// Encontrar un usuario por ID
const usuario = await Usuario.findById('60d5f483f8d2e30f8c8b4567');
```

### Paginación simple

Para evitar devolver grandes volúmenes de datos, se usa paginación con `limit` y `skip`:

#### CÓDIGO

```
const pagina = 2;
const limite = 10;
const usuarios = await Usuario.find()
  .skip((pagina - 1) * limite)
  .limit(limite);
```

### Manejo de errores y validaciones

Al crear documentos, Mongoose valida según el schema. Es importante capturar errores para informar al cliente:



#### CÓDIGO

```
try {
  const nuevo = await Usuario.create(datos);
} catch (error) {
  if (error.name === 'ValidationError') {
    // Manejar errores de validación
  } else {
    // Otros errores
  }
}
```

#### Nota

Siempre valida y sanitiza los datos recibidos para mantener la integridad y seguridad de la base de datos.

En un proyecto backend real, estas operaciones se implementan dentro de controladores que responden a rutas definidas en Express. Por ejemplo, un endpoint `POST /usuarios` usará `Model.create` para agregar un nuevo usuario, mientras que un `GET /usuarios` o `GET /usuarios/:id` usará `Model.find` o `Model.findById` para recuperar datos.

La paginación es crucial para mejorar el rendimiento y la experiencia del usuario cuando la base de datos crece. Además, el manejo adecuado de errores garantiza que la API sea robusta y que los clientes reciban respuestas claras y útiles.

Esta unidad te prepara para implementar estos endpoints básicos, que son la base para construir APIs RESTful completas y escalables con Node.js, Express y MongoDB.

## Clientes de base de datos y DBaaS con MongoDB Atlas

En esta unidad, exploraremos qué es MongoDB Atlas como servicio de base de datos en la nube (DBaaS), cómo crear y configurar un clúster básico, gestionar usuarios y redes, y obtener la cadena de conexión (URI) para integrarla en aplicaciones Node.js. Además, aprenderemos buenas prácticas para manejar credenciales mediante variables de entorno, garantizando seguridad y flexibilidad en el desarrollo backend.

¿Sabías que puedes tener una base de datos MongoDB completamente gestionada en la nube sin preocuparte por la infraestructura? MongoDB Atlas es la solución DBaaS que facilita el despliegue, la seguridad y el escalado de bases de datos MongoDB, permitiendo a los desarrolladores enfocarse en construir aplicaciones robustas. En esta unidad, aprenderás a crear y configurar un clúster básico en Atlas, gestionar accesos y obtener la cadena de conexión necesaria para integrar tu backend Node.js. Además, descubrirás cómo manejar tus credenciales de forma segura usando variables de entorno, una práctica esencial para cualquier proyecto profesional.

## MongoDB Atlas: Base de datos como servicio (DBaaS)

MongoDB Atlas es una plataforma en la nube que ofrece **MongoDB como servicio gestionado**. Esto significa que no necesitas instalar ni administrar servidores; Atlas se encarga de:

- Despliegue automático de clústeres
- Backups y restauración
- Monitoreo y alertas
- Seguridad y control de accesos
- Escalabilidad bajo demanda

**Ventajas clave para desarrolladores Node.js:** - Rápida puesta en marcha sin configuración compleja - Conexiones seguras y configurables - Integración sencilla con Mongoose y otros ODM

## Creación y configuración de un clúster básico

Para comenzar, en MongoDB Atlas debes:

1. Crear una cuenta gratuita en <https://www.mongodb.com/cloud/atlas>.
2. Crear un nuevo proyecto y dentro de él un clúster (puedes elegir el plan gratuito "M0" para desarrollo).
3. Configurar usuarios con roles y permisos específicos para tu aplicación.
4. Definir las redes autorizadas (IP whitelist) para controlar quién puede conectarse.

## URI de conexión y variables de entorno

La **URI de conexión** es una cadena que contiene toda la información necesaria para que tu aplicación Node.js se conecte a MongoDB Atlas. Tiene un formato similar a:

### CÓDIGO

```
mongodb+srv://<usuario>:<contraseña>@<cluster>.mongodb.net/<base_de_datos>?retryWrites=true&w=majority
```

### Importante

Nunca debes incluir esta cadena con credenciales directamente en tu código fuente.

La práctica recomendada es usar **variables de entorno** para almacenar la URI o sus componentes sensibles. Por ejemplo, en un archivo `.env`:

### CÓDIGO

```
MONGODB_URI=mongodb+srv://usuario:contraseña@cluster.mongodb.net/miBaseDatos?retryWrites=true&w=majority
```

Y luego acceder a ella en Node.js con `process.env.MONGODB_URI`.

Esto mejora la seguridad y facilita la configuración en diferentes entornos (desarrollo, producción).

## Buenas prácticas para entornos y credenciales

- Usa archivos `.env` y la librería `dotenv` para cargar variables de entorno.
- No subas archivos con credenciales a repositorios públicos.
- Configura usuarios con permisos mínimos necesarios.
- Controla las IPs autorizadas para evitar accesos no deseados.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

**¿Sabías que?** MongoDB Atlas también ofrece integración con servicios de autenticación avanzada y cifrado en reposo para proyectos que requieren alta seguridad.

En el desarrollo backend con Node.js, conectar tu aplicación a una base de datos MongoDB es fundamental para almacenar y gestionar datos. MongoDB Atlas simplifica esta tarea al ofrecer un servicio gestionado que elimina la necesidad de administrar servidores o configuraciones complejas.

Al crear un clúster en Atlas, configuras un entorno seguro y escalable para tu base de datos. La gestión de usuarios y redes garantiza que solo tu aplicación y usuarios autorizados puedan acceder.

La URI de conexión es el puente entre tu backend y la base de datos. Usar variables de entorno para manejar esta información sensible es una práctica estándar en la industria, que protege tus credenciales y facilita la portabilidad del proyecto.

Esta unidad te prepara para establecer una conexión segura y estable con MongoDB Atlas, un paso esencial antes de definir esquemas y modelos con Mongoose, que veremos en la siguiente unidad.

## Creación y configuración de un proyecto Node.js

En esta unidad aprenderás a inicializar un proyecto básico de Node.js, instalar las dependencias esenciales como Express y dotenv, y configurar variables de entorno para manejar de forma segura la URI de conexión a MongoDB. Este paso es fundamental para preparar tu entorno de desarrollo y sentar las bases para conectar tu aplicación a MongoDB Atlas en las siguientes unidades.

¿Alguna vez te has preguntado cómo iniciar un proyecto backend con Node.js que sea escalable y seguro? En esta unidad, darás los primeros pasos para crear una aplicación Node.js básica utilizando Express, el framework más popular para construir APIs REST. Además, aprenderás a manejar variables sensibles, como la URI de conexión a MongoDB, mediante variables de entorno para mantener la seguridad y flexibilidad de tu proyecto. Estos conocimientos son esenciales para preparar tu entorno y avanzar hacia la integración con MongoDB Atlas en las próximas unidades.

## Estructura básica de un proyecto Node.js con Express

Para comenzar a desarrollar una aplicación backend con Node.js, es fundamental entender cómo se estructura un proyecto básico y qué herramientas se utilizan.

### Inicialización del proyecto con `package.json`

El archivo `package.json` es el corazón de cualquier proyecto Node.js. Contiene metadatos, dependencias y scripts que facilitan la gestión del proyecto.

Para crear este archivo, se utiliza el comando:

#### CÓDIGO

```
npm init -y
```

Este comando genera un `package.json` con valores predeterminados, que luego puedes modificar según tus necesidades.

### Instalación de dependencias esenciales

Para nuestro proyecto, instalaremos dos paquetes clave:

- **Express:** Framework minimalista para crear servidores y definir rutas HTTP.
- **dotenv:** Permite cargar variables de entorno desde un archivo `.env`, manteniendo datos sensibles fuera del código.

Instalación:

#### CÓDIGO

```
npm install express dotenv
```

### Estructura recomendada de carpetas y archivos

Una estructura sencilla y clara facilita el mantenimiento y escalabilidad:

#### CÓDIGO

```
project-root/  
|  
├─ node_modules/ # Dependencias instaladas  
├─ .env # Variables de entorno (no subir a repositorios públicos)  
├─ package.json # Metadatos y dependencias  
├─ server.js # Archivo principal del servidor  
└─ README.md # Documentación del proyecto
```

## Configuración de variables de entorno con .env

El archivo `.env` contiene variables sensibles, como la URI de conexión a MongoDB. Ejemplo:

### CÓDIGO

```
MONGODB_URI=mongodb+srv://usuario: contraseña@cluster0.mongodb.net/miBaseDatos?
retryWrites=true&w=majority
PORT=3000
```

Para usar estas variables en tu código, debes configurar `dotenv` al inicio de tu archivo principal:

### CÓDIGO

```
require('dotenv').config();

const express = require('express');
const app = express();

const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log(`Servidor escuchando en puerto ${port}`);
});
```

### Nota

Nunca subas tu archivo `.env` a repositorios públicos para proteger tus credenciales.

## Conexión segura y buenas prácticas

- Usa variables de entorno para toda información sensible.
- Añade `.env` a tu archivo `.gitignore` para evitar subirlo a Git.
- Mantén la estructura del proyecto simple al inicio y ve escalando según necesidades.

Este conocimiento es la base para conectar tu aplicación a MongoDB Atlas, que veremos en la siguiente unidad.

En el desarrollo backend profesional, iniciar un proyecto con una estructura clara y segura es fundamental para evitar problemas futuros. Express es el estándar de facto para crear APIs REST en Node.js, y manejar variables de entorno con `dotenv` es una práctica recomendada para proteger credenciales y configuraciones sensibles.

Por ejemplo, en una startup que desarrolla una API para una aplicación móvil, tener un proyecto bien configurado desde el inicio permite que el equipo escale funcionalidades sin comprometer la seguridad ni la estabilidad.

Además, configurar correctamente la URI de MongoDB en variables de entorno facilita el despliegue en diferentes entornos (desarrollo, pruebas, producción) sin modificar el código fuente.

Esta unidad te prepara para conectar tu backend a MongoDB Atlas, un servicio DBaaS que ofrece alta disponibilidad y escalabilidad, asegurando que tu aplicación pueda crecer con confianza.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Operaciones CRUD: Update y Delete con Mongoose

En esta unidad aprenderás a realizar operaciones de actualización y eliminación de documentos en MongoDB utilizando Mongoose. Exploraremos métodos como `findByIdAndUpdate` y `findByIdAndDelete`, así como buenas prácticas para actualizaciones seguras y la implementación opcional de soft delete. Al finalizar, podrás aplicar estas técnicas para mantener la integridad y seguridad de tus datos en aplicaciones backend con Node.js y Express.

¿Alguna vez has necesitado modificar o eliminar datos en tu base de datos de forma segura y eficiente? En el desarrollo backend con Node.js y MongoDB, estas operaciones son fundamentales para mantener la integridad y relevancia de la información. En esta unidad, profundizaremos en cómo realizar actualizaciones y eliminaciones de documentos usando Mongoose, una herramienta que simplifica la interacción con MongoDB. Aprenderás a usar métodos como `findByIdAndUpdate` para actualizar datos de forma segura y a eliminar documentos con `findByIdAndDelete`, además de explorar la técnica de *soft delete* para preservar información sin eliminarla físicamente. Estas habilidades te permitirán construir APIs robustas y confiables, preparándote para implementar un CRUD completo en tus proyectos backend.

### Operaciones Update y Delete en Mongoose

Mongoose ofrece métodos específicos para modificar y eliminar documentos en MongoDB de manera sencilla y segura.

#### Actualización con `findByIdAndUpdate`

Este método permite buscar un documento por su `_id` y actualizarlo en una sola operación. Su sintaxis básica es:

##### CÓDIGO

```
Model.findByIdAndUpdate(id, update, options, callback)
```

- `id` : Identificador del documento a actualizar.
- `update` : Objeto con los campos a modificar.
- `options` : Configuraciones adicionales.

## Opciones clave

- `new: true` : Retorna el documento actualizado en lugar del original.
- `runValidators: true` : Ejecuta las validaciones definidas en el schema durante la actualización.

### Importante

Sin `runValidators: true`, Mongoose no valida los datos en actualizaciones, lo que puede introducir inconsistencias.

## Actualizaciones parciales vs reemplazo

- **Actualización parcial:** Modifica solo los campos especificados, ideal para cambios puntuales.
- **Reemplazo completo:** Sustituye todo el documento, menos el `_id`. Se usa con métodos como `save()` tras modificar el objeto.

## Eliminación con `findByIdAndDelete`

Permite eliminar un documento por su `_id` de forma directa:

### CÓDIGO

```
Model.findByIdAndDelete(id, callback)
```

Esta operación elimina físicamente el documento de la base de datos.

## Soft Delete (Eliminación lógica)

En lugar de borrar el documento, se marca como eliminado mediante un campo, por ejemplo, `isDeleted: true`. Esto permite:

- Mantener el historial y referencias.
- Evitar pérdida accidental de datos.

Implementar soft delete requiere modificar consultas para filtrar documentos "eliminados".

## Buenas prácticas

- Siempre usar `runValidators: true` para mantener integridad.
- Validar que el documento existe antes de actualizar o eliminar.
- Considerar soft delete en aplicaciones donde la recuperación de datos es crítica.

## Ejemplo de actualización segura:

#### CÓDIGO

```
const updatedUser = await User.findByIdAndUpdate(
  userId,
  { email: 'nuevoemail@example.com' },
  { new: true, runValidators: true }
);
```

#### Ejemplo de soft delete:

#### CÓDIGO

```
await User.findByIdAndUpdate(userId, { isDeleted: true }, { new: true });
```

Recuerda que para soft delete debes adaptar tus consultas para excluir documentos con `isDeleted: true`.

En proyectos backend reales, las operaciones de actualización y eliminación son comunes y deben manejarse con cuidado para evitar pérdida de datos o inconsistencias.

Por ejemplo, en una API de gestión de usuarios, es frecuente actualizar información como el correo electrónico o eliminar usuarios que ya no están activos. Usar `findByIdAndUpdate` con las opciones adecuadas garantiza que los datos modificados cumplan con las reglas definidas en el schema, evitando errores en la base de datos.

La eliminación física con `findByIdAndDelete` es directa, pero en muchos casos, especialmente en aplicaciones empresariales, se prefiere implementar *soft delete* para mantener un registro histórico y permitir la recuperación de datos si es necesario.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

Además, estas operaciones deben integrarse en las rutas y controladores de Express para ofrecer endpoints seguros y eficientes, que respondan correctamente a las solicitudes del cliente y manejen errores adecuadamente.

Esta unidad te prepara para implementar estas funcionalidades en tu API, asegurando que tus datos se mantengan consistentes y que las operaciones CRUD sean completas y confiables.

## Instalación y configuración de Mongoose en Node.js

Aprende a instalar Mongoose en tu proyecto Node.js, configurar las opciones de conexión esenciales y manejar eventos clave para asegurar una conexión estable y segura con MongoDB Atlas.

¿Alguna vez te has preguntado cómo conectar tu aplicación Node.js con una base de datos MongoDB de forma eficiente y segura? En esta unidad, darás el siguiente paso práctico para integrar Mongoose, el ODM que facilita la interacción con MongoDB, en tu proyecto. Aprenderás a instalarlo correctamen-



te, configurar las opciones esenciales para una conexión estable y manejar los eventos que te permitirán monitorear el estado de la conexión. Este conocimiento es fundamental para garantizar que tu backend funcione sin interrupciones y esté listo para manejar datos de manera confiable.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## ¿Qué es Mongoose?

Mongoose es un *Object Data Modeling* ( **ODM** ) para MongoDB y Node.js que proporciona una solución sencilla y estructurada para definir esquemas y modelos, facilitando la interacción con la base de datos.

## Instalación de Mongoose

Para añadir Mongoose a tu proyecto, puedes usar los siguientes comandos según el gestor de paquetes que prefieras:

### CÓDIGO

```
### Usando npm
npm install mongoose

### Usando yarn
yarn add mongoose
```

### Nota

Asegúrate de ejecutar estos comandos en la raíz de tu proyecto Node.js.

## Configuración de la conexión

Al conectar Mongoose con MongoDB Atlas, es importante configurar ciertas opciones para evitar advertencias y asegurar una conexión estable:

- `useNewUrlParser: true` — Utiliza el nuevo analizador de URL de MongoDB.
- `useUnifiedTopology: true` — Usa el nuevo motor de monitoreo de servidores para mejorar la estabilidad.

Ejemplo básico de conexión:

**CÓDIGO**

```
const mongoose = require('mongoose');

mongoose.connect('tu_cadena_de_conexion', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});
```

## Manejo de eventos de conexión

Mongoose emite eventos que permiten monitorear el estado de la conexión. Los más importantes son:

| Evento                    | Descripción                                    |
|---------------------------|------------------------------------------------|
| <code>connected</code>    | Se dispara cuando la conexión se establece.    |
| <code>error</code>        | Se dispara cuando ocurre un error de conexión. |
| <code>disconnected</code> | Se dispara cuando la conexión se pierde.       |

Ejemplo de manejo de eventos:

**CÓDIGO**

```
const db = mongoose.connection;

db.on('connected', () => {
  console.log('Conexión a MongoDB establecida');
});

db.on('error', (err) => {
  console.error('Error en la conexión:', err);
});

db.on('disconnected', () => {
  console.log('Conexión a MongoDB desconectada');
});
```

**Importante**

Manejar estos eventos te permite reaccionar ante problemas y mantener la aplicación robusta.

## Buenas prácticas

- Siempre maneja los eventos para detectar y responder a problemas de conexión.
- Usa variables de entorno para almacenar la cadena de conexión y evitar exponer credenciales.
- Verifica que la conexión esté establecida antes de realizar operaciones con la base de datos.

Este conocimiento te prepara para definir schemas y modelos en la siguiente unidad, asegurando que la conexión a MongoDB Atlas sea confiable y segura.

En el desarrollo backend con Node.js y MongoDB, una conexión estable y bien configurada es la base para cualquier aplicación que maneje datos. Mongoose no solo facilita la interacción con la base de datos, sino que también ofrece mecanismos para monitorear y manejar la conexión, lo que es crucial en entornos de producción donde la disponibilidad y la integridad de los datos son prioritarias.

Al configurar correctamente Mongoose, evitarás errores comunes que pueden interrumpir el servicio, como desconexiones inesperadas o problemas de compatibilidad con el driver de MongoDB. Además, el manejo adecuado de eventos te permitirá implementar estrategias de reconexión o alertas, mejorando la resiliencia de tu aplicación.

Este conocimiento es especialmente relevante cuando trabajas con MongoDB Atlas, un servicio DBaaS que ofrece alta disponibilidad y escalabilidad, pero que requiere una configuración segura y eficiente desde el cliente para aprovechar todas sus ventajas.

En la práctica, dominar esta configuración te permitirá avanzar con confianza hacia la definición de schemas y modelos, sabiendo que la base de datos está correctamente integrada y lista para recibir operaciones CRUD.

## Estructura de rutas y flujo del CRUD en Node.js

En esta unidad, aprenderás a diseñar rutas REST coherentes para recursos CRUD utilizando Express y a separar la lógica en controladores que interactúan con modelos Mongoose. También exploraremos el uso de middlewares para validaciones y manejo de errores, y cómo organizar la estructura del proyecto para mantener un código limpio y escalable.

¿Alguna vez te has preguntado cómo organizar las rutas y la lógica de tu API para que sea clara, mantenible y escalable? En esta unidad, abordaremos cómo estructurar las rutas REST para un CRUD completo usando Express y Mongoose, separando responsabilidades entre rutas y controladores, y aplicando middlewares para validar datos y manejar errores. Este enfoque es fundamental para construir APIs backend robustas y profesionales, y te preparará para implementar funcionalidades completas y probadas en tus proyectos.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### Diseño de rutas REST para CRUD en Express

Express es un framework minimalista para Node.js que facilita la definición de rutas y middlewares para construir APIs REST. En el contexto de un CRUD (Crear, Leer, Actualizar, Eliminar), las rutas corresponden a endpoints HTTP que responden a métodos específicos:

| Método HTTP | Acción CRUD | Descripción                    |
|-------------|-------------|--------------------------------|
| POST        | Crear       | Crear un nuevo recurso         |
| GET         | Leer        | Obtener uno o varios recursos  |
| PUT/PATCH   | Actualizar  | Modificar un recurso existente |
| DELETE      | Eliminar    | Borrar un recurso              |

## Separación de rutas y controladores

Para mantener un código organizado y escalable, es buena práctica separar la definición de rutas de la lógica que ejecutan. Las **rutas** definen los endpoints y métodos HTTP, mientras que los **controladores** contienen la lógica que interactúa con los modelos Mongoose para realizar operaciones en la base de datos.

Ejemplo básico de ruta y controlador:

### CÓDIGO

```
// routes/productRoutes.js
const express = require('express');
const router = express.Router();
const productController = require('../controllers/productController');

router.post('/', productController.createProduct);
router.get('/', productController.getProducts);
router.get('/:id', productController.getProductById);
router.put('/:id', productController.updateProduct);
router.delete('/:id', productController.deleteProduct);

module.exports = router;
```

**CÓDIGO**

```
// controllers/productController.js
const Product = require('../models/Product');

exports.createProduct = async (req, res, next) => {
  try {
    const product = new Product(req.body);
    const savedProduct = await product.save();
    res.status(201).json(savedProduct);
  } catch (error) {
    next(error); // Pasar error al middleware de manejo
  }
};

// Implementar getProducts, getProductById, updateProduct, deleteProduct de
forma similar
```

## Middleware para validaciones y manejo de errores

Los middlewares en Express son funciones que pueden interceptar y procesar solicitudes antes de llegar a los controladores o después de ellos. Son ideales para:

- Validar datos de entrada (por ejemplo, verificar que los campos requeridos estén presentes)
- Manejar errores de forma centralizada

Ejemplo de middleware de validación simple:

**CÓDIGO**

```
function validateProduct(req, res, next) {
  if (!req.body.name) {
    return res.status(400).json({ error: 'El nombre del producto es obligatorio' });
  }
  next();
}
```

Middleware de manejo de errores:

**CÓDIGO**

```
function errorHandler(err, req, res, next) {
  console.error(err);
  res.status(500).json({ error: 'Error interno del servidor' });
}
```

## Estructura recomendada del proyecto

Una estructura clara facilita el mantenimiento y escalabilidad:

### CÓDIGO

```
project-root/  
|  
├─ controllers/ # Lógica de negocio y acceso a datos  
├─ models/ # Definición de schemas y modelos Mongoose  
├─ routes/ # Definición de rutas Express  
├─ middlewares/ # Middlewares personalizados  
├─ app.js # Configuración principal de Express  
└─ server.js # Inicialización del servidor
```

Esta organización permite que cada parte tenga una responsabilidad clara y facilita la colaboración en equipo.

En proyectos reales de backend con Node.js y MongoDB, mantener una estructura clara y modular es clave para el éxito. Separar rutas y controladores permite que el equipo trabaje en paralelo y que el código sea más fácil de probar y mantener. Además, el uso de middlewares para validaciones y manejo de errores mejora la robustez y experiencia del usuario.

Por ejemplo, en una API para gestión de productos, las rutas definen cómo los clientes pueden interactuar con los recursos, mientras que los controladores implementan la lógica para crear, leer, actualizar o eliminar productos en la base de datos MongoDB usando Mongoose. Los middlewares aseguran que los datos enviados sean válidos y que los errores se manejen de forma consistente.

Esta unidad te prepara para implementar un CRUD completo y profesional, que podrás probar y extender en las siguientes unidades.

# MongoDB Avanzado y Optimización de Consultas

Domina cómo la indexación mejora el rendimiento de tus consultas en MongoDB. Partiendo de conocimientos básicos, aprenderás a distinguir entre escaneos completos y accesos optimizados mediante índices, identificar cuándo y qué tipo de índices aplicar, y usar `explain()` para evaluar y optimizar consultas reales. Este módulo te prepara para construir modelos eficientes y sentar las bases para optimizaciones avanzadas en poblaciones y agregaciones en el siguiente módulo.

## Importancia de la indexación y su impacto en las consultas

En esta unidad exploraremos qué es un índice en MongoDB, cómo mejora el rendimiento de las consultas al reducir el trabajo de lectura y escritura, y cuáles son los costos asociados a su mantenimiento. También introduciremos conceptos básicos para optimizar consultas mediante filtros, proyecciones y ordenamientos que aprovechan los índices existentes, preparando el terreno para un análisis más profundo en unidades posteriores.

¿Alguna vez has notado que una consulta en MongoDB tarda mucho más en ejecutarse que otra aparentemente similar? Esto suele deberse a cómo MongoDB accede a los datos internamente. En esta unidad, descubrirás cómo los índices actúan como atajos que permiten a la base de datos encontrar rápidamente la información que necesitas, evitando revisar cada documento uno por uno. Además, entenderás que aunque los índices aceleran las lecturas, también tienen un costo en las escrituras y almacenamiento, por lo que es crucial saber cuándo y cómo usarlos. Al finalizar, podrás explicar claramente por qué los índices son fundamentales para el rendimiento y cómo optimizar consultas básicas para aprovecharlos al máximo.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### ¿Qué es un índice en MongoDB?

Un *índice* en MongoDB es una estructura de datos que almacena una copia ordenada de uno o más campos de los documentos en una colección. Esto permite que las consultas que filtran o ordenan por esos campos puedan localizar los documentos relevantes sin necesidad de escanear toda la colección.

### ¿Cómo aceleran los índices las consultas?

Cuando una consulta no puede usar un índice, MongoDB realiza un **full collection scan** o escaneo completo, revisando cada documento para encontrar coincidencias. Esto implica un alto consumo de I/O y tiempo, especialmente en colecciones grandes.

Con un índice adecuado, MongoDB puede:

- Localizar rápidamente los documentos que cumplen el filtro.
- Reducir la cantidad de datos leídos desde disco.
- Mejorar el rendimiento de ordenamientos y proyecciones.

#### Ejemplo

Si tienes un índice en el campo `email`, una consulta que busque `{ email: "usuario@ejemplo.com" }` usará ese índice para encontrar el documento sin revisar toda la colección.

## Costos y trade-offs de los índices

Aunque los índices mejoran las lecturas, tienen un costo:

- **Espacio en disco:** Cada índice ocupa almacenamiento adicional.
- **Impacto en escrituras:** Al insertar, actualizar o eliminar documentos, MongoDB debe actualizar también los índices afectados, lo que puede ralentizar estas operaciones.

Por eso, es importante crear índices solo cuando aporten un beneficio claro en consultas frecuentes.

## Optimización básica de consultas

Para aprovechar los índices, las consultas deben:

- **Filtros claros:** Usar campos indexados en las condiciones.
- **Proyección eficiente:** Solicitar solo los campos necesarios para reducir datos transferidos.
- **Ordenamiento compatible:** Ordenar por campos indexados para evitar operaciones costosas.

#### Nota

MongoDB puede usar índices para ordenar resultados, evitando un paso extra de ordenamiento en memoria.

## Conexión con próximos conceptos

En unidades siguientes, aprenderás a usar `explain()` para analizar cómo MongoDB ejecuta tus consultas y a construir pipelines de agregación que aprovechen índices y optimizaciones avanzadas.

En el desarrollo backend con Node.js y MongoDB, la eficiencia en las consultas es clave para ofrecer aplicaciones rápidas y escalables. Por ejemplo, en una API que devuelve usuarios, un índice en el campo `username` puede reducir drásticamente el tiempo de respuesta al buscar un usuario específico.

Sin embargo, si agregamos índices innecesarios o mal diseñados, podemos afectar negativamente el rendimiento de inserciones y actualizaciones, además de consumir más espacio en disco.



Por eso, entender la importancia de la indexación y cómo optimizar consultas básicas es fundamental para cualquier desarrollador backend que trabaje con bases de datos MongoDB, especialmente en proyectos con grandes volúmenes de datos o alta concurrencia.

Esta unidad te prepara para aplicar estas buenas prácticas y sentar las bases para análisis más profundos y optimizaciones avanzadas en las siguientes unidades.

## Tipos de índices en MongoDB: compuestos, multikey, text y geoespaciales

Explora los diferentes tipos de índices en MongoDB, sus estructuras, casos de uso y mejores prácticas para diseñar índices compuestos que optimicen filtros y ordenamientos simultáneos en consultas.

¿Alguna vez te has preguntado cómo MongoDB puede acelerar consultas complejas que involucran múltiples campos o datos especiales como arrays, texto o ubicaciones geográficas? En esta unidad, profundizaremos en los distintos tipos de índices que MongoDB ofrece para optimizar el acceso a datos en escenarios reales. Desde índices compuestos que combinan varios campos, hasta índices multikey para manejar arrays, pasando por índices textuales para búsquedas de texto completo y geoespaciales para datos de ubicación, entenderás cuándo y cómo aplicarlos para mejorar el rendimiento de tus aplicaciones backend. Además, aprenderás a diseñar índices compuestos que soporten filtros y ordenamientos simultáneos, una habilidad clave para optimizar consultas en bases de datos grandes. Este conocimiento es fundamental para avanzar en la optimización de consultas y preparar el terreno para trabajar con pipelines de agregación y técnicas avanzadas en las siguientes unidades.

### Tipos de índices en MongoDB

MongoDB ofrece varios tipos de índices que permiten acelerar las consultas según la naturaleza de los datos y las operaciones que realizamos. A continuación, repasamos los principales tipos que debes conocer:

#### Índices Simples y Compuestos

- **Índice simple:** Indexa un solo campo. Es útil cuando las consultas filtran o ordenan por ese campo exclusivamente.
- **Índice compuesto:** Indexa múltiples campos en un orden específico. Es ideal para consultas que filtran y/o ordenan por más de un campo simultáneamente.

#### Ejemplo

Un índice compuesto { fecha: 1, usuario: 1 } optimiza consultas que filtran por fecha y luego por usuario.

## Índices Multikey

MongoDB crea índices multikey automáticamente cuando el campo indexado es un array. Esto permite indexar cada elemento del array para consultas eficientes.

- **Uso:** Cuando los documentos contienen arrays y se realizan consultas que buscan elementos dentro de esos arrays.
- **Limitaciones:** No se pueden crear índices compuestos multikey con más de un campo array.

## Índices Textuales

Permiten realizar búsquedas de texto completo en campos de tipo string.

- **Características:** Soportan búsquedas con palabras clave, frases y operadores booleanos.
- **Uso:** Ideal para implementar funcionalidades de búsqueda en aplicaciones.
- **Nota:** Solo un índice textual por colección.

## Índices Geoespaciales (2dsphere)

Permiten consultas eficientes sobre datos geográficos en formato GeoJSON.

- **Aplicaciones:** Búsquedas por proximidad, dentro de polígonos, rutas, etc.
- **Ejemplo:** Encontrar usuarios cercanos a una ubicación específica.

## Aggregations y Pipeline

El framework de Aggregation en MongoDB permite construir pipelines para procesar datos en etapas como `$match`, `$group`, `$lookup`, `$unwind` y `$project`. Los índices adecuados, especialmente compuestos, pueden acelerar significativamente estas etapas, especialmente `$match`.

## Optimización de Consultas: Filtros, Proyección y Ordenamiento

Para aprovechar al máximo los índices, es fundamental diseñar consultas que:

- Apliquen filtros que coincidan con los campos indexados.
- Proyecten solo los campos necesarios para reducir el trabajo.
- Ordenen los resultados usando campos que formen parte del índice.

Esto reduce el trabajo de MongoDB y mejora el rendimiento.

### Nota

El orden de los campos en un índice compuesto es crucial para que MongoDB pueda usarlo eficientemente en filtros y ordenamientos.

En entornos backend con bases de datos grandes, las consultas que involucran múltiples campos o tipos de datos complejos pueden volverse lentas si no se usan índices adecuados. Por ejemplo, una

aplicación que filtra usuarios por fecha de registro y estado, o que busca documentos que contienen arrays con ciertos valores, se beneficiará enormemente de índices compuestos y multikey.

Además, las búsquedas textuales y geoespaciales son comunes en aplicaciones modernas, desde motores de búsqueda internos hasta apps de ubicación. MongoDB ofrece índices especializados para estos casos, pero su uso requiere entender sus limitaciones y mejores prácticas.

Diseñar índices compuestos que soporten simultáneamente filtros y ordenamientos es una técnica avanzada que permite consultas más rápidas y eficientes, evitando escaneos completos de colección (full scans) y mejorando la experiencia del usuario.

Esta unidad te prepara para aplicar estos conceptos en tus proyectos, optimizando consultas y sentando las bases para trabajar con pipelines de agregación y técnicas avanzadas en las siguientes unidades.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Situaciones clave donde conviene indexar

Explora los patrones comunes de consultas y accesos a datos que justifican la creación de índices en MongoDB. Aprende a identificar cuándo y qué campos indexar para optimizar el rendimiento, considerando los beneficios y costos asociados, y cómo automatizar procesos con middlewares pre en Mongoose para mejorar la eficiencia.

¿Alguna vez has notado que ciertas consultas en tu aplicación backend con MongoDB son lentas, mientras que otras son rápidas? Esto suele estar relacionado con cómo MongoDB accede a los datos y si utiliza índices adecuados. En este módulo, profundizaremos en las situaciones clave donde conviene crear índices para mejorar el rendimiento de tus consultas. Veremos patrones comunes como filtros frecuentes, ordenamientos, campos usados en joins o `$lookup`, y cómo priorizar la creación de índices en colecciones grandes. Además, exploraremos cómo los middlewares pre en Mongoose pueden ayudarte a automatizar tareas relacionadas con consultas y optimizar el flujo de datos. Al finalizar, podrás identificar qué campos indexar y cuándo hacerlo, balanceando los beneficios con los costos de mantenimiento, para que tus aplicaciones sean más rápidas y eficientes.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Situaciones clave para indexar en MongoDB

MongoDB utiliza índices para acelerar la búsqueda y recuperación de documentos. Sin índices, las consultas realizan un *full collection scan*, revisando cada documento, lo que es costoso en colecciones grandes.

## Patrones comunes que justifican la indexación

### 1. Filtros frecuentes

Los campos que usas regularmente en filtros ( `find` , `findOne` ) son candidatos ideales para índices. Por ejemplo, si consultas a menudo por `email` o `status` , un índice en esos campos reduce el tiempo de búsqueda.

#### Ejemplo

##### CÓDIGO

```
db.usuarios.find({ email: 'usuario@ejemplo.com' })
```

Un índice en `email` acelera esta consulta.

### 2. Ordenamientos

Cuando usas `sort` en consultas, MongoDB puede usar índices para ordenar sin necesidad de cargar todos los documentos en memoria.

#### Ejemplo

##### CÓDIGO

```
db.ordenes.find().sort({ fechaCreacion: -1 })
```

Un índice en `fechaCreacion` mejora el rendimiento.

### 3. Campos usados en joins / `$lookup`

En agregaciones que usan `$lookup` para unir colecciones, los campos de unión deben estar indexados para evitar escaneos completos.

#### Ejemplo

##### CÓDIGO

```
{
  $lookup: {
    from: 'productos',
    localField: 'productoId',
    foreignField: '_id',
    as: 'detalleProducto'
  }
}
```

Un índice en `productos._id` es esencial.

#### 4. Campos para unicidad

Índices únicos garantizan que no haya valores duplicados en campos clave, como correos electrónicos o números de identificación.

#### Ejemplo

##### CÓDIGO

```
db.usuarios.createIndex({ email: 1 }, { unique: true })
```

#### Priorizar índices en colecciones grandes

Aunque los índices aceleran consultas, tienen un costo:

- **Espacio en disco y memoria:** Cada índice ocupa recursos.
- **Impacto en escrituras:** Insertar, actualizar o eliminar documentos es más lento porque MongoDB debe actualizar los índices.

Por eso, es crucial evaluar el beneficio de cada índice:

- ¿Con qué frecuencia se usa el campo en consultas?
- ¿Qué impacto tiene en el rendimiento?
- ¿Cuánto afecta a las operaciones de escritura?

#### Middlewares pre en Mongoose para optimizar consultas

Mongoose permite usar *middlewares pre* ( `pre hooks` ) para ejecutar código antes de operaciones como `find` o `save`. Esto es útil para automatizar `populate` o validar datos, evitando consultas costosas o innecesarias.

### Ejemplo

#### CÓDIGO

```
schema.pre('find', function() {  
  this.populate('usuario');  
});
```

Así, cada consulta `find` automáticamente incluye la población de la referencia `usuario`.

Sin embargo, hay que usar estos middlewares con cuidado para no generar consultas profundas o múltiples que afecten el rendimiento.

En proyectos backend con Node.js, Express y MongoDB, las consultas lentas pueden afectar la experiencia del usuario y la escalabilidad. Identificar correctamente qué campos indexar según los patrones de acceso es fundamental para optimizar el rendimiento.

Por ejemplo, en una aplicación de comercio electrónico, los filtros por categoría, precio o estado del pedido son frecuentes y deben estar indexados. Además, las relaciones entre colecciones, como pedidos y productos, requieren índices en campos usados en `$lookup` para evitar cuellos de botella.

El uso de middlewares pre en Mongoose permite automatizar la inclusión de datos relacionados, pero si no se controla, puede generar consultas costosas. Por eso, entender cuándo y cómo indexar, junto con buenas prácticas en middlewares, es clave para mantener un backend eficiente y escalable.

Este conocimiento te prepara para aplicar optimizaciones en consultas reales y para el siguiente módulo, donde profundizaremos en el análisis de planes de ejecución y ajustes avanzados.

## Concepto de population en MongoDB y referencias entre colecciones

En esta unidad exploraremos cómo representar relaciones entre documentos en MongoDB usando referencias y el método `populate` de Mongoose. Aprenderás cuándo es mejor usar referencias o documentos embebidos, y cómo la indexación y optimización de consultas impactan en el rendimiento al trabajar con datos relacionados.

¿Alguna vez te has preguntado cómo manejar relaciones entre datos en MongoDB, una base de datos NoSQL que no usa tablas ni joins tradicionales? En esta unidad, descubrirás cómo MongoDB permite modelar relaciones entre documentos mediante referencias y cómo Mongoose facilita su manejo con el método `populate`. Entenderás cuándo es mejor almacenar datos embebidos dentro de un documento y cuándo usar referencias para mantener la eficiencia y escalabilidad de tus aplicaciones backend. Además, exploraremos cómo la indexación y la optimización de consultas influyen en el rendimiento cuando trabajamos con datos relacionados. Este conocimiento es clave para construir APIs robustas y rápidas con Node.js, Express y MongoDB, especialmente cuando manejas bases de datos grandes y complejas.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Referencias vs Documentos Embebidos en MongoDB

MongoDB permite dos formas principales de modelar relaciones entre datos:

- **Documentos embebidos (Embed):** Incluir un documento dentro de otro. Ideal para datos que se acceden juntos y no cambian frecuentemente.
- **Referencias (References):** Guardar el `_id` de un documento relacionado en otro documento, manteniendo colecciones separadas.

## ¿Qué es `populate` en Mongoose?

`populate` es un método que permite resolver referencias automáticamente, reemplazando los IDs almacenados con los documentos completos relacionados.

### CÓDIGO

```
// Ejemplo básico de populate
const post = await Post.findById(postId).populate('author');
```

Esto facilita trabajar con datos relacionados sin hacer múltiples consultas manuales.

## Buenas prácticas y riesgos del `populate` (profundidad, circularidad)

Explora los riesgos comunes asociados al uso de `populate` en Mongoose, como consultas N+1, `populates` en cascada y respuestas pesadas. Aprende a identificar estos problemas y aplicar técnicas efectivas para mitigarlos, incluyendo proyecciones, limitación de campos y `fetch` selectivo, mejorando el rendimiento y evitando problemas de profundidad y circularidad.

¿Alguna vez has notado que una consulta con `populate` en Mongoose tarda mucho más de lo esperado? Este fenómeno suele estar relacionado con riesgos comunes como consultas N+1, `populates` en cascada o respuestas demasiado pesadas que afectan el rendimiento de tu aplicación backend. En esta unidad, profundizaremos en estos riesgos y aprenderemos a identificarlos y mitigarlos para optimizar tus consultas y mantener la eficiencia de tus APIs. Además, entenderemos la diferencia entre un escaneo completo de colección y una consulta que aprovecha índices, un concepto clave para diagnosticar problemas de rendimiento. Al finalizar, estarás capacitado para aplicar buenas prácticas que eviten profundidades excesivas y circularidades en tus poblaciones, mejorando la experiencia de usuario y la escalabilidad de tus proyectos con MongoDB y Mongoose.

## Riesgos comunes al usar `populate` en Mongoose

El método `populate` es una herramienta poderosa para gestionar referencias entre colecciones en MongoDB, pero su uso indebido puede generar problemas de rendimiento significativos. Veamos los riesgos más frecuentes:

## 1. Consultas N+1

Este problema ocurre cuando, para obtener datos relacionados, se ejecuta una consulta inicial (N) y luego múltiples consultas adicionales (1 por cada documento) para poblar referencias. Esto genera un alto número de consultas a la base de datos, aumentando la latencia.

**Ejemplo:** Si tienes 100 documentos y haces un `populate` sin optimización, podrías estar ejecutando 1 consulta para los documentos y 100 consultas adicionales para poblar cada referencia.

## 2. Populates en cascada o profundos

Poblar referencias anidadas o múltiples niveles puede generar respuestas muy pesadas y consultas complejas, afectando la velocidad y el consumo de recursos.

## 3. Respuestas pesadas

Poblar campos innecesarios o sin limitar los datos puede resultar en respuestas con gran volumen, lo que impacta en la transferencia de datos y el procesamiento en el cliente.

## Full collection scan vs consultas con índices

Antes de optimizar `populate`, es importante entender cómo MongoDB ejecuta las consultas:

- **Full collection scan:** MongoDB revisa todos los documentos de una colección para encontrar coincidencias. Es costoso y lento en colecciones grandes.
- **Consultas con índices:** MongoDB utiliza índices para localizar rápidamente documentos relevantes, mejorando el rendimiento.

El método `explain()` permite identificar si una consulta realiza un full scan o usa índices, ayudando a diagnosticar problemas.

## Técnicas para mitigar riesgos

### Proyecciones y limitación de campos

Limita los campos que se recuperan en el `populate` usando la opción `select`. Esto reduce el tamaño de la respuesta y la carga en la base de datos.

#### CÓDIGO

```
Model.find()  
  .populate({ path: 'referencia', select: 'campo1 campo2' })  
  .exec();
```

### Fetch selectivo

Evita poblar datos que no son necesarios para la operación actual. Evalúa si realmente necesitas toda la información relacionada.



## Limitar profundidad y evitar circularidad

- Define límites claros en la profundidad de `populate` para evitar consultas en cascada.
- Evita referencias circulares que pueden generar loops infinitos o consultas redundantes.

## Buenas prácticas para evitar problemas

- Planifica el modelado de referencias para minimizar la necesidad de `populates` profundos.
- Usa agregaciones cuando necesites transformaciones complejas en lugar de múltiples `populates`.
- Aplica middlewares en Mongoose para automatizar validaciones y evitar circularidades.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

## Conexión con `explain()` para análisis de rendimiento

Utiliza `explain()` para analizar el plan de ejecución de tus consultas con `populate` y detectar si hay full scans o etapas costosas.

### CÓDIGO

```
Model.find().populate('referencia').explain();
```

Esto te ayudará a identificar cuellos de botella y ajustar índices o consultas.

En proyectos backend con Node.js, Express y MongoDB, el uso de `populate` es común para manejar relaciones entre colecciones. Sin embargo, en bases de datos grandes o con estructuras complejas, un uso inadecuado puede degradar el rendimiento y afectar la experiencia del usuario.

Por ejemplo, una API que devuelve datos de usuarios junto con sus posts y comentarios puede sufrir si se hacen `populates` profundos sin control, generando múltiples consultas y respuestas pesadas.

Comprender los riesgos y aplicar técnicas para mitigarlos es esencial para construir APIs eficientes y escalables. Además, el conocimiento sobre cómo MongoDB ejecuta consultas (full scan vs índices) permite diagnosticar y optimizar consultas con mayor precisión.

Esta unidad te prepara para identificar estos problemas en tus proyectos y aplicar soluciones prácticas que mejoren el rendimiento y la mantenibilidad de tus aplicaciones.

## Full collection scan vs consultas optimizadas con índices

En esta unidad exploraremos la diferencia fundamental entre un full collection scan y una consulta que utiliza índices en MongoDB. Aprenderás a identificar cuándo una consulta está realizando un escaneo completo de la colección, las señales que indican consultas no optimizadas y cómo evitar problemas comunes relacionados con el uso de `populate` que pueden afectar el rendimiento.

¿Alguna vez has notado que una consulta en MongoDB tarda mucho más de lo esperado sin razón aparente? Muchas veces, la causa es que MongoDB está realizando un *full collection scan*, es decir, escaneando todos los documentos de una colección para encontrar los resultados, en lugar de usar un índice que acelere la búsqueda. En esta unidad, descubrirás cómo diferenciar entre estos dos tipos de consultas y cómo reconocer las señales que indican que una consulta no está optimizada. Además, revisaremos buenas prácticas para el uso de `populate` en Mongoose, una herramienta poderosa pero que puede generar problemas de rendimiento si no se usa con cuidado. Al finalizar, estarás preparado para identificar consultas costosas y sentar las bases para optimizarlas en unidades posteriores.

## Full Collection Scan vs Consultas con Índices

En MongoDB, una **consulta puede ejecutarse de dos formas principales**:

- **Full Collection Scan:** MongoDB revisa cada documento de la colección para encontrar los que cumplen con el filtro. Esto ocurre cuando no hay un índice adecuado para la consulta.
- **Consulta con Índice:** MongoDB utiliza un índice para localizar rápidamente los documentos relevantes, evitando revisar toda la colección.

**¿Por qué es importante?** Un full collection scan puede ser muy costoso en colecciones grandes, afectando el rendimiento y la escalabilidad de tu aplicación.

## ¿Cómo identificar un Full Collection Scan?

Cuando ejecutas una consulta, puedes usar el método `explain()` para obtener el plan de ejecución. Un plan que indica `COLLSCAN` significa que se está haciendo un full collection scan.

### CÓDIGO

```
db.usuarios.find({ edad: { $gt: 30 } }).explain('executionStats')
```

En el resultado, busca:

- `stage: "COLLSCAN"` → escaneo completo
- `stage: "IXSCAN"` → uso de índice

## Ejemplo ilustrativo

Supongamos una colección `productos` con 1000 documentos y un índice en el campo `categoria`.

- Consulta sin índice:

### CÓDIGO

```
db.productos.find({ precio: { $lt: 100 } }).explain('executionStats')
```

Si no hay índice en `precio`, verás `COLLSCAN` y un alto número de documentos examinados.

- Consulta con índice:

#### CÓDIGO

```
db.productos.createIndex({ categoria: 1 })
db.productos.find({ categoria: "Electrónica" }).explain('executionStats')
```

Aquí, el plan mostrará `IXSCAN` y menos documentos examinados.

## Buenas prácticas y riesgos del uso de `populate`

El método `populate` de Mongoose permite traer documentos relacionados de otras colecciones, pero puede generar consultas costosas si:

- Se hacen `populates` profundos o anidados excesivamente.
- Se generan múltiples consultas en cascada (N+1 problem).

### Recomendaciones:

- Limita la profundidad de `populate`.
- Selecciona solo los campos necesarios.
- Considera usar agregaciones con `$lookup` para consultas más complejas.

#### Ejemplo

Evita hacer `populate` dentro de un ciclo que recorra muchos documentos, ya que genera muchas consultas individuales.

**Resumen:** Entender cuándo ocurre un full collection scan y cómo evitarlo es clave para optimizar tus consultas. Además, usar `populate` con criterio evitará impactos negativos en el rendimiento.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

En el desarrollo backend con Node.js y MongoDB, las consultas lentas pueden afectar la experiencia del usuario y la escalabilidad del sistema. Identificar si una consulta está haciendo un full collection scan es el primer paso para mejorar su rendimiento. Por ejemplo, en una API que devuelve productos filtrados, un full collection scan puede generar tiempos de respuesta altos y carga innecesaria en la base de datos.

Además, el uso de `populate` en Mongoose, aunque facilita la obtención de datos relacionados, puede introducir consultas adicionales que, si no se controlan, degradan el rendimiento. Por eso, es fundamental aplicar buenas prácticas para evitar consultas profundas o innecesarias.

Esta unidad te prepara para detectar estos problemas en tus consultas reales, usando herramientas como `explain()`, y para aplicar estrategias que mejoren la eficiencia, preparando el camino para optimizaciones más avanzadas en unidades posteriores.

## Middlewares pre en Mongoose para automatizar populate

Aprende a implementar middlewares `pre('find')` y `pre('findOne')` en Mongoose para automatizar la función `populate`, optimizando consultas y manteniendo un código limpio y eficiente. Evalúa el impacto en rendimiento y conoce cuándo es adecuado usar estos middlewares en proyectos backend con MongoDB.

¿Alguna vez has tenido que repetir la misma lógica de `populate` en múltiples consultas en tus modelos de Mongoose? Esto no solo genera código repetitivo, sino que también puede llevar a errores y dificultades para mantener el proyecto. En este módulo, exploraremos cómo los middlewares pre en Mongoose pueden automatizar estas tareas, aplicando `populate` de forma transparente antes de ejecutar consultas como `find` o `findOne`.

Además, repasaremos cuándo es clave indexar tus colecciones para optimizar estas consultas, reforzando conceptos previos sobre patrones de acceso y filtros frecuentes. Al finalizar, podrás implementar middlewares que mejoren la eficiencia y limpieza de tu código, evaluando también su impacto en el rendimiento y alternativas para evitar consultas costosas. Este conocimiento es esencial para cualquier desarrollador backend que busque optimizar aplicaciones Node.js con MongoDB y Mongoose.

**IMAGEN** Imagen de referencia del material original (no disponible en esta versión de lectura).

### Middlewares pre en Mongoose: Automatización de populate

Mongoose permite definir middlewares que se ejecutan antes o después de ciertas operaciones. Los **middlewares pre** ( `pre hooks` ) se ejecutan antes de la operación especificada, permitiendo modificar la consulta o el documento.

#### ¿Qué es un middleware pre?

Un middleware pre es una función que intercepta una operación, como `find` o `save`, y puede modificar el comportamiento antes de que se ejecute. En el contexto de consultas, se usa para agregar automáticamente `populate` sin repetir código.

#### CÓDIGO

```
schema.pre(['find', 'findOne'], function(next) {
  this.populate('campoRelacionado');
  next();
});
```

Este ejemplo aplica `populate` automáticamente en todas las consultas `find` y `findOne`.

## Situaciones clave para indexar (REINFORCE)

Para que estas consultas con `populate` sean eficientes, es fundamental tener índices adecuados. Recuerda que:

- **Filtros frecuentes:** Campos usados en filtros ( `find({ campo: valor })` ) deben estar indexados para evitar `full collection scans`.
- **Ordenamientos:** Campos usados en `sort()` también se benefician de índices.
- **Joins/lookup:** Las referencias usadas en `populate` deben estar indexadas para acelerar la búsqueda.

### Nota

Indexar mejora el rendimiento de lectura, pero tiene un costo en escritura y almacenamiento. Por eso, es clave indexar solo cuando el patrón de acceso lo justifique.

## Impacto en rendimiento y mantenimiento

- **Ventajas:**
  - Código más limpio y DRY (Don't Repeat Yourself).
  - Consistencia en las consultas.
- **Desventajas:**
  - Puede generar consultas más pesadas si se abusa del `populate` automático.
  - Dificulta el control granular de qué campos se populan en cada consulta.
  - Impacto en rendimiento si se populan campos con grandes volúmenes de datos o relaciones profundas.

## Buenas prácticas

- Limitar los campos a popular con `.populate('campo', 'subcampo1 subcampo2')`.
- Evitar `populates` profundos o circulares.
- Evaluar si el middleware es la mejor opción o si conviene popular explícitamente en cada consulta.

Para más detalles, consulta la documentación oficial de Mongoose sobre `populate`: [Mongoose Populate](#)

En proyectos backend con Node.js y MongoDB, es común manejar relaciones entre colecciones usando referencias y `populate` en Mongoose. Sin embargo, repetir la lógica de `populate` en cada consulta puede ser tedioso y propenso a errores.

Implementar middlewares pre para automatizar `populate` ayuda a mantener el código limpio y consistente, especialmente en APIs REST donde múltiples endpoints requieren datos relacionados.

Además, entender cuándo y cómo indexar los campos relacionados es crucial para que estas consultas automáticas no degraden el rendimiento. Por ejemplo, si consultas frecuentes filtran por un campo referenciado, ese campo debe estar indexado para evitar escaneos completos de la colección.

Este conocimiento permite diseñar consultas eficientes y escalables, mejorando la experiencia del usuario y reduciendo la carga en la base de datos.

En la siguiente sección, aplicaremos estos conceptos creando middlewares pre para automatizar populate y evaluaremos su impacto en un caso práctico.

## Aggregations en MongoDB y etapas del pipeline (\$match, \$group, etc.)

En esta unidad exploraremos cómo construir pipelines de Aggregation en MongoDB para filtrar, transformar y agrupar datos de manera eficiente. Reforzaremos conceptos clave sobre tipos de índices y aplicaremos técnicas de optimización en consultas usando filtros, proyecciones y ordenamientos. Al finalizar, podrás decidir cuándo usar Aggregations frente a consultas simples para maximizar el rendimiento en tus aplicaciones backend.

¿Alguna vez te has preguntado cómo procesar y analizar grandes volúmenes de datos en MongoDB de forma eficiente? Las consultas simples son útiles, pero cuando necesitas filtrar, agrupar y transformar datos complejos, el framework de Aggregation es tu mejor aliado. En esta unidad, reforzaremos tu conocimiento sobre los tipos de índices en MongoDB y aprenderás a construir pipelines de Aggregation usando etapas clave como `$match`, `$group`, `$lookup`, `$unwind` y `$project`. Además, aplicaremos técnicas para optimizar tus consultas, mejorando el rendimiento de tus aplicaciones backend con Node.js y Mongoose. Prepárate para llevar tus habilidades al siguiente nivel y tomar decisiones informadas sobre cuándo usar Aggregations o consultas simples para maximizar la eficiencia.

### Tipos de índices en MongoDB (Reforzando conceptos)

MongoDB ofrece varios tipos de índices que aceleran las consultas y mejoran el rendimiento:

- **Índices simples:** Indexan un solo campo, ideales para consultas directas.
- **Índices compuestos:** Indexan múltiples campos, útiles para consultas que filtran por varios criterios.
- **Índices multikey:** Permiten indexar campos que contienen arrays, facilitando búsquedas dentro de listas.
- **Índices textuales:** Soportan búsquedas de texto completo en campos string.
- **Índices geoespaciales:** Para consultas basadas en ubicación y proximidad.

### Nota

La correcta elección y uso de índices es fundamental para que las etapas del pipeline de Aggregation sean eficientes y no provoquen escaneos completos.

## Framework de Aggregation y pipeline

El framework de Aggregation en MongoDB permite construir pipelines que procesan documentos en etapas secuenciales para filtrar, transformar y agrupar datos.

### Etapas comunes del pipeline

- **\$match** : Filtra documentos según criterios específicos, similar a un filtro `find()` . Es recomendable usarlo temprano para reducir el volumen de datos.
- **\$group** : Agrupa documentos por un campo o expresión y calcula agregados como sumas, promedios, conteos.
- **\$lookup** : Realiza un join con otra colección, permitiendo combinar datos relacionados.
- **\$unwind** : Descompone arrays en documentos individuales para procesarlos por separado.
- **\$project** : Modifica la forma de los documentos, incluyendo o excluyendo campos, o creando nuevos campos calculados.

Estos operadores permiten construir pipelines potentes para análisis y transformación de datos complejos.

### Optimización de consultas: filtros, proyección y ordenamiento

Para maximizar el rendimiento:

- Aplica **\$match** lo antes posible para reducir datos procesados.
- Usa **\$project** para limitar campos y reducir carga.
- Ordena con **\$sort** después de filtrar para aprovechar índices.

El método `explain()` es útil para analizar el plan de ejecución y detectar posibles cuellos de botella.

### Cuándo usar Aggregations vs consultas simples

- Usa **consultas simples** ( `find()` ) para operaciones directas y sencillas.
- Prefiere **Aggregations** para transformaciones complejas, agrupamientos y joins.

Elegir correctamente evita sobrecargar la base de datos y mejora la experiencia del usuario.

En el desarrollo backend con Node.js y MongoDB, es común enfrentarse a escenarios donde se requiere procesar datos complejos para reportes, dashboards o APIs que entregan información agregada. Por ejemplo, una aplicación de e-commerce puede necesitar calcular ventas totales por categoría, combinar datos de productos con reseñas o paginar resultados con filtros avanzados.

El uso adecuado de pipelines de Aggregation permite realizar estas tareas directamente en la base de datos, evitando la transferencia innecesaria de datos y procesamiento en la aplicación, lo que mejora la escalabilidad y tiempos de respuesta.

Además, comprender cómo los índices afectan estas operaciones es clave para diseñar consultas eficientes y evitar escaneos completos que degradan el rendimiento.

En esta unidad, veremos ejemplos prácticos que podrás aplicar en tus proyectos para optimizar consultas y construir pipelines efectivos con MongoDB y Mongoose.

**IMAGEN** *Imagen de referencia del material original (no disponible en esta versión de lectura).*



## MÓDULO 10

# Proyecto Final

---

Consolida tu conocimiento sobre Programación Backend I con un Proyecto Final.

## Evaluación Final: Proyecto backend con Node.js y Express

**Llegaste a la instancia final del curso, donde vas a integrar todo lo aprendido en el desarrollo de un proyecto backend completo.**

A lo largo de la cursada trabajaste sobre la construcción progresiva de una aplicación, incorporando desde lógica de programación hasta persistencia de datos y comunicación en tiempo real.

En esta evaluación, deberás llevar ese proyecto a una versión más robusta y profesional, aplicando buenas prácticas de desarrollo backend.

**En este proyecto pondrás en juego los principales conceptos trabajados durante el curso:**

- Desarrollo de servidores con Node.js y Express
- Organización de rutas mediante Express Router
- Manejo de asincronía con async/await
- Persistencia de datos con FileSystem y MongoDB (Mongoose)
- Modelado de datos y relaciones
- Implementación de CRUD completo
- Uso de WebSockets para actualización en tiempo real
- Optimización de consultas mediante filtros, paginación y ordenamiento

El objetivo es que puedas integrar estos conceptos en una aplicación funcional, similar a un entorno real de desarrollo backend.

**Vas a trabajar sobre una API de e-commerce, que permite gestionar:**

- Productos
- Carritos de compra

Este tipo de aplicación representa un caso real de uso en la industria, donde el backend se encarga de:

- Exponer endpoints para consumir datos
- Gestionar lógica de negocio
- Persistir información en base de datos

- Mantener sincronización con el frontend

A lo largo del curso fuiste construyendo esta aplicación por partes. En esta instancia, deberás consolidarla, mejorarla y completarla.

A partir del proyecto que desarrollaste durante la cursada, deberás entregar una versión final que cumpla con los siguientes requisitos:

## 1. Servidor

- Implementar un servidor con Node.js y Express
- El servidor debe ejecutarse en el puerto 8080
- Organizar las rutas en:
  - /api/products
  - /api/carts

## 2. Gestión de productos

### GET /api/products

Debe permitir:

- limit (cantidad de resultados)
- page (paginación)
- query (filtro por categoría o disponibilidad)
- sort (orden ascendente o descendente por precio)

Valores por defecto:

- limit = 10
- page = 1

Formato de respuesta:

```
{  
  "status": "success",  
  "payload": [],  
  "totalPages": 0,  
  "prevPage": null,  
  "nextPage": null,
```

```
"page": 1,  
"hasPrevPage": false,  
"hasNextPage": false,  
"prevLink": null,  
"nextLink": null  
}
```

### **GET /api/products/:pid**

- Obtener un producto por ID

### **POST /api/products**

- Crear producto con los siguientes campos:
  - title
  - description
  - code
  - price
  - status
  - stock
  - category
  - thumbnails
- El ID debe generarse automáticamente

### **PUT /api/products/:pid**

- Actualizar producto existente
- No se debe modificar el ID

### **DELETE /api/products/:pid**

- Eliminar producto

## **3. Gestión de carritos**

### **POST /api/carts**

- Crear carrito con ID autogenerado

### **GET /api/carts/:cid**

- Listar productos del carrito
- Utilizar populate para traer información completa de los productos

### **POST /api/carts/:cid/products/:pid**

- Agregar producto al carrito
- Si ya existe, incrementar la cantidad

### **DELETE /api/carts/:cid/products/:pid**

- Eliminar producto del carrito

### **PUT /api/carts/:cid**

- Actualizar todos los productos del carrito

### **PUT /api/carts/:cid/products/:pid**

- Actualizar únicamente la cantidad de un producto

### **DELETE /api/carts/:cid**

- Vaciar el carrito completo

## **4. Persistencia**

- Implementar MongoDB con Mongoose
- Base de datos: ecommerce
- Colecciones:
  - products
  - carts

Mantener la estructura del proyecto con:

- carpeta dao
- carpeta models

No eliminar la implementación previa con FileSystem

## 5. Vistas y tiempo real

### /products

- Listado de productos con paginación

### /products/:pid

- Vista de detalle del producto
- Opción para agregar al carrito

### /carts/:cid

- Visualización de un carrito específico

### WebSockets

- Implementar actualización en tiempo real de productos
- Los cambios deben reflejarse automáticamente en la vista

### Requisitos técnicos

- Uso de Express Router
- Uso de middleware
- Manejo de asincronía con async/await
- Uso de Mongoose
- Código modular y organizado
- Manejo básico de errores

### Entregable

#### Tipo: URL

El proyecto debe estar **desarrollado y funcionando**, pero la entrega se realizará en formato de presentación para permitir su correcta evaluación.

#### Se deberá entregar:

Un Google Slides (URL pública) que documente el desarrollo completo del proyecto.

#### La presentación debe incluir:

**1. Definición del proyecto:** Nombre, problema que resuelve, público objetivo y funcionalidades principales.

**2. Evidencia del desarrollo técnico (obligatorio):** Capturas del código (archivos clave, estructura de carpetas, endpoints, lógica implementada). Fragmentos relevantes del código explicados.

**3. Estructura del proyecto:** Organización del backend (rutas, controllers, models, etc.) Explicación de la arquitectura utilizada.

**4. Implementación de funcionalidades:** Evidencia del funcionamiento de los endpoints (capturas de Postman/Thunder Client o similar). Ejemplos de requests y responses.

**5. Persistencia de datos:** Capturas y explicación del uso de base de datos (MongoDB / FileSystem).

## **6. Funcionalidades clave**

- Evidencia del CRUD completo.
- Gestión de productos y carritos.
- Casos relevantes del sistema.

**7. Tiempo real (si aplica):** Implementación de WebSockets y evidencia de funcionamiento.

**8. Evidencia de funcionamiento (obligatorio):** Capturas, GIF o video mostrando el uso completo del sistema.

## **9. Repositorio (referencia)**

Link a GitHub con el proyecto completo.

(No será evaluado directamente por la herramienta automática, pero debe existir como respaldo del desarrollo).

## **10. Cierre del proyecto**

- Dificultades encontradas.
- Soluciones implementadas.
- Mejoras futuras.

## **Condiciones importantes**

- La aplicación **debe estar desarrollada y funcional**, aunque no se evalúe directamente el repositorio.
- El Slides debe permitir **validar el desarrollo técnico a través de evidencia concreta**.
- No se evaluarán entregas únicamente teóricas o sin capturas del funcionamiento real.

## **Tipo de entrega**

- Formato: Google Slides
- Entrega: URL pública
- El contenido debe ser claro, ordenado y suficiente para validar el desarrollo

### Rubrica — Evaluación Final: Backend Node.js (Entrega en Google Slides)

Evalúa, mediante la presentación (Google Slides) entregada, si el proyecto backend desarrollado cumple los requisitos funcionales, de persistencia y de evidencia solicitados en la instrucción (servidor, endpoints de productos y carritos, persistencia con MongoDB/Mongoose y evidencia de vistas/tiempo real). La evaluación se basa exclusivamente en los elementos explícitamente requeridos en las instrucciones y en la presentación pública indicada.

| Criterio                                                      | Descripción                                                                                                                                                                                                                                                                                                                                                                                                                                 | Peso |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Funcionalidad del servidor y endpoints (Productos y Carritos) | Verifica que la presentación documente y muestre evidencia del servidor Node.js/Express en puerto 8080 y del correcto funcionamiento de los endpoints solicitados para /api/products y /api/carts, incluyendo los parámetros, comportamientos y formatos de respuesta exigidos en las instrucciones.                                                                                                                                        | 50%  |
| Persistencia y estructura del proyecto                        | Comprueba que la presentación documente el uso de MongoDB con Mongoose (base de datos 'ecommerce' y colecciones 'products' y 'carts'), la existencia de las carpetas requeridas (dao y models) y que la implementación previa con FileSystem se mantiene (no eliminada). La evidencia debe permitir validar la persistencia y la organización del proyecto tal como se solicitó.                                                            | 25%  |
| Presentación pública (Google Slides) y evidencia demostrable  | Evalúa si el Google Slides público incluye todas las secciones y evidencias pedidas en el enunciado, de forma que permita validar el desarrollo técnico: definición del proyecto, capturas de código y estructura, evidencia de endpoints (requests/responses), persistencia, CRUD, tiempo real (WebSockets) si aplica, evidencia de funcionamiento (capturas/GIF/video), link al repositorio y cierre con dificultades/soluciones/mejoras. | 25%  |

Total: 100 pts | Aprobación: 65 pts



## APÉNDICE — Material complementario y cierre

### Palabras clave

**Backend:** El backend se refiere a la parte de una aplicación que ejecuta la lógica del servidor, gestiona el acceso y la persistencia de datos y procesa las peticiones de los clientes. Incluye servidores, bases de datos y la lógica de negocio que no es visible directamente para el usuario final. En proyectos modernos, el backend se diseña para ser escalable, seguro y eficiente, utilizando patrones como APIs RESTful o arquitecturas de microservicios. **Node.js:** Node.js es un entorno de ejecución de JavaScript basado en el motor V8 que permite ejecutar código JavaScript fuera del navegador en el servidor. Su modelo de E/S no bloqueante y basado en eventos lo hace ideal para aplicaciones de red y APIs que requieren alta concurrencia. En la práctica se usa junto con frameworks como **Express** para construir servidores HTTP, manejar rutas, middlewares y conectar con bases de datos. **Express.js:** Express.js es un framework minimalista para Node.js que facilita la creación de servidores y APIs HTTP mediante una API sencilla para definir rutas, middlewares y controladores. Proporciona abstracciones para manejar peticiones, respuestas, parsing de cuerpos ( `body-parser` ), y composición de middlewares para tareas transversales como autenticación o logging. Es la base de muchas arquitecturas backend en JavaScript por su flexibilidad y ecosistema. **MongoDB:** MongoDB es una base de datos NoSQL orientada a documentos que almacena registros en formatos BSON/JSON, lo que ofrece flexibilidad en esquemas y escalabilidad horizontal. Es especialmente útil para aplicaciones con requisitos cambiantes de datos o modelos denormalizados, permitiendo consultas ricas, agregaciones y diferentes tipos de índices. En producción suele desplegarse en clústeres o servicios gestionados como MongoDB Atlas para alta disponibilidad. **Mongoose:** Mongoose es un ODM (Object Document Mapper) para Node.js que proporciona una capa de modelado sobre MongoDB, permitiendo definir **Schemas**, validaciones, hooks (middlewares) y métodos estáticos/ de instancia. Facilita la interacción con la base de datos al mapear documentos a objetos JavaScript con contratos más explícitos, reduciendo errores y simplificando operaciones CRUD y relaciones mediante `populate`. **REST y APIs HTTP:** REST es un conjunto de principios arquitectónicos para diseñar APIs que utilizan HTTP para manipular recursos a través de métodos como `GET`, `POST`, `PUT`, `PATCH` y `DELETE`. Una API REST bien diseñada expone endpoints claros, usa códigos de estado HTTP apropiados y mantiene separación entre cliente y servidor. En backend, respetar principios REST y diseñar rutas coherentes mejora la mantenibilidad y la interoperabilidad entre servicios. **WebSockets:** WebSockets es un protocolo que permite una comunicación bidireccional y persistente entre cliente y servidor, superando las limitaciones del modelo request/response de HTTP para aplicaciones en tiempo real. Permite enviar mensajes desde el servidor al cliente sin que el cliente realice una petición explícita cada vez, siendo útil en chats, notificaciones en tiempo real y dashboards. Bibliotecas como `Socket.IO` abstraen detalles de transporte y reconexión. **Indexación en MongoDB:** La indexación en MongoDB consiste en crear estructuras que aceleran la búsqueda y ordenamiento de documentos a costa de espacio adicional y sobrecarga en escritura. Existen índices simples, compuestos, multikey, de texto y geoespaciales, cada uno

optimizando diferentes patrones de consulta. Usar índices adecuados mejora considerablemente el rendimiento de lecturas, pero requiere monitoreo con `explain()` y diseño consciente para evitar índices innecesarios. **Middleware:** Un middleware es una función intermedia que procesa la petición y/o respuesta en una cadena antes de llegar al controlador final. En `Express` los middlewares se usan para tareas como autenticación, validación, logging, manejo de errores y parseo de cuerpos ( `JSON` o `form-data` ). Los middlewares permiten separar responsabilidades y componer comportamientos reutilizables en la pipeline de la aplicación. **Variables de Entorno y Seguridad:** Las variables de entorno sirven para almacenar configuraciones sensibles y específicas del entorno, como cadenas de conexión, claves secretas y parámetros de configuración. Mantener estos valores fuera del código fuente, por ejemplo usando `process.env` y archivos `.env` (no versionados), reduce riesgos de exposición. En producción se recomienda usar secretos gestionados por el proveedor o soluciones de vault para mayor seguridad.

## Análisis profundo

### Manejo de asincronía con Async/Await y Promesas

Comprender la asincronía es fundamental para construir servidores Node.js robustos y legibles. Async/await y las promesas transformaron la manera de escribir código asíncrono, eliminando el "callback hell" y facilitando el flujo de control y el manejo de errores. Al usar `async` y `await` puedes escribir secuencias asíncronas que se leen casi como código síncrono, pero es crucial manejar excepciones con `try/catch` y entender que `await` bloquea la ejecución de la función asíncrona, no del event loop.

En la práctica, combina promesas y `Promise.all` para ejecutar operaciones concurrentes cuando no dependen entre sí, y evita usar `await` en bucles si puedes paralelizar con `map` y `Promise.all`. Además, ten en cuenta las implicaciones de memoria y límites de concurrencia cuando lanzas muchas operaciones simultáneas; usa técnicas como pools o librerías como `p-limit` cuando sea necesario.

## Buenas prácticas y ejemplos

Usa `try/catch` para capturar errores y propagar mensajes claros a la capa HTTP. Cuando trabajes con acceso a bases de datos, asegúrate de cerrar conexiones y manejar timeouts. A continuación un ejemplo típico:

#### CÓDIGO

```
async function getUserAndPosts(userId) {
  try {
    const user = await User.findById(userId);
    const posts = await Post.find({ author: userId });
    return { user, posts };
  } catch (err) {
    // Manejo y logging
    throw err;
  }
}
```

Para optimizar, si `user` y `posts` son independientes puedes paralelizarlos:

#### CÓDIGO

```
const [user, posts] = await Promise.all([
  User.findById(userId),
  Post.find({ author: userId })
]);
```

Cierre: dominar promesas y `async/await` te permite escribir código más predecible y mantener mejor el flujo de errores, pero exige disciplina para evitar bloqueos innecesarios y fugas de recursos.

## Diseño de APIs RESTful y buenas prácticas

Diseñar una API RESTful coherente es clave para la mantenibilidad y escalabilidad de aplicaciones backend. Una buena API define recursos claros, usa métodos HTTP apropiados, devuelve códigos de estado significativos y documenta contratos de entrada/salida. Además, la separación entre rutas, controladores, servicios y modelos facilita pruebas unitarias y reutilización.

En el diseño práctico, prioriza la claridad de los endpoints y la consistencia en el manejo de errores. Usa paginación para colecciones grandes, filtros y proyecciones para limitar datos retornados y versionado de la API cuando introduces cambios incompatibles. Autenticación y autorización deben implementarse como middlewares reutilizables, y el uso de estándares como OAuth2 o JWT suele ser habitual en aplicaciones modernas.

## Ejemplo y consideraciones

Evita endpoints procedurales (por ejemplo `/getUserAndPosts`) y prefiere recursos RESTful:

#### CÓDIGO

```
GET /users/:id
GET /users/:id/posts
POST /users
PATCH /posts/:id
DELETE /posts/:id
```

Documenta tu API con herramientas como OpenAPI/Swagger para facilitar la integración con clientes y pruebas automáticas. Prioriza respuestas que incluyan metadatos útiles (paginación, límites, enlaces) y considera usar HATEOAS para APIs hipertextuales si el dominio lo justifica. Finalmente, mide y monitorea latencia, tasas de error y uso de endpoints para iterar sobre diseño y rendimiento.

## Modelado de datos en MongoDB con Mongoose

Modelar datos en MongoDB requiere equilibrar flexibilidad y consistencia. Aunque MongoDB permite esquemas dinámicos, usar `Mongoose` para definir `Schemas` aporta validaciones, tipos y middlewares que ayudan a mantener integridad lógica. Diseñar modelos implica decidir entre documentos embebidos o referencias según el patrón de acceso: embebidos para datos que se leen/escriben juntos y referencias ( `populate` ) para relaciones que se consultan de manera independiente.

Al diseñar esquemas considera índices en campos de consulta frecuente, límites de tamaño de documentos y la necesidad de normalizar frente a denormalizar. `Mongoose` ofrece middlewares `pre / post` para operaciones como `save` o `find` , lo que es útil para tareas como hashing de contraseñas o `populate` automático.

## Ejemplo práctico

Un ejemplo simple de esquema con validación y hook:

#### CÓDIGO

```
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  profile: { bio: String, avatar: String }
});

userSchema.pre('save', async function(next) {
  if (this.isModified('password')) {
    this.password = await hashPassword(this.password);
  }
  next();
});
```

Cierre: pensar en patrones de acceso y en escalabilidad te ayudará a elegir entre embebidos y referencias, y Mongoose te da herramientas para aplicar reglas y automatizaciones que mantienen la base de datos coherente.

## Optimización de consultas e indexación en MongoDB

La optimización de consultas en MongoDB pasa por entender índices, patrones de acceso y la herramienta `explain()` para analizar planes de ejecución. Crear índices adecuados acelera búsquedas y ordenamientos, pero incrementa el coste en inserciones/actualizaciones y consumo de almacenamiento. Por eso es importante medir antes de indexar y evitar índices redundantes.

Usa índices compuestos para consultas que filtran por múltiples campos, índices multikey para arrays y `text` para búsquedas de texto. Emplea `explain('executionStats')` para revisar si una consulta realiza un COLLSCAN o utiliza un índice, y ajusta proyecciones para reducir I/O retornando solo los campos necesarios. Para conjuntos de datos grandes, combina paginación basada en cursores con límites seguros.

## Herramientas y ejemplo

Un uso típico de `explain`:

### CÓDIGO

```
const plan = await db.collection('posts').find({ author: id }).sort({ createdAt: -1 }).explain('executionStats');
console.log(plan.executionStats);
```

Además considera `aggregation` pipelines para transformaciones complejas y realiza pruebas de carga para validar impacto de índices. Finalmente, monitoriza métricas de rendimiento y ajusta índices conforme evolucionan los patrones de uso.

## Seguridad en conexiones y manejo de secretos

La seguridad en el backend abarca desde la protección de credenciales hasta la validación de entradas y la gestión de permisos. Mantener cadenas de conexión, claves y secretos fuera del código fuente es esencial; utiliza variables de entorno y servicios de gestión de secretos en producción. Asegura las conexiones a bases de datos usando TLS/SSL y restringe el acceso mediante reglas de red, IP whitelisting o VPC cuando sea posible.

Además de proteger secretos, valida y sanitiza entradas para prevenir inyección, aplica políticas de CORS estrictas, y maneja errores sin filtrar información sensible al cliente. Implementa autenticación robusta (por ejemplo JWT con expiración y rotación de refresh tokens) y autorización basada en roles cuando sea necesario. Para despliegues en nube, aprovecha features del proveedor como secretos gestionados y logs centralizados para auditoría.

## Recomendaciones prácticas

- No comites archivos `.env` al repositorio.
- Usa `process.env` y valida la presencia de variables críticas al inicio de la aplicación.
- Configura conexiones a MongoDB con opciones de TLS y parámetros de tiempo de espera.

Cerrar: una estrategia de seguridad proactiva y herramientas adecuadas reducen riesgos operativos y protegen tanto la infraestructura como los datos de tus usuarios.

## Preguntas frecuentes

**P:** ¿Cómo estructuro un proyecto Express para mantenerlo escalable?

**R:** Una estructura modular mejora la mantenibilidad y permite escalar el proyecto con el tiempo. Comienza separando responsabilidades en carpetas como `routes`, `controllers`, `services` y `models`, donde `routes` definen endpoints, `controllers` manejan la lógica de entrada/salida, `services` encapsulan la lógica de negocio y `models` contienen los esquemas de datos. Añade una carpeta `middlewares` para autenticación, validación y logging, y `utils` para helpers comunes.

Este patrón facilita pruebas unitarias y permite que equipos trabajen en capas diferentes sin conflictos. Un ejemplo simple de arranque es crear un `app.js` que registre middlewares globales y monte routers por dominio, y un `server.js` que inicie la conexión a la base de datos y el servidor HTTP. Mantén además un archivo `config` o usa variables de entorno para separar configuración de código.

**P:** ¿Cuándo usar Mongoose en lugar del driver nativo de MongoDB?

**R:** Usa Mongoose cuando necesites esquemas claros, validaciones integradas, middlewares (`pre/post` hooks) y una API orientada a objetos que facilite el desarrollo rápido con contratos definidos. Mongoose es ideal para equipos que prefieren estructuras y convivencia con modelos bien definidos.

El driver nativo es preferible cuando buscas máximo rendimiento, necesitas control fino sobre operaciones avanzadas del driver o deseas evitar la sobrecarga de abstraer esquemas rígidos. En proyectos que manejan modelos muy dinámicos o realizan agregaciones complejas a gran escala, el driver nativo puede ser más directo. En muchos proyectos se combinan ambas aproximaciones según necesidades.

**P:** ¿Cómo depuro consultas lentas en MongoDB?

**R:** Primero reproduce la consulta y usa `explain('executionStats')` para analizar el plan de ejecución y ver si se realiza un COLLSCAN. Revisa los índices existentes con `db.collection.getIndexes()` y compara con los campos utilizados en filtros y ordenamientos. Si la consulta no usa un índice adecuado, crea un índice compuesto o multikey según corresponda.

También verifica la proyección para retornar solo campos necesarios y considera usar paginación basada en cursores en lugar de `skip` con grandes offsets. Para transformaciones complejas, explora

pipelines de `aggregation` y evalúa el uso de `allowDiskUse` si es necesario. Finalmente, monitoriza métricas de I/O y latencia en el clúster para identificar cuellos de botella.

**P:** ¿Cómo implemento subida de archivos con Multer en Express?

**R:** Multer es un middleware que procesa `multipart/form-data` y facilita recibir archivos en Express. Configura un storage (por ejemplo `diskStorage` o un middleware personalizado para enviarlos a cloud storage) y define límites y filtros para validar tipo y tamaño de archivo. Luego usa el middleware en la ruta correspondiente para extraer el archivo y acceder a `req.file` o `req.files`.

Ejemplo básico:

#### CÓDIGO

```
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });

app.post('/upload', upload.single('avatar'), (req, res) => {
  // req.file contiene información del archivo
  res.send({ file: req.file });
});
```

En producción valida tipos MIME, aplica límites razonables y considera almacenar archivos en servicios como S3 o soluciones equivalentes para disponibilidad y escalabilidad.

**P:** ¿Cuál es la mejor forma de paginar resultados en APIs REST?

**R:** La forma más robusta y escalable es usar paginación basada en cursores (cursor-based pagination), que evita los problemas de `skip` en conjuntos de datos grandes y mejora la consistencia cuando hay escrituras concurrentes. Para casos sencillos, la paginación por `offset` y `limit` es más fácil de implementar y suficiente en datasets pequeños o moderados.

Incluye metadatos en la respuesta como `total`, `limit`, `nextCursor` o `page` para que los clientes puedan navegar. En MongoDB, la paginación por cursores se puede implementar con filtros que utilicen un campo ordenado (por ejemplo `createdAt` y `_id`) para obtener la siguiente página sin realizar `skip` costosos.

**P:** ¿Cuándo usar WebSockets en vez de HTTP tradicional?

**R:** Usa WebSockets cuando la aplicación requiere comunicación bidireccional y en tiempo real con baja latencia, como en chats, juegos, actualizaciones en vivo o paneles en tiempo real. HTTP tradicional (request/response) es suficiente y más simple para la mayoría de operaciones CRUD y APIs REST donde el cliente inicia la interacción.

Si eliges WebSockets, considera usar bibliotecas como `Socket.IO` que manejan reconexiones, compatibilidad con navegadores y fallbacks. También evalúa el coste de mantener conexiones persistentes en términos de recursos y el diseño para escalar con múltiples instancias (por ejemplo usando un broker como Redis para eventos entre servidores).

**P:** ¿Cómo protejo las cadenas de conexión a MongoDB en producción?

**R:** Nunca almacenes cadenas de conexión ni credenciales en el repositorio de código. Usa variables de entorno y valida su existencia al iniciar la aplicación. En entornos de nube utiliza servicios de gestión de secretos o vaults (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault) que permiten rotación y políticas de acceso.

Además, configura la conexión a MongoDB con TLS/SSL, restringe IPs o redes que pueden conectarse y usa roles y usuarios con privilegios mínimos necesarios. Para despliegues en contenedores, inyecta secretos por el orquestador (Kubernetes secrets, por ejemplo) y audita accesos regularmente.

**P:** ¿Cómo manejo cambios de esquema o migraciones en MongoDB?

**R:** Aunque MongoDB es schemaless, en proyectos productivos es necesario aplicar migraciones cuando cambian estructuras lógicas. Utiliza herramientas de migración (por ejemplo `migrate-mongo` o soluciones propias) que registren el estado y permitan aplicar/upgradear scripts en forma controlada. Diseña migraciones idempotentes y prueba en entornos staging antes de ejecutar en producción.

Otra estrategia es aplicar cambios en forma incremental: primero extender el esquema para soportar ambos formatos, desplegar código que maneje la coexistencia, y posteriormente realizar una migración de datos y limpieza. Esto minimiza downtime y reduce riesgos durante despliegues en producción.

## Conclusiones

### Habilidades clave adquiridas

Has consolidado capacidades prácticas que te permiten diseñar y construir backends avanzados con Node.js, Express y MongoDB. Ahora comprendes cómo estructurar proyectos, manejar asincronía con `async/await`, modelar datos con Mongoose y optimizar consultas mediante índices y `explain()`. Estas habilidades te capacitan para implementar APIs robustas, manejar operaciones en tiempo real con WebSockets y aplicar patrones de seguridad como manejo de secretos y conexión TLS.

### Aplicación inmediata

Podrás aplicar estas competencias en proyectos reales creando APIs escalables, integrando autenticación, optimizando consultas y desplegando conexiones seguras a bases de datos gestionadas. La práctica con estas herramientas incrementa tu productividad y te prepara para enfrentar desafíos comunes en producción.

### Relevancia profesional y oportunidades

La combinación de conocimientos en Node.js, Express y MongoDB es altamente demandada en el mercado para roles de backend y fullstack. Saber diseñar APIs bien estructuradas, optimizar



rendimiento y manejar seguridad te diferencia como profesional capaz de llevar proyectos desde prototipos hasta producción. Además, el dominio de herramientas como `Socket.IO`, `Mongoose` y prácticas de deployment te abre puertas en startups, empresas medianas y proyectos escalables.

## Crecimiento en el puesto

Con estas bases puedes avanzar hacia arquitecturas más complejas: microservicios, despliegue en contenedores, integración continua y soluciones serverless. La experiencia práctica con casos reales y métricas de rendimiento será clave para escalar tu perfil profesional.

## Siguientes pasos y aprendizaje continuo

Continúa consolidando lo aprendido mediante proyectos reales y contribuciones a código abierto. Aprende a monitorear y perfilar aplicaciones en producción, profundiza en patrones de escalado (sharding, réplica) y explora herramientas complementarias como Redis para caché y colas, o herramientas de observabilidad como Prometheus y ELK. Mantén una práctica de pruebas automatizadas y CI/CD para reducir riesgos en despliegues.

Cierre: la tecnología evoluciona constantemente; combinar fundamentos sólidos con experiencia práctica y aprendizaje continuo te permitirá adaptarte y liderar soluciones backend eficientes y seguras.

---